Fully succinct noninteractive oblivious tensor evaluation from LWE

Damiano Abram, Giulio Malavolta, Lawrence Roy

arxiv:2508.09673v3; CC BY 4.0

Editorial note. Only fully-succinct NI-OTE theorem counts. Full author body, all proofs, drawings and bibliography retained. Publisher metadata adapted to article/natbib; native unchanged.

Allowed standard prerequisites (editorial).

- Standard graph algorithms and ETH cited by authors. All source-local proofs and necessary context retained.

Abstract

We propose the notion of succinct oblivious tensor evaluation (OTE), where two parties compute an additive secret sharing of a tensor product of two vectors $\mathbf{x} \otimes \mathbf{y}$, exchanging two simultaneous messages. Crucially, the size of both messages and of the CRS is independent of the dimension of $\mathbf{x}$. We present a construction of OTE with optimal complexity from the standard learning with errors (LWE) problem. Then we show how this new technical tool enables a host of cryptographic primitives, all with security reducible to LWE, such as:

- Adaptively secure laconic function evaluation for depth-D functions $f : \{0,1\}^m \to \{0,1\}^\ell$ with communication $m + \ell + D \cdot \mathsf{poly}(\lambda)$.
- A trapdoor hash function for all functions.
- An (optimally) succinct homomorphic secret sharing for all functions.
- A rate-1/2 laconic oblivious transfer for batch messages, which is best possible.

In particular, we obtain the first laconic function evaluation scheme that is adaptively secure from the standard LWE assumption, improving upon Quach, Wee, and Wichs (FOCS 2018). As a key technical ingredient, we introduce a new notion of *adaptive lattice encodings*, which may be of independent interest.

1 Introduction

Consider the scenario where Alice holds a *long* vector $\mathbf{x}$, Bob holds a *smaller* secret vector $\mathbf{y}$ and, after a single round of simultaneous messages, they should be able to locally compute an additive secret share of the tensor product $\mathbf{x} \otimes \mathbf{y}$ while preserving the privacy of $\mathbf{y}$. That is, after one simultaneous round of messages, Alice computes α and Bob computes β such that:

$$\alpha + \beta = \mathbf{x} \otimes \mathbf{y}.$$

We refer to this problem as *non-interactive oblivious tensor evaluation* (NI-OTE). In this work, we are interested in the communication complexity of secure NI-OTE, i.e., the minimum size of the messages needed in order to compute a correct additive secret sharing, while preserving the privacy of $\mathbf{y}$. While one may intuitively expect that Alice and Bob's messages should be long enough to fully specify both the vectors, this is in fact not so. Counterintuitively, we show that it is possible to complete the above protocol with communication complexity *logarithmic* in the dimensions of the input $\mathbf{x}$.

The objective of this work is to construct explicit protocols for NI-OTE with minimal communication, and to explore the cryptographic consequences of this primitive.

1.1 Our Results

Our main technical contribution is a protocol for NI-OTE with minimal communication complexity, where the security is proven against the standard learning with errors (LWE) assumption [22]. We prove this result in two steps: First, we construct an elementary (half-succinct) NI-OTE protocol where only the message of one party is short, whereas the message of the other party can depend arbitrarily on $|\mathbf{x}|$. Then we show a generic *bootstrapping* procedure that makes the scheme fully succinct, i.e., the messages of both parties are short. Overall, our main result is captured by the following informal theorem statement (treating the security parameter as constant).

Theorem 1 (Informal)**.** *If the LWE problem is hard, then there exists an NI-OTE protocol for $\mathbf{x} \in \mathbb{Z}_q^m$ and $\mathbf{y} \in \mathbb{Z}_q^\ell$ with communication complexity $\ell \cdot \mathsf{poly}(\lambda) + \mathsf{poly}(\lambda, \log m)$ and CRS of size $\mathsf{poly}(\lambda, \log m)$.*

Notice that we consider the problem in its strongest form, where Alice and Bob send each other a single, simultaneous message. However, outside the realm of obfuscation, it was unknown how to construct OTE with the same parameters, even if interaction is allowed. Besides being a primitive of independent interest, we show that the existence of our succinct NI-OTE protocol has surprising applications in cryptography.

Application I: Trapdoor Hash Functions. For starters, we show how succinct NI-OTE, combined with recent results in laconic function evaluation

[16, 19, 21], enables a construction of a trapdoor hash (TDH) function [17] for all functions, or even RAM programs, from Ring-LWE.

In a TDH, Alice holds a function f and Bob holds an input x. They simultaneously exchange a message depending only on their own input and, in a second stage, they can locally compute an additive secret sharing of $f(x)$. Importantly, one wants Alice's message to be short. In our TDH, the size of the hash is constant, and the size of the encoding key depends only on the Turing machine description of the function f, which is optimal. To the best of our knowledge, prior constructions of TDHs [17, 23] only supported linear functions. This result is summarized by the following (informal) theorem statement.

Theorem 2 (Informal)**.** *If the LWE problem is hard, then there exists a TDH for functions with depth D, where the size of the encoding is bounded by $|f| \cdot \mathsf{poly}(\lambda, D)$. Additionally, assuming the hardness of* circular *LWE, we obtain a bound on the size of the encodings of $|f| \cdot \mathsf{poly}(\lambda)$.*

Application II: Succint Homomorphic Secret Sharing and More. As a direct consequence of the above result, we obtain a new protocol homomorphic secret sharing (HSS) scheme [8]. In this work we consider the (stronger) public-key variant of the primitive: Suppose that Alice and Bob respectively hold inputs $\mathbf{x}$ and $\mathbf{y}$ and they want to evaluate the function f. Alice and Bob send each other (simultaneously) a single message, dependent on their input. Based on some local computation, Alice and Bob obtain an additive secret sharing of $f(\mathbf{x}, \mathbf{y})$.

In these settings, we obtain a *succinct* [3] protocol for all functions, where the communication complexity is logarithmic in Alice's input $\mathbf{x}$, which is optimal. The only prior work on succinct HSS [3] only supported NC_1 circuits and had communication complexity proportional to $|\mathbf{x}|^{\varepsilon}$, for some $\varepsilon \in O(1)$. In addition, we obtain a new *batched* laconic oblivious transfer protocol [14], with constant-size receiver's message and with rate 1/2, which is best possible.

We refer the reader to Section 1.3 for a more detailed and precise discussion on these primitives, along with additional applications of succinct NI-OTE such as spooky encryption, and pseudorandom correlation generators.

Application III: Laconic Function Evaluation. Finally, we show how to leverage succinct NI-OTE to construct a new laconic function evaluation (LFE) [21] protocol. An LFE is a two-party protocol where Alice holds a function f and Bob holds an input x. Alice sends a *hash* of her function f to Bob, who computes an encoding of his input, depending on the hash sent by Alice. From Bob's message, Alice can recover $f(x)$ but nothing more. We propose the first LFE protocol that is simultaneously:

- *Adaptively secure*: The attacker can choose the input adaptively, possibly depending on the public parameters.
- *Rate-1*: The size of the encoding equals the size of the input, plus the size of the output, plus an additive factor.

Prior to our work, even constructing LFE with either of the two properties was considered an open problem. The question of adaptively-secure LFE from LWE was raised in [21], where they proposed a construction provable against the *adaptive* LWE assumption, whereas our construction is adaptively secure against the *standard* LWE assumption. This is not a cosmetic improvement: We also show a counterexample against the adaptive LWE assumption. This translates into an adaptive attack against [21], underscoring the need for constructions proven adaptively secure against standard assumptions.

The question of rate-1 LFE was considered in [24], where they proposed a construction from ℓ-succinct LWE, a recently-introduced variant of the LWE assumption. We improve upon this work by relying only on the standard LWE problem. Overall, our results can be summarized as follows:

Theorem 3 (Informal)**.** *If the LWE problem is hard, there exists an adaptively secure LFE for depth-D functions $f : \{0,1\}^m \to \{0,1\}^\ell$ with communication $m + \ell + D \cdot \mathsf{poly}(\lambda)$.*

We shall mention here that, allowing additional interaction beyond two rounds, such a result can also be achieved using rate-1 fully-homomorphic encryption [13], so the difficulty of the problem is to construct such primitive in two rounds, which matches the standard LFE syntax. On the other hand, for all other applications, the results are new even if we allow arbitrary interaction.

As a key technical ingredient, we present a new variant of *homomorphic lattice encodings* [7] that naturally supports adaptive security. This is the first construction of homomorphic lattice encodings that departs from the framework of [7], and we expect it to find other applications in the future.

A diagram summarizing our results is given in Figure 1.

1.2 Technical Outline

From now on, we call Alice *the hasher* and Bob *the encoder*. We start by presenting a half-succinct OTE, i.e., an OTE protocol where only the hasher's message is succinct in its input size. This is inspired by the work of [3] and we extend their ideas in the context of tensor products. Suppose that we work over $\mathbb{Z}_q$, the hasher's input is a vector over $\mathbb{Z}_2^m$, while the encoder's input lies in $\mathbb{Z}_q^\ell$.

The construction relies on a setup that outputs a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ where $m > n$. To hash $\mathbf{x}$, we simply compute an SIS-based hash $\mathbf{d} \leftarrow \mathbf{A} \cdot \mathbf{x}$. To encode $\mathbf{y}$, on the other hand, we compute

$$\mathbf{C} \leftarrow \mathbf{A}^\intercal \cdot \mathbf{S} + \mathbf{E} + \mathbf{I}_m \otimes \mathbf{y}^\intercal$$

where $\mathbf{S} \xleftarrow{\$} \mathbb{Z}_q^{n \times (m \cdot \ell)}$ and $\mathbf{E} \xleftarrow{\$} \chi(1^\lambda)$. Above, we use $\mathbf{I}_m$ to denote the $m \times m$ identity matrix and χ to denote a low-norm distribution. Notice that, under LWE, $\mathbf{C}$ leaks no information about $\mathbf{y}$. Suppose that Alice and Bob exchanged $\mathbf{C}$ and $\mathbf{d}$. Alice can compute a "noisy" share of $\mathbf{x} \otimes \mathbf{y}$ by computing

$$\begin{aligned}\mathbf{v} := \mathbf{C}^\intercal \cdot \mathbf{x} &= \mathbf{S}^\intercal \cdot \mathbf{A} \cdot \mathbf{x} + \mathbf{E}^\intercal \cdot \mathbf{x} + (\mathbf{I}_m \otimes \mathbf{y}) \cdot \mathbf{x} \\ &= \mathbf{S}^\intercal \cdot \mathbf{A} \cdot \mathbf{x} + \mathbf{E}^\intercal \cdot \mathbf{x} + (\mathbf{x} \otimes \mathbf{y}).\end{aligned}$$

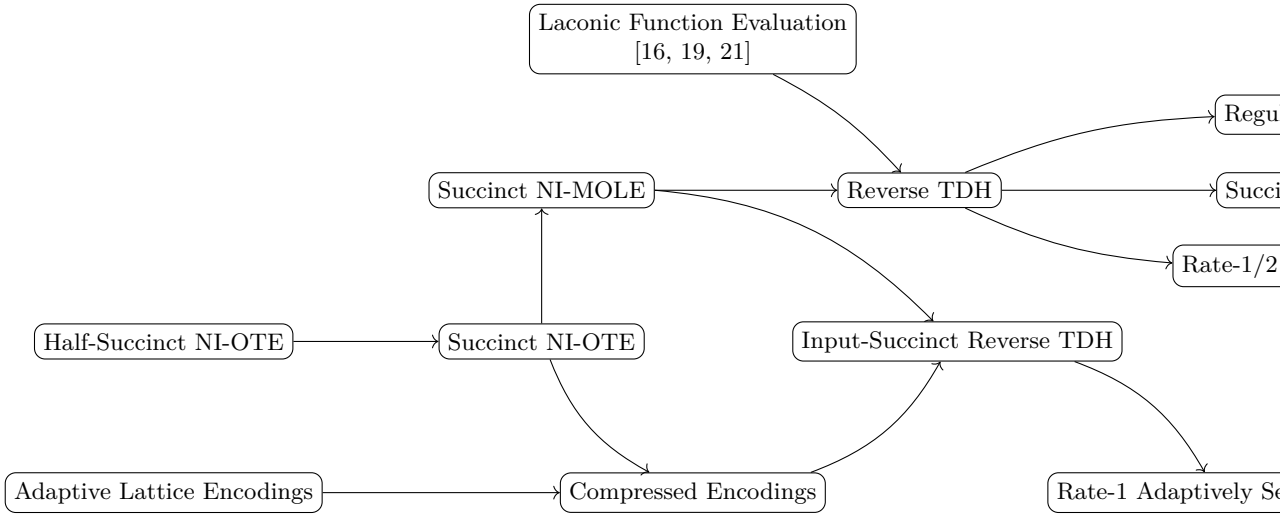


Figure 1: A schematic representation of our results, and a summary of the implications to different cryptographic primitives.

Bob can derive instead his own "noisy" share by computing

$$\mathbf{w} := -\mathbf{S}^\intercal \cdot \mathbf{d} = -\mathbf{S}^\intercal \cdot \mathbf{A} \cdot \mathbf{x}.$$

It is easy to see that $\mathbf{v} + \mathbf{w} = \mathbf{x} \otimes \mathbf{y} + \mathbf{E}^\intercal \cdot \mathbf{x}$. Moreover, since $\mathbf{x} \in \mathbb{Z}_2^m$, the magnitude of $\mathbf{E}^\intercal \cdot \mathbf{x}$ is small. From this we can derive a fully correct, half succinct OTE over $\mathbb{Z}_p$ where p is a sufficiently small divisor of q: Instead of hashing $\mathbf{x}$, hash its bit decomposition, instead of encoding $\mathbf{y}$, encode $q/p \cdot \mathbf{y} \otimes \mathbf{g}_q$ where $\mathbf{g}_q^\intercal$ is the gadget (row) vector $(1, 2, \ldots, 2^{\log q})$. To reconstruct an exact secret-sharing of $\mathbf{x} \otimes \mathbf{y}$ mod p, it is sufficient to apply a linear operation on the shares and round the result over $\mathbb{Z}_p$ following the ideas of [15] (see 10).

An Attempt at Bootstrapping. We now describe a first attempt to obtain *full* succinctness. In the half-succinct construction above, the encoder's message has size $m\ell$, whereas the hash size is independent of both m and ℓ. Thus the remaining bottleneck is Bob's message. Following [3], we exploit two simple features of the protocol: It is non-interactive, and the functionality is linear. Suppose that Alice's input is much longer, say $\mathbf{x} \in \mathbb{Z}_q^M$ with $M \gg m, \ell$. Write

$$\mathbf{x} = \begin{pmatrix} \mathbf{x}_0 \\ \vdots \\ \mathbf{x}_{N-1} \end{pmatrix}, \qquad \mathbf{x}_i \in \mathbb{Z}_q^m,$$

where $N = M/m$. Alice hashes each block separately, obtaining digests

$$\mathbf{d}_0, \ldots, \mathbf{d}_{N-1}.$$

Bob, on the other hand, sends a *single* encoding of $\mathbf{y} \in \mathbb{Z}_q^\ell$. Since the same encoding can be evaluated against every digest, the parties obtain an additive secret-sharing of $\mathbf{x}_i \otimes \mathbf{y}$ for every $i \in [N]$. Concatenating these shares yields an additive secret-sharing of $\mathbf{x} \otimes \mathbf{y}$.

This already makes Bob's communication sublinear in M: His message depends only on m and ℓ, not on the full length of $\mathbf{x}$. The drawback is that Alice must now send all N digests, so her communication grows by a factor of M/m. In other words, we have traded a large encoding for many digests. The next natural question is whether these N digests can themselves be compressed. Towards this, let us take a closer look at Bob's shares in the noisy half-succinct scheme: For each block i, Bob's share can be obtained from $\mathbf{d}_i \otimes \mathsf{vec}(\mathbf{S})$[1] by applying a public linear map. Therefore, if Alice and Bob could obtain an additive secret-sharing of

$$\mathbf{d}_i \otimes \mathsf{vec}(\mathbf{S}) \qquad \text{for every } i \in [N],$$

then they could recover Bob's noisy shares for all blocks using only local linear operations. Combining these with Alice's local shares would give a noisy secret-sharing of $\mathbf{x} \otimes \mathbf{y}$, which could then be rounded exactly as above.

This suggests a recursive strategy. Let

$$\mathbf{d}' := \begin{pmatrix} \mathbf{d}_0 \\ \vdots \\ \mathbf{d}_{N-1}) \end{pmatrix}$$

be the concatenation of all block digests, we apply our half-succinct OTE *again*, this time to obtain a secret-sharing of $\mathbf{d}' \otimes \mathsf{vec}(\mathbf{S})$. If this worked, then Alice would no longer need to send the entire vector $\mathbf{d}'$, but rather its hash. In fact, one could imagine iterating the same idea and, pushing this recursion to its limit, one might hope for a protocol in which Alice sends only a single short commitment to $\mathbf{x}$, while Bob sends one encoding per recursion level.

Unfortunately, this naive recursion fails because the second input grows too quickly. After the first step, Bob is no longer encoding $\mathbf{y}$, but rather $\mathsf{vec}(\mathbf{S})$. At the next level, he must encode the randomness used to encode $\mathsf{vec}(\mathbf{S})$, and so on. Since each recursion step increases the dimension of the randomness by a factor of about $n \cdot m$, Bob's messages grow by the same factor at every level. Thus the recursion indeed compresses Alice's side, but it causes Bob's side to blow up. To make the approach work, we need a way to keep the randomness small while preserving the linear structure that lets Bob's share be recovered from $\mathbf{d}_i \otimes \mathsf{vec}(\mathbf{S})$.

Decreasing the size of S. Instead of sampling a random $\mathbf{S}$, we generate a pseudorandom one using LWE. Specifically, include $m \cdot \ell$ random $n \times n$ matrices $\mathbf{B}_0, \ldots, \mathbf{B}_{\ell \cdot m - 1}$ as part of the setup. Then, at encoding time, we sample a random vector $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$ and we set the i-th row of $\mathbf{S}$ to be $\mathbf{B}_i \cdot \mathbf{s} + \mathbf{e}_i$ where

[1] $\mathsf{vec}(\mathbf{S})$ denotes the vectorisation of $\mathbf{S}$.

$\mathbf{e}_i \xleftarrow{\$} \chi(1^\lambda)$. In matrix notation, we obtain that $\mathbf{S} = \mathbf{B} \cdot (\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}) + \hat{\mathbf{E}}$, i.e.,

$$\mathbf{S} = \underbrace{\begin{pmatrix}\mathbf{B}_0 & \dots & \mathbf{B}_{\ell\cdot m-1}\end{pmatrix}}_{\mathbf{B}} \cdot \underbrace{\begin{pmatrix}\mathbf{s} & & \\ & \ddots & \\ & & \mathbf{s}\end{pmatrix}}_{m\cdot\ell \text{ times}} + \underbrace{\begin{pmatrix}\mathbf{e}_0 & \dots & \mathbf{e}_{\ell\cdot m-1}\end{pmatrix}}_{\hat{\mathbf{E}}}.$$

The encoding of $\mathbf{y}$ becomes

$$\mathbf{C} = \mathbf{A}^\intercal \cdot \mathbf{B} \cdot (\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}) + \mathbf{A}^\intercal \cdot \hat{\mathbf{E}} + \mathbf{E} + \mathbf{I}_m \otimes \mathbf{y}^\intercal.$$

We also introduce another modification: Instead of sampling the entries of $\mathbf{A}$ and $\mathbf{B}$ uniformly at random over $\mathbb{Z}_q$, we sample them uniformly over $\mathbb{Z}_2$. This trick ensures that the magnitude of $\mathbf{A}^\intercal \cdot \hat{\mathbf{E}}$ remains small, while, at the same time, it does not compromise security: LWE with respect to random *binary* matrices is known to be as hard as standard LWE [6]. With these modifications to our half-succinct OTE, Alice's share becomes

$$\begin{aligned}\mathbf{v} &:= \mathbf{C}^\intercal \cdot \mathbf{x} \\ &= (\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}^\intercal) \cdot \mathbf{B}^\intercal \cdot \mathbf{A} \cdot \mathbf{x} + \hat{\mathbf{E}}^\intercal \cdot \mathbf{A} \cdot \mathbf{x} + \mathbf{E}^\intercal \cdot \mathbf{x} + (\mathbf{I}_m \otimes \mathbf{y}) \cdot \mathbf{x} \\ &= (\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}^\intercal) \cdot \mathbf{B}^\intercal \cdot \mathbf{A} \cdot \mathbf{x} + \hat{\mathbf{E}}^\intercal \cdot \mathbf{A} \cdot \mathbf{x} + \mathbf{E}^\intercal \cdot \mathbf{x} + \mathbf{x} \otimes \mathbf{y}.\end{aligned}$$

Bob's share becomes instead

$$\mathbf{w} := -(\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}^\intercal) \cdot \mathbf{B}^\intercal \cdot \mathbf{d} = -(\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}^\intercal) \cdot \mathbf{B}^\intercal \cdot \mathbf{A} \cdot \mathbf{x}.$$

This allows us to apply recursion without blowing up the size of $\mathbf{s}$: Since $\mathbf{w}$ is a bilinear function of $\mathbf{d}$ and $\mathbf{s}$, if the parties hold a "noisy" secret-sharing of $\mathbf{d}' \otimes \mathbf{s}$, they can easily convert it into a "noisy" secret-sharing of Bob's share by computing a local linear operation depending on $\mathbf{B}$. Moreover, since $\mathbf{B}$ is a binary matrix, this linear computation will not significantly increase the "noisiness" of the secret-sharing. At each recursion step, the size of $\mathbf{d}'$ decreases by a factor of $t(\lambda) := m/n$; the size of $\mathbf{s}$, on the other hand, remains always the same. The final result is an LWE-based succinct OTE where the digest and the CRS dimensions are $\mathsf{poly}(\lambda)$ and the encoding dimension is $O(\log M) \cdot \ell \cdot \mathsf{poly}(\lambda)$.

Succinct MOLEs and VOLEs. No, this paragraph is not about tiny mammals: It's about two cryptographic primitives called *matrix oblivious linear evaluation* and (non-interactive) *vector oblivious linear evaluation*. In a MOLE, Alice holds a matrix $\mathbf{M} \in \mathbb{Z}_q^{m\times\ell}$ where $m \gg \ell$, whereas Bob holds a secret vector $\mathbf{x} \in \mathbb{Z}_q^\ell$. Their goal is to derive a secret-sharing of $\mathbf{M}\cdot\mathbf{x}$ in one round and without revealing any information about $\mathbf{x}$. A VOLE corresponds to a MOLE in the special case $\ell = 1$.

We would like to minimize the communication complexity of these protocols, especially in relation to m. It is easy to see that the succinct OTE protocol we just presented gives immediately a succinct MOLE where communication scales

logarithmically in m: First, we use the succinct OTE protocol to compute a secret-sharing of $\mathsf{vec}(\mathbf{M}) \otimes \mathbf{x}$, then, we apply local linear operations on the shares to obtain a secret-sharing of $\mathbf{M} \cdot \mathbf{x}$. Along the way, this solves a question left open in [3]: We have just built the first non-interactive VOLE with logarithmic communication in m from LWE.

Reverse Trapdoor Hashing. We observe that many laconic function evaluation schemes have a particular structure [16, 19, 21, 24]. First of all, their digests consist of matrices $\mathbf{A}_f$ with a constant number of columns and a number of rows proportional to the output size of f. Moreover, the encoding can be split into two parts: A function-independent pre-encoding E and an input-independent post-encoding c consisting of an LWE-like sample where the matrix is (essentially) the digest $\mathbf{A}_f$ and the secret $\mathbf{s}$ is a random vector generated by the pre-encoding procedure. Finally, the output is computed by rounding the sum $c + \mathsf{Eval}(E, f)$.

We observe that, if we ignore the noise, LWE samples are essentially matrix-vector multiplications. Therefore, by relying on the succinct MOLE we just built, the parties can derive a noisy secret-sharing of the post-encoding c in a single round of simultaneous interaction and with logarithmic communication in the output size of f. Moreover, the encoder can send the pre-encoding E along with its MOLE message. In this way, the parties can derive a secret-sharing of the output without the need for further interaction.

This yields the first rate-1 *reverse* trapdoor hashing scheme. Usually, in rate-1 trapdoor hashing, we obtain a secret-sharing of $f(x)$ by sending a digest of x and generating an encoding key for f. In reverse trapdoor hashing, we do the opposite: We send a digest for f and an encoding key for x, so rate in this context is defined as a function of $|f|$. In our construction, the former corresponds to the MOLE hash of $\mathbf{A}_f$, whereas the latter corresponds to the pre-encoding E and the MOLE encoding of $\mathbf{s}$.[2]

Adaptive Lattice Encodings. Much of the recent advancement in lattice-based homomorphic cryptography can be traced back to a single technique introduced in 2014 by Boneh et al. [7]: BGG^+ encodings. Suppose that we are provided with a CRS consisting of a matrix $\mathbf{A} \in \mathbb{Z}_q^{k \times (\ell \cdot k \cdot \log q)}$. A BGG^+ encoding of a bit string $\mathbf{x} \in \{0,1\}^\ell$ consists of the vector

$$\mathbf{c}^\intercal = \mathbf{s}^\intercal \cdot (\mathbf{A} - \mathbf{x}^\intercal \otimes \mathbf{G}) + \mathbf{e}^\intercal$$

where $\mathbf{G} := \mathbf{I}_k \otimes \mathbf{g}_q^\intercal$, $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$ and $\mathbf{e} \xleftarrow{\$} \chi(1^\lambda)$. These encodings have amazing homomorphic properties: For any function $f : \{0,1\}^\ell \to \{0,1\}$, there exist efficiently computable, low-norm matrices $\mathbf{H}_f$ and $\mathbf{H}_{f,x}$ (independent of the random $\mathbf{s}$ and the noise $\mathbf{e}$ of the encoding) such that

$$\mathbf{c}^\intercal \cdot \mathbf{H}_{f,x} \approx \mathbf{s}^\intercal \cdot (\mathbf{A}_f - f(x) \cdot \mathbf{G})$$

[2] We mention that we can recover the usual notion of trapdoor hashing via universal circuits. On the other hand, the reverse implication does not seem to trivially hold, since the universal circuit would introduce an efficiency penalty in the size of the encoding.

where $\mathbf{A}_f := \mathbf{A} \cdot \mathbf{H}_f$. Alas, BGG^+ encodings are secure only in the selective setting: If $\mathbf{x}$ is independent of $\mathbf{A}$, the encoding $\mathbf{c}$ looks like a random vector, if, however, $\mathbf{x}$ is adaptively chosen after seeing $\mathbf{A}$, there exists an attack that allows us to recover $\mathbf{s}$. To see why, observe that there exists a binary matrix $\mathbf{H}'$ such that, for any matrix $\mathbf{M} \in \mathbb{Z}_q^{k \times k}$, if $\mathbf{x} = \mathsf{Bits}(\mathbf{M})$, we have that

$$\mathbf{c}^\intercal \cdot \mathbf{H}' \approx \mathbf{s}^\intercal \cdot (\mathbf{A} \cdot \mathbf{H}' - \mathbf{M}).$$

Suppose for convenience that q is a power of 2. To recover the i-th most significant bit of $\mathbf{s}$, it is sufficient to set $\mathbf{M} := \mathbf{A} \cdot \mathbf{H}' - 2^i \cdot \mathbf{I}_k$ and compute the most significant bit of $\mathbf{c}^\intercal \cdot \mathbf{H}'$. This proves that the Adaptive LWE assumption of [21] does not hold in the optimistic parameter setting in which ℓ can be arbitrarily bigger than k. Notice in the provable parameter setting (where the encodings are secure under the *subexponential* hardness of LWE), our attack fails as the bound on ℓ is too small to encode $\mathbf{M} := \mathbf{A} \cdot \mathbf{H}' - 2^i \cdot \mathbf{I}_k$.

To circumvent the attack without relying on complexity leveraging (and therefore obtain better asymptotic parameters), we introduce a new version of lattice encodings: The encoding of $\mathbf{x}$ is now

$$\mathbf{c}^\intercal = \mathbf{s}^\intercal \cdot \mathbf{A} + \mathbf{r}^\intercal \cdot (\mathbf{x}^\intercal \otimes \mathbf{G}) + \mathbf{e}^\intercal$$

where $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$, $\mathbf{r} \xleftarrow{\$} \mathbb{Z}_q^k$ and $\mathbf{e} \xleftarrow{\$} \chi(1^\lambda)$. We call $\mathbf{s}$ *the encryption key* of the encoding, whereas we refer to $\mathbf{r}$ as *the authentication key*. We say that $\mathbf{c}$ is a $(\mathbf{s}, \mathbf{r})$-encoding. Observe that BGG^+ encodings correspond to the special case in which $\mathbf{r} = -\mathbf{s}$. It is easy to see also that these encodings are adaptively secure under standard LWE, no matter the value of $\mathbf{x}$ and $\mathbf{r}$.

The drawback, however, becomes clear when we look at the homomorphic properties of the modified scheme. It is easy to see that the construction is linearly homomorphic, however, we no longer know how to perform multiplications. Or at least, we do not know *when the factors are encoded using the same keys*. If instead the authentication key of the first encoding matches the encryption key of the second one, there is a solution: Suppose that we want to multiply the encodings $\mathbf{c}_x^\intercal = \mathbf{s}^\intercal \cdot \mathbf{A} + \mathbf{r}^\intercal \cdot (x \cdot \mathbf{G}) + \mathbf{e}_x^\intercal$ and $\mathbf{c}_y^\intercal = \mathbf{r}^\intercal \cdot \mathbf{B} + \mathbf{t}^\intercal \cdot (y \cdot \mathbf{G}) + \mathbf{e}_y^\intercal$. We observe that

$$\mathbf{c}_z^\intercal := -\mathbf{c}_x^\intercal \cdot \mathbf{G}^{-1}(\mathbf{B}) + x \cdot \mathbf{c}_y^\intercal \approx -\mathbf{s}^\intercal \cdot \mathbf{A}\mathbf{G}^{-1}(\mathbf{B}) + \mathbf{t}^\intercal \cdot (xy \cdot \mathbf{G}).$$

In other words, we have obtained an encoding of $x \cdot y$ using $\mathbf{s}$ as encryption key and $\mathbf{t}$ as authentication key. Moreover, the matrix relative to the encoding is $-\mathbf{A} \cdot \mathbf{G}^{-1}(\mathbf{B})$. Notice that this is publicly computable, no need to know the keys or the plaintexts! To summarise, we are able to compute linear operations between $(\mathbf{s}, \mathbf{r})$-encodings obtaining other $(\mathbf{s}, \mathbf{r})$-encodings. Moreover, we are able to perform multiplications between $(\mathbf{s}, \mathbf{r})$-encodings and $(\mathbf{r}, \mathbf{t})$-encodings, obtaining $(\mathbf{s}, \mathbf{t})$-encodings as results.

Compressing Adaptive Lattice Encodings. We present a procedure to compress our adaptive lattice encodings, where a compressed encoding of $\mathbf{x} \in$

$\{0,1\}^\ell$ will have size $\mathsf{poly}(\log \ell, \lambda)$. By leveraging the knowledge of $\mathbf{x}$, it can then be re-expanded into a standard adaptive encoding. Once again, our techniques rely on our succinct OTE protocol and the (bi)linear structure of Bob's share derivation. Specifically, the compressed encoding is composed of two parts $\mathbf{h}$ and E. The former consists of an adaptive lattice encoding of $\mathbf{d} := \mathsf{OTE.Hash}(\mathbf{x})$. Let $\mathbf{r}$ be the authentication key of $\mathbf{h}$; then E consists of an OTE encoding of a fresh authentication key $\mathbf{t}$ where $\mathbf{r}$ is used as randomness for E (this is secure as $\mathbf{h}$ looks random even when $\mathbf{r}$ is leaked). We observe that

$$\begin{aligned}\mathbf{h}^\intercal &\approx \mathbf{s}^\intercal \cdot \mathbf{A} + \mathbf{r}^\intercal \cdot (\mathbf{d}^\intercal \otimes \mathbf{G}) \\ &= \mathbf{s}^\intercal \cdot \mathbf{A} + \mathbf{r}^\intercal \cdot (\mathbf{d}^\intercal \otimes \mathbf{I}_k \otimes \mathbf{g}_q^\intercal) \\ &= \mathbf{s}^\intercal \cdot \mathbf{A} + (\mathbf{d}^\intercal \otimes \mathbf{r}^\intercal \otimes \mathbf{g}_q^\intercal).\end{aligned}$$

In other words, $\mathbf{h}$ can be viewed as some sort of encoding of $\mathbf{d} \otimes \mathbf{r}$ where $\mathbf{r}$ is the randomness of E. Now, Bob's share of $\mathbf{x} \otimes \mathbf{t}$ would be $\mathbf{w} := \mathbf{P} \cdot (\mathbf{d} \otimes \mathbf{r})$, where $\mathbf{P}$ is a public low-norm matrix derived from $\mathbf{B}$. Thus, by multiplying $\mathbf{h}$ on the right by $\mathbf{P}$, we derive

$$\hat{\mathbf{h}}^\intercal := \mathbf{h}^\intercal \cdot \mathbf{P} \approx \mathbf{s}^\intercal \cdot \mathbf{AP} + \mathbf{w}^\intercal \otimes \mathbf{g}_q^\intercal.$$

Using $\mathbf{x}$ and E, we can also derive the other share $\mathbf{v}$. We conclude by observing that

$$\begin{aligned}\hat{\mathbf{h}}^\intercal + \mathbf{v}^\intercal \otimes \mathbf{g}_q^\intercal &\approx \mathbf{s}^\intercal \cdot \mathbf{AP} + (\mathbf{x}^\intercal \otimes \mathbf{t}^\intercal \otimes \mathbf{g}_q^\intercal) \\ &= \mathbf{s}^\intercal \cdot \mathbf{AP} + \mathbf{t}^\intercal \cdot (\mathbf{x}^\intercal \otimes \mathbf{G})\end{aligned}$$

We have just obtained a $(\mathbf{s}, \mathbf{t})$-encoding of $\mathbf{x}$.

Rate-1 Adaptive LFE. We build our rate-1 adaptive LFE scheme using our compressed adaptive encodings in two steps: First, we construct a weak variant of (adaptive) reverse TDH for the functions that map pairs $(\mathbf{x}, \mathbf{a})$ to $f(\mathbf{x}) \otimes \mathbf{a}$ where $f \in \mathsf{NC}_1$. The scheme satisfies all the security properties of standard reverse TDH, however, in order for correctness to hold, the hasher needs to know $\mathbf{x}$ (but not $\mathbf{a}$) for the derivation of her share. However, importantly, the scheme additionally achieves polylogarithmic communication in the size of $\mathbf{x}$ (but not $\mathbf{a}$). As a second step, we use our *input-succinct* reverse TDH to build an adaptive LFE scheme for all depth-D functions $f : \{0,1\}^m \to \{0,1\}^\ell$. The size of the hash will be $\mathsf{poly}(\lambda)$ whereas the size of the encoding will be $\ell + m + D \cdot \mathsf{poly}(\lambda)$.

Step I: Input-Succinct Reverse TDH. We start by describing the input-succinct reverse TDH. Suppose that we want to evaluate functions of depth at most $\log d$: Each of them can be converted into an RMS program of depth d [8]. We recall that an RMS program consists of an arithmetic circuit over $\mathbb{Z}$ where multiplications are allowed only if one of the factors is an input.

We start by describing the generation of encoding keys: We sample a chain of d keys $\mathsf{s}_0, \ldots, \mathsf{s}_{d-1}$, we set $\mathbf{s}_d := \mathbf{a}$ and we generate compressed encodings $(\mathbf{h}_i, E_i)_{i\in[d]}$ so that, when we expand $(\mathbf{h}_i, E_i)$, we obtain a $(\mathbf{s}_i, \mathbf{s}_{i+1})$-encoding of $(\mathbf{x}, 1)$. Notice that we can homomorphically evaluate f on these encodings: The operation proceeds by levels, starting from level 1 (the inputs) to level d (the output). A level-i encoding consists of any $(\mathbf{s}_0, \mathbf{s}_i)$-encoding. We observe that $(\mathbf{h}_0, E_0)$ gives us a level-1 encoding of the inputs.

In the previous paragraphs, we showed that the levels are closed under linear operations. Moreover, we can also perform multiplications by inputs: If the first factor belongs to level i, we can multiply it by the $(\mathbf{s}_i, \mathbf{s}_{i+1})$-encoding of the other factor (the input), which can be derived from $(\mathbf{h}_i, E_i)$. Finally, we can perform hops across levels (always from level i to level $i+1$) by performing multiplications by 1[3]. To summarise, given $\mathbf{x}$ and $(\mathbf{h}_i, E_i)_{i\in[d]}$, Alice is able to derive an encoding

$$\begin{aligned}\mathbf{c}_f^\intercal &\approx \mathbf{s}_0^\intercal \cdot \mathbf{A}_f + \mathbf{a}^\intercal \cdot (f(\mathbf{x})^\intercal \otimes \mathbf{G}) \\ &= \mathbf{s}_0^\intercal \cdot \mathbf{A}_f + (f(\mathbf{x})^\intercal \otimes \mathbf{a}^\intercal \otimes \mathbf{g}_q^\intercal).\end{aligned}$$

To derive a secret-sharing of $f(\mathbf{x}) \otimes \mathbf{a}$, it is therefore sufficient that the parties run a succinct MOLE to compute a secret-sharing of $\mathbf{A}_f^\intercal \cdot \mathbf{s}_0$ similarly to what we did for the other reverse TDH we built. At that point, it is just a matter of applying local linear computations and rounding.

Step II: Building Rate-1 Adaptive LFE. It is finally time to talk about the rate-1 adaptive LFE scheme. Our approach is the following: We pick a *constant* d and we decompose our function f as $f_{L-1} \circ \cdots \circ f_0$ where each f_i is described by an RMS program of depth d. Our goal is to use our reverse TDH to evaluate all f_i until we obtain the output. The issue is that Bob does not know what input to encode for f_i.

Instead of evaluating f_i, we evaluate a function that allows us to retrieve the encoding key for $\mathbf{x}_i := (f_i \circ \cdots \circ f_0)(\mathbf{x})$. Specifically, for every $i \in [L]$, define[4]

$$\hat{f}_i : (\mathbf{x}, \mathbf{a}) \longmapsto \mathsf{OTE.Hash}(f_i(\mathbf{x})) \otimes \mathbf{a}.$$

Suppose that Alice sent a hash for $\hat{f}_i$ for every $i \in [L]$. Suppose also that we somehow managed to find a way to provide Alice with an encoding key for $(\mathbf{x}_i, \mathbf{a}_{i+1})$ where $\mathbf{a}_{i+1} \xleftarrow{\$} \mathbb{Z}_q^k$. Under these premises, the parties can derive an additive secret-sharing $\mathbf{y}_{i+1}^{\mathsf{A}} + \mathbf{y}_{i+1}^{\mathsf{B}} = \mathbf{d}_{i+1} \otimes \mathbf{a}_{i+1}$ where $\mathbf{d}_{i+1} = \mathsf{OTE.Hash}(\mathbf{x}_{i+1})$. We can convert this into a compressed adaptive encoding of $\mathbf{x}_{i+1}$ by making Bob send

$$\mathbf{c}_{i+1}^\intercal = \mathbf{s}_{i+1}^\intercal \cdot \mathbf{A} + \mathbf{e}_{i+1}^\intercal + (\mathbf{y}_{i+1}^{\mathsf{B}})^\intercal \otimes \mathbf{g}_q^\intercal$$

where $\mathbf{s}_{i+1} \xleftarrow{\$} \mathbb{Z}_q^k$ and $\mathbf{e}_{i+1} \xleftarrow{\$} \chi(1^\lambda)$. By adding $(\mathbf{y}_{i+1}^{\mathsf{A}})^\intercal \otimes \mathbf{g}_q^\intercal$ to $\mathbf{c}_{i+1}$, Alice can

[3] Remember that the expansion of $(\mathbf{h}_i, E_i)$ provides also an $(\mathbf{s}_i, \mathbf{s}_{i+1})$-encodings of 1.

[4] In our construction, $\mathsf{OTE.Hash}$ has depth 0 because it is linear.

derived a $(\mathbf{s}_{i+1}, \mathbf{a}_{i+1})$-encoding of $\mathbf{d}_{i+1}$. Indeed,

$$\begin{aligned}\mathbf{c}_{i+1}^{\intercal} + (\mathbf{y}_{i+1}^{\mathsf{A}})^{\intercal} \otimes \mathbf{g}_q^{\intercal} &\approx \mathbf{s}_{i+1}^{\intercal} \cdot \mathbf{A} + (\mathbf{y}_{i+1}^{\mathsf{B}} + \mathbf{y}_{i+1}^{\mathsf{A}})^{\intercal} \otimes \mathbf{g}_q^{\intercal} \\ &= \mathbf{s}_{i+1}^{\intercal} \cdot \mathbf{A} + (\mathbf{d}_{i+1}^{\intercal} \otimes \mathbf{a}_{i+1}^{\intercal} \otimes \mathbf{g}_q^{\intercal}) \\ &= \mathbf{s}_{i+1}^{\intercal} \cdot \mathbf{A} + \mathbf{a}_{i+1}^{\intercal} \cdot (\mathbf{d}_{i+1}^{\intercal} \otimes \mathbf{G}).\end{aligned}$$

Bob can of course send also the other information needed to complete the encoding key for $(\mathbf{x}_{i+1}, \mathbf{a}_{i+2})$ as this is independent of f and $\mathbf{x}_{i+1}$. Notice that the size of all this material that Bob sends is independent of the input size and the output size of f. By continuing in this way, the parties end up with an additive secret-sharing of $f(\mathbf{x}) \otimes \mathbf{a}_L$ where $\mathbf{a}_L \xleftarrow{\$} \mathbb{Z}_q^k$.

There is still one matter we need to take care of: In order to perform the operations we just described, Alice needs to know $\mathbf{x}$. So, how can we achieve privacy of the input? The trick is the same as in [12]: We provide Alice with a GSW [18] encryption of $\mathbf{x}$ using $\mathbf{a}_L$ as a secret key (we also send the corresponding encoding key, which is polylogarithmic in size). Then, instead of evaluating f, we evaluate $f' := \mathsf{GSW.Eval}(f, \cdot)$. At the end, the parties obtain a secret-sharing of $\mathsf{Bits}(\mathsf{ct}) \otimes \mathbf{a}_L$ where ct is a GSW encryption of $f(\mathbf{x})$. Given that $\mathbf{a}_L$ is the secret-key and the GSW decryption consists of linear operations between $\mathbf{a}_L$ and ct (followed by rounding), the parties can obtain a secret-sharing of $f(\mathbf{x})$ by applying only local operations. This immediately gives rate-1 communication in the output. What about rate-1 communication in the input? Well, instead of sending a GSW encryption of $\mathbf{x}$, send $\mathbf{z} := \mathbf{x} \oplus \mathsf{PRG}(K)$ for $K \xleftarrow{\$} \{0,1\}^\lambda$ and a GSW encryption of K. Then, during the evaluation of f', we remove the one-time-pad inside FHE.

1.3 Other Applications

We outline a few additional additional applications of our results. In particular, we discuss some of the new implications from our construction of reverse TDH.

Regular Trapdoor Hash. As an immediate implication of our reverse TDH, we obtain a (regular) TDH [17] with close to optimal parameters, that is: The size of the encoding key is $|f| \cdot \mathsf{poly}(\lambda, \log m)$ where $|f|$ denotes the size of the description of the encoded function and m denotes the size of its input. For instance, notice that if f is a point function with domain of size L, $|f| = \log L$. We achieve this with a simple application of universal circuits: Instead of hashing a function, we hash the universal circuit $U_{\mathbf{x}}$ with the input $\mathbf{x}$ hardwired, that on input a function f, returns $f(\mathbf{x})$. Note that the size of the digest is anyway constant (ignoring factors in the security parameter) so the complexity of the TDH only grows with the bit description of f.

To our knowledge, this (along with a concurrent work [10]) is the first trapdoor hashing schemes that support all functions. This is also the first LWE-based TDH constructions achieving nearly optimal communication for a non-trivial class of functions. To our knowledge, the only other construction where

the size of the encoding key is sublinear in the dimension of the input is a recently built pairing-based TDH for point functions [5]. Such construction, however, pays the succinctness of the encoding key with a non-succinct CRS of size $O(m)$.

Homomorphic Secret Sharing, Spooky Encryption and Public-Key PCGs. In addition, note that the reconstruction of our reverse TDH is additive over $\mathbb{Z}_2$, thus, a reverse trapdoor hash also implies the existence of a (public-key) 2-party homomorphic secret sharing [8] scheme as follows: Suppose that Alice and Bob respectively hold inputs $\mathbf{x}$ and $\mathbf{y}$ and they want to evaluate the function f. Suppose also that $\mathbf{x}$ is much longer than $\mathbf{y}$. Alice proceeds by hashing the function $f_{\mathbf{x}}$ that maps any $\mathbf{y}$ to $f(\mathbf{x}, \mathbf{y})$. She sends the digest to Bob and keeps the randomness as her part of the share of $\mathbf{x}$. Bob, on the other hand, sends an encoding key ek for $\mathbf{y}$ to Alice. He keeps the corresponding trapdoor td as his share of $\mathbf{y}$. By the correctness of reverse TDH, the parties can locally derive a secret sharing of the output. Notice also that the scheme is succinct in $\mathbf{x}$: The total communication of the protocol is $|\mathbf{y}| \cdot \mathsf{poly}(\lambda, \log|\mathbf{x}|)$!

Previously, succinct homomorphic secret sharing had been built by Abram, Roy and Scholl [3] under several assumptions, including LWE, DCR and DDH over class groups. Their constructions, however, supported only a limited class of functions: Alice and Bob could only compute secret-sharings of $\mathbf{x}^\intercal \cdot C(\mathbf{y})$ for any circuit $C \in \mathsf{NC}_1$. Their constructions have also a second drawback: The total complexity of the protocol is $|\mathbf{y}| \cdot |\mathbf{x}|^\varepsilon \cdot \mathsf{poly}(\lambda)$ for a constant $\varepsilon \in (0, 1)$. Our solution instead scales polylogarithmically in $|\mathbf{x}|$. On the negative side, unlike [3], our solution does not allow the parties to evaluate a function that is adaptively chosen *after* the secret-sharing phase. We can however plug our MOLE in the constructions of [3] to obtain an HSS scheme that allows the evaluation of any *adaptively* chosen function $\mathbf{x}^\intercal \cdot C(\mathbf{y})$ with improved communication $|\mathbf{y}| \cdot \mathsf{poly}(\lambda, \log|\mathbf{x}|)$.

Observe that our succinct HSS scheme can be also viewed as a form of 2-party spooky encryption for additively shared correlation [15] where one of the parties can just send a small hash of its input instead of a full-size ciphertext. Once again, differently from [15], our construction does not allow us to choose the correlation after we committed to the inputs.

Finally, our HSS scheme can have interesting applications in the context of (public-key) pseudorandom correlation generators (PCGs) [2, 9, 20], especially when the tackled additively-shared correlation takes a long input from Alice: Let $\mathcal{C}(x)$ be the correlation function with long input. Alice can sample $s_0 \xleftarrow{\$} \{0,1\}^\lambda$ and hash the function that maps y to $(\mathcal{C}(x; r_i))_{i \in [n]}$ where $(r_0, \ldots, r_{n-1}) \leftarrow \mathsf{PRG}(s_0 \oplus y)$. Bob instead picks a random $y \xleftarrow{\$} \{0,1\}^\lambda$ and sends its encoding to Alice.

Rate-1/2 Laconic Oblivious Transfer. Since our construction of reverse TDH also extends to RAM programs using [16], we obtain a new construction of laconic oblivious transfer [14] with rate 1/2 in the batch settings (which is

best possible), where one transfers a set of messages with respect to different indices. The receiver hashes the function f_D that has hardwired a database D and, on input a PRF key k, does the following for all indices i and all bits b:

- If $b = D_i$: Return 0.
- Else return $\mathsf{PRF}(k, i)$.

Then, on input a set of indices I and pairs of bits $\{m_{i,0}, m_{i,1}\}_{i \in I}$, the sender samples a key k and sends $\mathsf{ek} \xleftarrow{\$} \mathsf{Gen}(\mathsf{hk}, k)$, where Gen is the encoding generation algorithm of the TDH, along with:

$$c_{i,b} := \{\mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d)_{i,b} \oplus m_{i,b}\}_{i,b}$$

where we abuse the notation and assume that the TDH decoding algorithm $\mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d)_{i,b}$ returns the (i, b)-bit of the share (which can be computed by a RAM program in time independent of the size of the database). Note that if $b = D_i$, then $\mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d)_{i,b} = \mathsf{Enc}(\mathsf{hk}, \mathsf{ek}, f_D, \rho)_{i,b}$ and therefore the receiver can recover $m_{i,b}$ running the TDH encoding algorithm Enc. Otherwise the pseudorandomness of PRF guarantees that the message is computationally hidden.

Attribute-Based Non-Interactive Key-Exchange. Finally, reverse TDH implies the existence of a non-interactive key exchange (NIKE) with the following additional property: One of the two parties can include the hash of a function f as part of their public key, whereas the other party holds an input $\mathbf{x}$. The NIKE succeeds if $f(\mathbf{x}) = 1$ and otherwise the key of either party is computationally indistinguishable from random. This can be constructed from reverse TDH in a natural manner: The former party hashes the function F_{f,k_0} that takes as input some $\mathbf{x}$ and a key k_1 and returns 0 if $f(\mathbf{x}) = 1$ and $\mathsf{PRF}(k_0, \mathbf{x}) \oplus \mathsf{PRF}(k_1, \mathbf{x})$ otherwise. If f is satisfied, then both parties hold the same share, that can be used as a shared key, otherwise the pseudorandomness of PRF protects the share of either party.

1.4 Concurrent Work

A concurrent work by Boyle et al. [10] also construct a family of TDHs for all functions $f : \{0,1\}^m \to \{0,1\}^\ell$, using a similar idea. An important difference is that they rely on the $|\mathbf{x}|^{2/3}$-succinct VOLE protocol from [3], whereas we (implicitly) construct a $\mathsf{polylog}(|\mathbf{x}|)$-succinct one, derived from our OTE. This translates in different parameters for the encoding key of the TDH: The encoding key of [10] has size $(|f| + \ell^{2/3} + D) \cdot \mathsf{poly}(\lambda)$, whereas in our TDH the encoding key has size $|f| \cdot \mathsf{poly}(\lambda, \log m)$, which is close to optimal.

Besides TDHs, the results in [10] are largely orthogonal to ours. We also mention here that plugging in our OTE protocol in [10] leads to similar parameter improvements for their other applications. For instance, following the outline of [10], OTE yields a rate-1 fully homomorphic encryption (FHE) with optimal parameters. We fully credit [10] for discovering the connection, and

we sketch here the transformation only for the sake of completeness. Take any FHE scheme with almost-linear decryption [13], i.e., where decryption is a linear function is the secret key $\mathbf{s}$, followed by a rounding, such as [18]. Then add to the public key an MOLE encoding $\mathsf{Enc}(\mathbf{s})$. We can compress m ciphertexts $(\mathbf{c}_1, \ldots, \mathbf{c}_m)$ as follows: Stack them into a matrix $\mathbf{C}$, then compute the MOLE-hash $\mathsf{Hash}(\mathbf{C})$, and run the Hash-Eval algorithm, to obtain a additive share of $\mathsf{Round}(\mathbf{C} \cdot \mathbf{s}) \in \{0,1\}^m$. Return the hash $\mathsf{Hash}(\mathbf{C})$, along with the bits of the share. Decryption works by simply running the Encoder-Eval algorithm of the MOLE, and reconstructing the output. Crucially, the compressed ciphertext consists of a hash (whose size is a fixed polynomial in the security parameter) plus m bits, i.e., it is $\mathsf{poly}(\lambda) + m$, which is optimal. This improves upon [13] since the size of the public key does not depend on m (no amortization is needed).

2 Preliminaries

Notation. We denote the security parameter by λ. We say that a function $\mathsf{negl}(\lambda)$ is negligible if it vanishes faster than any polynomial, i.e., $0 \leq \mathsf{negl}(\lambda) \leq \lambda^{-\omega(1)}$. We say that an event happens with overwhelming probability, if it occurs with probability negligibly close to 1. For any $n \in \mathbb{N}$, we use $[n]$ to denote the set $\{0, 1, \ldots, n-1\}$.

All vectors are denoted using lowercase bold font, whereas matrices are denoted using uppercase bold font; by default, all vectors are columns. Given a matrix $\mathbf{M}$, we denote its transpose by $\mathbf{M}^\intercal$, whereas $\mathsf{vec}(\mathbf{M})$ denotes its vectorisation, i.e., stacking the columns of $\mathbf{M}$ one underneath the other. For any $n \in \mathbb{N}$, we denote the $n \times n$ identity matrix by $\mathbf{I}_n$ and the n-dimensional row vector where all entries are equal to 1 by $\mathbf{1}^n$. We define the Kronecker product between two $n \times m$ matrices $\mathbf{A} \otimes \mathbf{B}$ to be

$$\mathbf{A} \otimes \mathbf{B} := \begin{pmatrix} a_{1,1}\mathbf{B} & \ldots & a_{1,m}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{n,1}\mathbf{B} & \ldots & a_{n,m}\mathbf{B} \end{pmatrix}.$$

For a vector $\mathbf{x} \in \mathbb{Z}_q^n$, we denote by $\|\mathbf{x}\|_\infty$ its infinity norm, i.e., the maximal distance from 0 of its coordinate in the standard centered representative convention, and we extend this notation to matrices by taking the maximum over their columns. For any integer $q > 0$, let $\mathbf{g}_q^\intercal$ be the gadget row-vector $(1, 2, \ldots, 2^{\log q})$ and we omit the subscript when clear from the context. Let $\mathbf{G}^{-1}$ be the algorithm that takes as input a matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times m}$ and outputs a matrix $\mathbf{M}' \in \mathbb{Z}_2^{(n \cdot \log q) \times m}$, where each column is derived by stacking the bit-decompositions of the entries in the corresponding column of $\mathbf{M}$. Notice that $(\mathbf{I}_n \otimes \mathbf{g}_q^\intercal) \cdot \mathbf{G}^{-1}(\mathbf{M}) = \mathbf{M}$. We use $\mathsf{Bits}(\mathbf{M})$ to denote $\mathsf{vec}(\mathbf{G}^{-1}(\mathbf{M}))$. Given an integer $x \in \mathbb{Z}_q$ and an integer p that divides q, we use $\lceil x \rfloor_p$ to denote rounding to $\mathbb{Z}_p$, in other words, if $x = y \cdot q/p + z$ where $z \in [-q/2p, q/2p)$, we have $\lceil x \rfloor_p = y$.

We state the following useful lemma about matrices.

Lemma 4 (Matrix Linearisation)**.** *There exists a deterministic polynomial-time computable function:*

$$\mathsf{Lin} : \mathbb{Z}_q^{(t\cdot\ell)\times m} \to \mathbb{Z}_q^{t\times(m\cdot\ell)}$$

such that, for any matrix $\mathbf{B} \in \mathbb{Z}_q^{(t\cdot\ell)\times m}$ *and any pair of vectors* $\mathbf{x} \in \mathbb{Z}_q^m$ *and* $\mathbf{s} \in \mathbb{Z}_q^\ell$*, it holds that:*

$$\mathsf{Lin}(\mathbf{B})(\mathbf{x}\otimes\mathbf{s}) = (\mathbf{I}_t \otimes \mathbf{s}^\intercal)\mathbf{B}\mathbf{x} \quad \textit{and} \quad \|\mathsf{Lin}(\mathbf{B})\|_\infty = \|\mathbf{B}\|_\infty.$$

Proof. Note that the h-th entry of $(\mathbf{I}_t \otimes \mathbf{s}^\intercal)\mathbf{B}\mathbf{x}$ is:

$$\sum_{i\in[\ell]}\sum_{j\in[m]} \mathbf{B}_{\ell\cdot h+i,j}\cdot(\mathbf{s}_i\cdot\mathbf{x}_j).$$

In other words, there exists a matrix $\mathsf{Lin}(\mathbf{B})$, obtained by rearranging the entries of $\mathbf{B}$ such that $\mathsf{Lin}(\mathbf{B})(\mathbf{x}\otimes\mathbf{s}) = (\mathbf{I}_t \otimes \mathbf{s}^\intercal)\mathbf{B}\mathbf{x}$. Since rearranging the entries does not change the infinity norm, the claim follows. □

We also recall the definition of RMS program. Essentially this consists of an algebraic circuit over $\mathbb{Z}$ where we can multiply two wires only if at least one of them is an input.

Definition 5 (Restricted Multiplication Straightline)**.** *A restricted multiplication straightline program (RMS) consists of a polynomial-sized family of algebraic circuits over* $\mathbb{Z}$ *where the only allowed gates are the following:*

- *Additions: this gate takes as input two wires* x *and* y *and outputs their sum* $x + y$*.*
- *Scalar multiplication: each of these gates is parametrised by a constant* $\alpha \in \mathbb{Z}$*. It takes as input a single wire* x *and outputs* $\alpha \cdot x$*.*
- *Multiplication: this gate takes as input two wires* x *and* y where y is an input to the RMS program. *The output is their product* $x \cdot y$*.*

We say that the program has depth d *if the* multiplicative depth *of the program is* d*.*

Let T *be a positive integer. We say that an RMS program is* T*-bounded if, during any evaluation over* binary *inputs, the absolute value of the wires never exceeds* T*.*

We recall the following result which states that any circuit in NC_1 can be converted into a polynomial size RMS program (paying exponentially in the depth).

Theorem 6 ([4, 8])**.** *Let* $f : \{0,1\}^n \to \{0,1\}$ *be described by a boolean circuit of size* s *and depth* d *made entirely of NAND gates. Then,* f *can be computed using a* 1*-bounded RMS program of depth at most* 2^d *and size* $O(s \cdot 2^d)$*.*

2.1 Lattices and Learning with Errors

Throughout the paper, we often rely on a low-norm distribution $\chi(\lambda)$ over $\mathbb{Z}$. We say that $\chi(\lambda)$ is $B(\lambda)$-bounded if:

$$\Pr\left[|e| \leq B(\lambda) \middle| e \xleftarrow{\$} \chi(\lambda)\right] = 1.$$

Sometimes, we abuse notation and we write $\mathbf{v} \xleftarrow{\$} \chi(\lambda)$, even if $\mathbf{v}$ is a vector: With this, we mean that each entry of $\mathbf{v}$ is sampled from $\chi(\lambda)$ independently of all the others. Henceforth, we always assume that $\chi(\lambda)$ is a $B(\lambda)$-bounded distribution.

We recall the learning with error assumption, introduced for the first time by Regev [22].

Definition 7 (Learning with Errors)**.** *Let $k := k(\lambda)$, $m := m(\lambda)$ and $q := q(\lambda)$ be positive integers. Let $\chi(\lambda)$ be a low-norm distribution over $\mathbb{Z}$. We say that the (χ, k, m, q)-LWE problem is hard if, for every PPT adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\lambda)$ such that, for every $\lambda \in \mathbb{N}$, we have*

$$\left| \Pr\left[\mathcal{A}(1^\lambda, \mathbf{M}, \mathbf{u}_b) = b \middle| \begin{array}{l} b \xleftarrow{\$} \{0,1\}, \mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k \\ \mathbf{M} \xleftarrow{\$} \mathbb{Z}_q^{m \times k}, \mathbf{e} \xleftarrow{\$} \chi(\lambda)^m \\ \mathbf{u}_0 \leftarrow \mathbf{M} \cdot \mathbf{s} + \mathbf{e}, \mathbf{u}_1 \xleftarrow{\$} \mathbb{Z}_q^m \end{array} \right] - \frac{1}{2} \right| \leq \mathsf{negl}(\lambda).$$

Suppose that $\chi(\lambda)$ is $B(\lambda)$-bounded. We call the quantity $\alpha := q/B$ the modulus-noise ratio.

We also define the *non-uniform LWE* assumption identically as above, except that the matrix $\mathbf{M}$ is no longer uniformly random over $\mathbb{Z}_q^{m \times k}$ but over $\mathbb{Z}_2^{m \times k}$. It is shown in [6] that non-uniform LWE is at least as hard as LWE, with a slightly increased parameter size.

Theorem 8 ([6])**.** *Assume the hardness of (χ, k, m, q)-LWE. Then, $(\chi, k \cdot \log q, m, q)$-NLWE is hard.*

3 Non-Interactive Oblivious Tensor Evaluation

In the following we define and construct a succinct Non-Interactive Oblivious Tensor Evaluation (NI-OTE) protocol.

3.1 Definitions

We begin by defining the syntax of NI-OTE.

Definition 9 (Non-interactive oblivious tensor evaluation)**.** *Let $m := m(\lambda)$, $\ell := \ell(\lambda)$, and $q := q(\lambda)$ be a positive integer. A NI-OTE for $\mathbb{Z}_q^m \otimes \mathbb{Z}_q^\ell$ consists of a tuple of PPT algorithms* $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval})$ *with the following syntax:*

$\mathsf{Setup}(1^\lambda)$**:** *The setup algorithm is probabilistic, takes as input the security parameter* 1^λ *and outputs a public key* pk*.*

$\mathsf{Hash}(\mathsf{pk}, \mathbf{x})$**:** *The hashing algorithm is probabilistic and takes as input a public key* pk *and the description of a vector* $\mathbf{x} \in \mathbb{Z}_q^m$*. The output is a digest* d *and hasher's private information* ψ*.*

$\mathsf{Enc}(\mathsf{pk}, \mathbf{y})$**:** *The encoding algorithm is probabilistic and takes as input a public key* pk*, and a vector* $\mathbf{y} \in \mathbb{Z}_q^\ell$*. The output is an encoding* E *and encoder's private information* ϕ*.*

$\mathsf{HashEval}(\mathsf{pk}, E, \psi)$**:** *The hasher's evaluation algorithm is deterministic and takes as input a public key* pk*, an encoding* E *and hasher's private information* ψ*. The output is a vector* $\mathbf{v} \in \mathbb{Z}_q^{m \cdot \ell}$*.*

$\mathsf{EncEval}(\mathsf{pk}, d, \phi)$**:** *The encoder's evaluation algorithm is deterministic and takes as input a public key* pk*, a digest* d *and encoder's private information* ϕ*. The output is a vector* $\mathbf{w} \in \mathbb{Z}_q^{m \cdot \ell}$*.*

Sometimes it will be convenient for us to fix the private information ϕ and provide it as an input to the encoding algorithm. In a slight abuse of notation, we denote this by $\mathsf{Enc}(\mathsf{pk}, \mathbf{y}, [\phi])$, in which case, the algorithm just outputs E. If the NI-OTE scheme satisfies this syntactical requirement, which in particular means that ϕ does not depend on $\mathbf{y}$, we say that the scheme is *programmable*.

Additionally, we say that an NI-OTE has a *bilinear encoder evaluation* if:

- The digest d is a vector in $\mathbb{Z}_q^n$.
- The encoder secret information ϕ is a vector in $\mathbb{Z}_q^k$.
- The encoder evaluation algorithm consists of

$$\mathsf{EncEval}(\mathsf{pk}, d, \phi) := \mathbf{P} \cdot (d \otimes \phi \otimes \mathbf{g}_q)$$

  where the matrix $\mathbf{P} \in \mathbb{Z}_q^{(m \cdot \ell) \times (n \cdot k \cdot \log q)}$ can be publicly derived from pk.

We say that an NI-OTE is *succinct* if the size of the hash and the size of the encodings are sublinear in the size of the hasher's input. Depending on the context, we will make the dependence explicit. If only the hash is sublinear, then we say that the scheme is *half-succinct*.

Correctness. Next, we define (approximate) correctness for an NI-OTE, parametrized by an error function α. If $\alpha = 0$, then we say that the NI-OTE is perfectly correct, or simply correct.

Definition 10 (α-Correctness)**.** *An NI-OTE scheme* $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval})$ *is* α*-adaptively correct if there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every adversary* $\mathcal{A}$ *and sufficiently large* $\lambda \in \mathbb{N}$*, we have that:*

$$\Pr\left[\|\mathbf{v} + \mathbf{w} - \mathbf{x} \otimes \mathbf{y}\|_\infty > \alpha \cdot \|\mathbf{x}\|_\infty\right] \leq \mathsf{negl}(\lambda)$$

where the probability is taken over the random choice of $\mathsf{pk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$, $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{A}(1^\lambda, \mathsf{pk})$, $(d, \psi) \xleftarrow{\$} \mathsf{Hash}(\mathsf{pk}, \mathbf{x})$, $(E, \phi) \xleftarrow{\$} \mathsf{Enc}(\mathsf{pk}, \mathbf{y})$, *and we define* $\mathbf{v} := \mathsf{HashEval}(\mathsf{pk}, E, \psi)$ $\mathbf{w} := \mathsf{EncEval}(\mathsf{pk}, d, \phi)$.

The definition of approximate correctness will be sufficient for our applications. Nevertheless, for future reference, we mention here that there is a general (standard) method to drive the correctness error down to $\alpha = 0$, which may be more convenient to work with. Specifically, let p to be a divisor of q with $q = p \cdot \alpha \cdot \|\mathbf{x}\|_\infty \cdot \lambda^{\omega(\log \lambda)}$. Instead of encoding $\mathbf{y}$, one encodes $q/p \cdot \mathbf{y}$ and sends along a random PRG seed seed. Then, we return

$$\mathbf{v} := \left\lceil \mathsf{HashEval}(\mathsf{pk}, E, \psi) + \mathsf{PRG}(\mathsf{seed}) \right\rfloor_p, \qquad \text{and} \qquad \mathbf{w} := \left\lceil \mathsf{EncEval}(\mathsf{pk}, d, \phi) - \mathsf{PRG}(\mathsf{seed}) \right\rfloor_p.$$

By the α-correctness guarantee of the protocol, it holds that:

$$\mathsf{HashEval}(\mathsf{pk}, E, \psi) + \mathsf{EncEval}(\mathsf{pk}, d, \phi) = q/p \cdot \mathbf{x} \otimes \mathbf{y} + \mathbf{e}, \qquad \|\mathbf{e}\|_\infty \leq \alpha \|\mathbf{x}\|_\infty.$$

Moreover, by the PRG security, $\mathbf{v}$ is indistinguishable to a random element in $\mathbb{Z}_q^{m\ell}$, independent of $\mathbf{x}$, $\mathbf{y}$ and the public parameters. Following [15], with overwhelming probability over the randomness of seed, we obtain

$$\begin{aligned}
\mathbf{v} + \mathbf{w} &= \left\lceil \mathsf{HashEval}(\mathsf{pk}, E, \psi) + \mathsf{PRG}(\mathsf{seed}) + \mathsf{EncEval}(\mathsf{pk}, d, \phi) - \mathsf{PRG}(\mathsf{seed}) \right\rfloor_p \\
&= \left\lceil q/p \cdot \mathbf{x} \otimes \mathbf{y} + \mathbf{e} \right\rfloor_p \\
&= \lceil q/p \cdot \mathbf{x} \otimes \mathbf{y} \rfloor_p \\
&= \mathbf{x} \otimes \mathbf{y} \pmod{p}.
\end{aligned}$$

Notice that with these modifications, the protocol outputs the correct output with overwhelming probability, and this holds even if the inputs are chosen by the adversary after seeing the public parameters.

Encoder Privacy. We define encoder privacy, which guarantees that the encoding is simulatable, without knowing the underlying vector. This is formalized as follows.

Definition 11 (Encoder Privacy)**.** *Consider the following experiment* $\mathsf{EncExp}_{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ *and a simulator* Sim*:*

- *Sample a public key* $\mathsf{pk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$.
- *Activate the adversary* $(\mathbf{y}, \mathsf{aux}) \xleftarrow{\$} \mathcal{A}_0(1^\lambda, \mathsf{pk})$.
- *Sample a random bit* $b \xleftarrow{\$} \{0, 1\}$.
- *If* $b = 0$ *compute* $(E_0, \phi_0) \xleftarrow{\$} \mathsf{Enc}(\mathsf{pk}, \mathbf{y})$, *else compute* $E_1 \xleftarrow{\$} \mathsf{Sim}(1^\lambda, \mathsf{pk})$.
- *Compute* $b' \leftarrow \mathcal{A}_1(E_b, \mathsf{aux})$.

- *Return* 1 *if and only if* $b = b'$.

A non-interactive OTE scheme $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval})$ *is encoder-private if there exists a PPT simulator* Sim*, such that for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left|\frac{1}{2} - \Pr\left[\mathsf{EncExp}_{\mathcal{A},\mathsf{Sim}}(1^\lambda) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

Hasher Privacy. We also define hasher privacy, which guarantees that the hash is simulatable, without knowing the underlying vector. This is formalized as follows.

Definition 12 (Encoder Privacy)**.** *Consider the following experiment* $\mathsf{HashExp}_{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ *and a simulator* Sim*:*

- *Sample a public key* $\mathsf{pk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$.
- *Activate the adversary* $(\mathbf{x}, \mathsf{aux}) \xleftarrow{\$} \mathcal{A}_0(1^\lambda, \mathsf{pk})$.
- *Sample a random bit* $b \xleftarrow{\$} \{0,1\}$.
- *If* $b = 0$ *compute* $(d_0, \psi_0) \xleftarrow{\$} \mathsf{Hash}(\mathsf{pk}, \mathbf{x})$*, else compute* $d_1 \xleftarrow{\$} \mathsf{Sim}(1^\lambda, \mathsf{pk})$.
- *Compute* $b' \leftarrow \mathcal{A}_1(d_b, \mathsf{aux})$.
- *Return* 1 *if and only if* $b = b'$.

A non-interactive OTE scheme $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval})$ *is hasher-private if there exists a PPT simulator* Sim*, such that for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left|\frac{1}{2} - \Pr\left[\mathsf{HashExp}_{\mathcal{A},\mathsf{Sim}}(1^\lambda) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

3.2 Half-Succinct NI-OTE from LWE

We present our first NI-OTE protocol that only satisfies a weak form of succinctness, namely that only the size of the digest is sublinear in the size of the hasher's input. Let $n := k \cdot \log q$, where $k = \Theta(\lambda)$, and suppose that for convenience that q is a power of 2. Our protocol is described in Construction 1.

The scheme is trivially programmable, since ϕ consists of a uniformly sampled vector $\mathbf{s}$ that in particular is independent from $\mathbf{y}$. Then, we observe that the scheme has indeed a bilinear encoding evaluation algorithm. By Lemma 4, it holds that:

$$-(\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}^\intercal \otimes \mathbf{g}_q^\intercal) \cdot \mathbf{B}^\intercal \cdot \mathbf{d} = \mathsf{Lin}(-\mathbf{B}^\intercal)(\mathbf{d} \otimes \mathbf{s} \otimes \mathbf{g}_q) \tag{1}$$

Construction 1: Half-Succinct NI-OTE

$\mathsf{Setup}(1^\lambda)$**:** Sample $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_2^{n\times m}$ and $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_2^{n\times(m\cdot\ell\cdot n)}$, then return $\mathsf{pk} := (\mathbf{A}, \mathbf{B})$.

$\mathsf{Hash}(\mathsf{pk}, \mathbf{x})$**:** Compute $\mathbf{d} := \mathbf{A} \cdot \mathbf{x}$ and return $d := \mathbf{d}$ and $\psi := \mathbf{x}$.

$\mathsf{Enc}(\mathsf{pk}, \mathbf{y})$**:** Sample $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^k$, $\hat{\mathbf{E}} \xleftarrow{\$} \chi(\lambda)$, and $\overline{\mathbf{E}} \xleftarrow{\$} \chi(\lambda)$. Compute

$$\mathbf{C} := \mathbf{A}^\intercal \cdot \mathbf{B} \cdot (\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s} \otimes \mathbf{g}_q) + \mathbf{A}^\intercal \cdot \hat{\mathbf{E}} + \overline{\mathbf{E}} + \mathbf{I}_m \otimes \mathbf{y}^\intercal.$$

Return $E := \mathbf{C}$ and $\phi := \mathbf{s}$.

$\mathsf{HashEval}(\mathsf{pk}, E, \psi)$**:** Return $\mathbf{v} := \mathbf{C}^\intercal \cdot \mathbf{x}$.

$\mathsf{EncEval}(\mathsf{pk}, d, \phi)$**:** Return $\mathbf{w} := -(\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s}^\intercal \otimes \mathbf{g}_q^\intercal) \cdot \mathbf{B}^\intercal \cdot \mathbf{d}$.

with $\|\mathsf{Lin}(-\mathbf{B}^\intercal)\|_\infty = \|-\mathbf{B}^\intercal\|_\infty \le 1$. We can also bound the norm of the digest by

$$\|\mathbf{d}\|_\infty = \|\mathbf{A} \cdot \mathbf{x}\|_\infty \le m \cdot \|\mathbf{A}\|_\infty \cdot \|\mathbf{x}\|_\infty \le m \cdot \|\mathbf{x}\|_\infty \tag{2}$$

with a triangle inequality. Finally, it is easy to see that the scheme is half succinct, since the hash $\mathbf{d}$ is n-dimensional vector over $\mathbb{Z}_q$, whose size is in particular independent of the length of $\mathbf{x}$ and $\mathbf{y}$. On the other hand, the encoding consists of an $m \times (m \cdot \ell)$ matrix over $\mathbb{Z}_q$.

Next, we argue that the scheme satisfies approximate correctness. Indeed, let us rewrite

$$\mathbf{C} = \mathbf{A}^\intercal \cdot \mathbf{Z} + \widetilde{\mathbf{E}} + \mathbf{I}_m \otimes \mathbf{y}^\intercal,$$

where $\mathbf{Z} := \mathbf{B} \cdot (\mathbf{I}_{m\cdot\ell} \otimes \mathbf{s} \otimes \mathbf{g}_q)$ and $\widetilde{\mathbf{E}} := \mathbf{A}^\intercal \cdot \hat{\mathbf{E}} + \overline{\mathbf{E}}$. Substituting, we obtain

$$\begin{aligned}
\mathbf{v} + \mathbf{w} &= \mathbf{C}^\intercal \cdot \mathbf{x} - \mathbf{Z}^\intercal \cdot \mathbf{d} \\
&= (\mathbf{Z}^\intercal \cdot \mathbf{A}) \cdot \mathbf{x} + \widetilde{\mathbf{E}}^\intercal \cdot \mathbf{x} + (\mathbf{I}_m \otimes \mathbf{y}) \cdot \mathbf{x} - \mathbf{Z}^\intercal \cdot \mathbf{d} \\
&= \mathbf{Z}^\intercal \cdot \mathbf{d} + \widetilde{\mathbf{E}}^\intercal \cdot \mathbf{x} + \mathbf{x} \otimes \mathbf{y} - \mathbf{Z}^\intercal \cdot \mathbf{d} \\
&= \mathbf{x} \otimes \mathbf{y} + \widetilde{\mathbf{E}}^\intercal \cdot \mathbf{x}.
\end{aligned}$$

Since $\mathbf{A}$ is a matrix in $\mathbb{Z}_2$ with n rows, the entries of $\widetilde{\mathbf{E}} = \mathbf{A}^\intercal \cdot \hat{\mathbf{E}} + \overline{\mathbf{E}}$ is obtained by adding at most $n+1$ entries of the vectors $\overline{\mathbf{E}}$ and $\hat{\mathbf{E}}$, which are $B(\lambda)$-bounded. In other words,

$$\left\| \mathbf{A}^\intercal \cdot \hat{\mathbf{E}} + \overline{\mathbf{E}} \right\|_\infty \le (n+1) \cdot B(\lambda).$$

Therefore we can bound the correctness error by

$$\left\| \widetilde{\mathbf{E}}^\intercal \cdot \mathbf{x} \right\|_\infty \le m \cdot (n+1) \cdot B(\lambda) \cdot \|\mathbf{x}\|_\infty. \tag{3}$$

with a triangle inequality. Finally, we show that the scheme satisfies encoder privacy.

Theorem 13 (Encoder Privacy)**.** *Assuming the hardness of LWE, Construction 1 satisfies encoder privacy.*

Proof. Consider the following sequence of hybrids.

- Hybrid $\mathcal{H}_0$: This is the original distribution.
- Hybrid $\mathcal{H}_1$: We define

  $$\mathbf{C} := \mathbf{A}^\intercal \cdot \mathbf{U} + \overline{\mathbf{E}} + (\mathbf{I}_m \otimes \mathbf{y}^\intercal)$$

  where $\mathbf{U} \xleftarrow{\$} \mathbb{Z}_q^{n \times (m \cdot \ell)}$.

  We claim that this hybrid is indistinguishable from the previous once, by the LWE assumption. To see why, observe that the encoding provided to the adversary in the previous hybrid equals:

  $$\mathbf{C} = \mathbf{A}^\intercal \cdot \mathbf{U} + \overline{\mathbf{E}} + (\mathbf{I}_m \otimes \mathbf{y}^\intercal),$$

  where $\mathbf{U} := \mathbf{B} \cdot (\mathbf{I}_{m \cdot \ell} \otimes \mathbf{s} \otimes \mathbf{g}_q) + \hat{\mathbf{E}}$ for $\hat{\mathbf{E}} \xleftarrow{\$} \chi(\lambda)$ and $\overline{\mathbf{E}} \xleftarrow{\$} \chi(\lambda)$. This last term consists of an LWE sample with secret $\mathbf{s}$ and public matrix obtained by splitting $\mathbf{B} \cdot (\mathbf{I}_{m \cdot \ell \cdot k} \otimes \mathbf{g}_q)$ in $m \cdot \ell$ blocks in $\mathbb{Z}_2^{n \times k}$ and stacking them one underneath the other.

  In more details, consider a reduction that receives a matrix $\mathbf{M} \in \mathbb{Z}_q^{(n \cdot m \cdot \ell) \times k}$ and a vector $\mathbf{u} \in \mathbb{Z}_q^{n \cdot m \cdot \ell}$ that is either an LWE sample with respect to $\mathbf{M}$ or a uniformly random vector. The reduction splits the rows of $\mathbf{M}$ into $\ell \cdot m$ blocks in $\mathbb{Z}_q^{n \times k}$, denoted by

  $$\mathbf{M} = \begin{pmatrix} \mathbf{M}_0 \\ \vdots \\ \mathbf{M}_{m \cdot \ell - 1} \end{pmatrix}.$$

  For every $j \in [m \cdot \ell]$, set $\mathbf{B}_j := \left(\mathbf{G}^{-1}(\mathbf{M}_j^\intercal)\right)^\intercal$. Notice that $\mathbf{B}_j$ is a random matrix over $\mathbb{Z}_2^{n \times n}$, since $\mathbf{M}_j$ is uniformly sampled and q is a power of 2. Finally, construct $\mathbf{B}$ as

  $$\mathbf{B} = (\mathbf{B}_0, \ldots, \mathbf{B}_{m \cdot \ell - 1})$$

  and generate the columns in $\mathbf{U}$ by splitting $\mathbf{u}$ into $m \cdot \ell$ vectors in $\mathbb{Z}_q^n$. Then set $\mathbf{C}$ to $\mathbf{A}^\intercal \cdot \mathbf{U} + \overline{\mathbf{E}} + (\mathbf{I}_m \otimes \mathbf{y}^\intercal)$.

- Hybrid $\mathcal{H}_2$: We sample $\mathbf{C} \xleftarrow{\$} \mathbb{Z}_q^{(m) \times (m \cdot \ell)}$.

  Indistinguishability from the previous hybrid follows by another reduction to the LWE problem. Indeed, notice that $\mathbf{A}^\intercal \cdot \mathbf{U} + \overline{\mathbf{E}}$ is a batch of $m \cdot \ell$ LWE samples with $\mathbf{A}^\intercal$ as public matrix. The LWE secrets consist of the columns of $\mathbf{U}$.

The proof is concluded by defining the simulator Sim to output a uniformly random $\mathbf{C} \xleftarrow{\$} \mathbb{Z}_q^{(m) \times (m \cdot \ell)}$ as the encoding. □

3.3 Bootstrapping to Fully-Succinct NI-OTE

We now show a bootstrapping procedure to turn the NI-OTE constructed in Section 3.2 into a fully succinct NI-OTE for $\mathbb{Z}_q^m \otimes \mathbb{Z}_q^\ell$. We assume that $m := t^r \cdot n$, for some $t, r \in \mathbb{Z}$, and we assume the existence of a half-succinct α-correct $\mathsf{hOTE} = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval})$ for $\mathbb{Z}_q^{t \cdot n} \otimes \mathbb{Z}_q^n$ with bilinear encoder evaluation, where $d \in \mathbb{Z}_q^n$ and $\phi \in \mathbb{Z}_q^k$, with $n := k \cdot \log q$. We describe our scheme in Construction 2.

Construction 2: Fully Succinct NI-OTE

$\mathsf{Setup}(1^\lambda)$**:** Return $\mathsf{pk} \xleftarrow{\$} \mathsf{hOTE.Setup}(1^\lambda)$.

$\mathsf{Hash}(\mathsf{pk}, \mathbf{x})$**:** Set $\mathbf{x}_0 := \mathbf{x}$. Then for every $i \in [r]$ proceed as follows:

- Parse $\mathbf{x}_i$ as the vertical concatenation of $\mathbf{x}_{i,0}, \ldots, \mathbf{x}_{i,t^{r-i-1}-1}$ where $\mathbf{x}_{i,j} \in \mathbb{Z}_q^{t \cdot n}$.
- For every $j \in [t^{r-i-1}]$, compute

$$(\mathbf{d}_{i,j}, \psi_{i,j}) \xleftarrow{\$} \mathsf{hOTE.Hash}(\mathsf{pk}, \mathbf{x}_{i,j}).$$

- Define $\mathbf{x}_{i+1}$ to be the vertical concatenation of the $\mathbf{d}_{i,j}$.

Return $d := \mathbf{x}_r$ and $\psi := \{\psi_{i,j}\}_{i \in [r], j \in [t^{r-i-1}]}$.

$\mathsf{Enc}(\mathsf{pk}, \mathbf{y})$**:** Set $\mathbf{y}_0 := \mathbf{y}$, then for $i \in [r]$, compute

$$(E_i, \phi_{i+1}) \xleftarrow{\$} \mathsf{Enc}(\mathsf{pk}, \mathbf{y}_i) \quad \text{and} \quad \mathbf{y}_{i+1} := \phi_{i+1} \otimes \mathbf{g}_q$$

Output $E := \{E_i\}_{i \in [r]}$ and $\phi := \phi_r$.

$\mathsf{HashEval}(\mathsf{pk}, E, \psi)$**:** Let $\mathbf{P}$ be the public matrix derived from pk. For all $i \in [r]$ and $j \in [t^{r-i-1}]$, compute

$$\mathbf{P}_i := (\mathbf{I}_{t^{r-i}} \otimes \mathbf{P}) \quad \text{and} \quad \mathbf{v}_{i,j} := \mathsf{hOTE.HashEval}(\mathsf{pk}, E_i, \psi_{i,j}).$$

Let $\mathbf{v}_i$ be the vertical concatenation of $\{\mathbf{v}_{i,j}\}_{j \in [t^{r-i-1}]}$. Return $\mathbf{v} := \sum_{i \in [r]} \left(\prod_{k=1}^{i} \mathbf{P}_k \right) \cdot \mathbf{v}_i$.

$\mathsf{EncEval}(\mathsf{pk}, d, \phi)$**:** Let $\mathbf{P}_i$ be defined as above. Return

$$\mathbf{w} := \left(\prod_{i=1}^{r-1} \mathbf{P}_i \right) \cdot \mathsf{hOTE.EncEval}(\mathsf{pk}, d, \phi_r).$$

The scheme is clearly programmable, if the underlying hOTE is. Furthermore, note that the scheme maintains a bilinear encoder evaluation, since, by

Equation (1) we have that:

$$\mathsf{EncEval}(\mathsf{pk}, d, \phi) = \left(\prod_{i=1}^{r-1} \mathbf{P}_i\right) \cdot \mathbf{P} \cdot (d \otimes \phi \otimes \mathbf{g}_q). \tag{4}$$

To bound the norm of the public matrix, recall that $\mathbf{P} \in \mathbb{Z}_q^{n^2 \times (t \cdot n^2)}$ and $\|\mathbf{P}\|_\infty \leq 1$. So, for every $i \in [r]$, each row of $\mathbf{P}_i = (\mathbf{I}_{t^{r-i}} \otimes \mathbf{P})$ has at most n^2 non-zero entries. Thus, multiplying by $\mathbf{P}_i$ increases the infinity norm at most by a factor n^2. Thus we have that:

$$\left\|\prod_{i=1}^{r-1} \mathbf{P}_i \cdot \mathbf{P}\right\|_\infty \leq n^{2r}. \tag{5}$$

On the other hand, we can bound the norm of the digest by recalling that $\mathbf{x}_{i,j} \in \mathbb{Z}_q^{t \cdot n}$ and by Equation (2) the norm of the digest is increased by a factor $t \cdot n$ each time it is hashed. Thus:

$$\|d\|_\infty = \|\mathbf{x}_r\|_\infty \leq (tn)^r \cdot \|\mathbf{x}\|_\infty. \tag{6}$$

Furthermore, the dimension of the digest is n, and therefore independent of the dimensions of the input vector, and its bit-length is at most $n \cdot (r \cdot \log tn + \log\|\mathbf{x}\|_\infty)$. The size of the encoding is $r = O(\log m)$ times the size of an encoding of hOTE (whose size is independent of m). Thus, the scheme is fully succinct.

Approximate Correctness. Towards proving approximate correctness, we begin by observing that, by the α-correctness of hOTE, for every $i \in [r]$ and $j \in [t^{r-i-1}]$, we have:

$$\mathsf{EncEval}(\mathsf{pk}, \mathbf{d}_{i,j}, \phi_{i+1}) + \mathsf{HashEval}(\mathsf{pk}, E_i, \psi_{i,j}) = \mathbf{x}_{i,j} \otimes \mathbf{y}_i + \mathbf{e}_{i,j}, \tag{7}$$

where $\|\mathbf{e}_{i,j}\|_\infty \leq \alpha \cdot \|\mathbf{x}_{i,j}\|_\infty \leq \alpha \cdot \|\mathbf{x}_i\|_\infty \leq tn(n+1)B(\lambda) \cdot \|\mathbf{x}_i\|_\infty$, by Equation (3). To establish correctness, it suffices to prove the following.

Lemma 14 (Approximate Correctness)**.** *For every $i \in [r]$:*

$$\sum_{j=i}^{r-1} \prod_{k=1}^{j} \mathbf{P}_k \cdot \mathbf{v}_j + \mathbf{w} = \left(\prod_{k=1}^{i} \mathbf{P}_k\right) \cdot (\mathbf{x}_i \otimes \mathbf{y}_i) + \widetilde{\mathbf{e}}_i$$

where $\|\widetilde{\mathbf{e}}_i\|_\infty \leq \alpha \cdot \left(\sum_{k=i}^{r-1} (t \cdot n^3)^k\right) \cdot \|\mathbf{x}\|_\infty$.

Before proceeding with the proof, we can indeed see that, setting $i = 0$, we obtain:

$$\mathbf{v} + \mathbf{w} = (\mathbf{x} \otimes \mathbf{y}) + \mathbf{e}$$

where $\|\mathbf{e}\|_\infty \leq \alpha \cdot (t \cdot n^3)^r \cdot \|\mathbf{x}\|_\infty$. Thus, the scheme satisfies $\alpha(t \cdot n^3)^r$-correctness.

Proof. We proceed by induction starting from $i = r-1$ and going all the way down to $i = 0$. It is easy to see that $\mathbf{v}_{r-1} = \mathsf{hOTE.HashEval}(\mathsf{pk}, E_{r-1}, \psi_{r-1,0})$, whereas $d = \mathbf{d}_{r-1,0}$. Therefore, by Equation (7), we obtain:

$$\begin{aligned}\mathbf{v}_{r-1} \cdot \prod_{k=1}^{r-1} \mathbf{P}_k + \mathbf{w} &= \left(\prod_{k=1}^{r-1} \mathbf{P}_k\right) \cdot \Big(\mathsf{hOTE.HashEval}(\mathsf{pk}, E_{r-1}, \psi_{r-1,0}) \\ &\quad + \mathsf{hOTE.EncEval}(\mathsf{pk}, d, \phi_r)\Big) \\ &= \left(\prod_{k=1}^{r-1} \mathbf{P}_k\right) \cdot (\mathbf{x}_{r-1,0} \otimes \mathbf{y}_{r-1}) + \left(\prod_{k=1}^{r-1} \mathbf{P}_k\right) \cdot \mathbf{e}_{r-1,0} \\ &= \left(\prod_{k=1}^{r-1} \mathbf{P}_k\right) \cdot (\mathbf{x}_{r-1} \otimes \mathbf{y}_{r-1}) + \widetilde{\mathbf{e}}_{r-1}\end{aligned}$$

where $\widetilde{\mathbf{e}}_{r-1} := \left(\prod_{k=1}^{r-1} \mathbf{P}_k\right) \cdot \mathbf{e}_{r-1,0}$. Notice that

$$\begin{aligned}\|\widetilde{\mathbf{e}}_{r-1}\|_\infty &\leq n^{2(r-1)} \cdot \|\mathbf{e}_{r-1,0}\|_\infty \\ &\leq \alpha n^{2(r-1)} \cdot \|\mathbf{x}_{r-1}\|_\infty \\ &\leq \alpha (tn^3)^{r-1} \cdot \|\mathbf{x}\|_\infty\end{aligned}$$

by Equation (5) and Equation (6). This establishes the base case. For the inductive step, we first appeal to the induction hypothesis to show that:

$$\begin{aligned}\sum_{j=i-1}^{r-1} \left(\prod_{k=1}^{j} \mathbf{P}_k\right) \cdot \mathbf{v}_j + \mathbf{w} &= \left(\prod_{k=1}^{i-1} \mathbf{P}_k\right) \cdot \mathbf{v}_{i-1} + \left(\prod_{k=1}^{i} \mathbf{P}_k\right) \cdot (\mathbf{x}_i \otimes \mathbf{y}_i) + \widetilde{\mathbf{e}}_i \\ &= \left(\prod_{k=1}^{i-1} \mathbf{P}_k\right) \cdot \Big(\mathbf{v}_{i-1} + \mathbf{P}_i \cdot (\mathbf{x}_i \otimes \mathbf{y}_i)\Big) + \widetilde{\mathbf{e}}_i.\end{aligned}$$

Moreover:

$$\begin{aligned}\mathbf{P}_i \cdot (\mathbf{x}_i \otimes \mathbf{y}_i) &= \begin{pmatrix} \mathbf{P} \cdot (\mathbf{d}_{i-1,0} \otimes \phi_i \otimes \mathbf{g}_q) \\ \mathbf{P} \cdot (\mathbf{d}_{i-1,1} \otimes \phi_i \otimes \mathbf{g}_q) \\ \vdots \\ \mathbf{P} \cdot (\mathbf{d}_{i-1,t^{r-i}-1} \otimes \phi_i \otimes \mathbf{g}_q) \end{pmatrix} \\ &= \begin{pmatrix} \mathsf{hOTE.EncEval}(\mathsf{pk}, \mathbf{d}_{i-1,0}, \phi_i) \\ \mathsf{hOTE.EncEval}(\mathsf{pk}, \mathbf{d}_{i-1,1}, \phi_i) \\ \vdots \\ \mathsf{hOTE.EncEval}(\mathsf{pk}, \mathbf{d}_{i-1,t^{r-i}-1}, \phi_i) \end{pmatrix}\end{aligned}$$

We also observe that:

$$\mathbf{v}_{i-1} = \begin{pmatrix} \mathbf{v}_{i-1,0} \\ \mathbf{v}_{i-1,1} \\ \vdots \\ \mathbf{v}_{i-1,t^{r-i}-1} \end{pmatrix} = \begin{pmatrix} \mathsf{hOTE.EncHash}(\mathsf{pk}, E_{i-1}, \psi_{i-1,0}) \\ \mathsf{hOTE.EncHash}(\mathsf{pk}, E_{i-1}, \psi_{i-1,1}) \\ \vdots \\ \mathsf{hOTE.EncHash}(\mathsf{pk}, E_{i-1}, \psi_{i-1,t^{r-i}-1}) \end{pmatrix}.$$

Furthermore, by Equation (7), we have:

$$\mathbf{v}_{i-1} + \mathbf{P}_i \cdot (\mathbf{x}_i \otimes \mathbf{y}_i) = \begin{pmatrix} (\mathbf{x}_{i-1,0} \otimes \mathbf{y}_{i-1}) + \mathbf{e}_{i-1,0} \\ (\mathbf{x}_{i-1,1} \otimes \mathbf{y}_{i-1}) + \mathbf{e}_{i-1,1} \\ \vdots \\ (\mathbf{x}_{i-1,t^{r-i}} \otimes \mathbf{y}_{i-1}) + \mathbf{e}_{i-1,t^{r-i}-1} \end{pmatrix}$$
$$= (\mathbf{x}_{i-1} \otimes \mathbf{y}_{i-1}) + \mathbf{e}_{i-1},$$

where $\mathbf{e}_{i-1}^{\intercal} := (\mathbf{e}_{i-1,0}^{\intercal}, \mathbf{e}_{i-1,1}^{\intercal}, \ldots, \mathbf{e}_{i-1,t^{r-i}-1}^{\intercal})$. Notice that $\|\mathbf{e}_{i-1}\|_\infty \leq \alpha \cdot \|\mathbf{x}_{i-1}\|_\infty \leq \alpha \cdot (tn)^{i-1} \cdot \|\mathbf{x}\|_\infty$ by Equation (5) and Equation (6). We conclude that

$$\sum_{j=i-1}^{r-1} \left(\prod_{k=1}^{j} \mathbf{P}_k \right) \cdot \mathbf{v}_j + \mathbf{w} = \left(\prod_{k=1}^{i-1} \mathbf{P}_k \right) \cdot (\mathbf{x}_{i-1} \otimes \mathbf{y}_{i-1}) + \widetilde{\mathbf{e}}_{i-1}$$

where $\widetilde{\mathbf{e}}_{i-1} := \widetilde{\mathbf{e}}_i + \left(\prod_{k=1}^{i-1} \mathbf{P}_k \right) \cdot \mathbf{e}_{i-1}$. Observe that

$$\|\widetilde{\mathbf{e}}_{i-1}\|_\infty \leq \|\widetilde{\mathbf{e}}_i\|_\infty + \alpha \cdot (t \cdot n^3)^{i-1} \cdot \|\mathbf{x}\|_\infty \leq \alpha \cdot \left(\sum_{k=i-1}^{r-1} (t \cdot n^3)^k \right) \cdot \|\mathbf{x}\|_\infty$$

as desired. □

Encoder Privacy. The following theorem establishes encoder privacy.

Theorem 15. *Assuming that* hOTE *is encoder private, Construction 2 is encoder private.*

Proof. The proof follows by a standard hybrid argument, where we gradually substitute the encodings $\{E_j\}_{j \in [r]}$ with the outputs of the simulator $\mathsf{Sim}(1^\lambda, \mathsf{pk})$, provided by the definition of encoder privacy of hOTE. □

How to Build a Fully Succinct OTE for $\mathbb{Z}_q^m \otimes \mathbb{Z}_q^\ell$. Construction 2 presents a non-interactive OTE for $\mathbb{Z}_q^m \otimes \mathbb{Z}_q^n$ where the hash size is a vector in $\mathbb{Z}_q^n$ and the encoding consists of r matrices of size $(tn) \times (tn^2)$. A trivial way to build a fully succinct OTE for $\mathbb{Z}_q^m \otimes \mathbb{Z}_q^\ell$ is to instantiate Construction 2 with $n := \ell$. If $\ell = O(\lambda \cdot \log q)$, this is secure, however, the encoder size would scale as $r \cdot \ell^3$ while the hash size would be linear in ℓ. We can do better: just split $\mathbf{y}$ into blocks $\mathbf{y}_0, \ldots, \mathbf{y}_N$ of size n. Then, compute an OTE encoding of each of them using Construction 2 and send them to the hasher. Given the hasher's digest (notice, a single digest is sufficient for all $\mathbf{y}_0, \ldots, \mathbf{y}_N$), we can then derive shares for $\mathbf{x} \otimes \mathbf{y}_0, \ldots, \mathbf{x} \otimes \mathbf{y}_N$. By reordering these, we obtain a secret-sharing of $\mathbf{x} \otimes \mathbf{y}$.

Parameters. Let $q = q(\lambda)$ be a power of 2, let $\chi = \chi(\lambda)$ be a $B(\lambda)$-bounded error distribution, and let $k = k(\lambda)$ be such that the (χ, k, m, q)-LWE problem is hard. Set $n := k \log q$ and fix integers $t = t(\lambda) \geq 2$ and $r = r(\lambda) \geq 1$ with $m = t^r \cdot n$. Instantiate Construction 2 using as underlying half-succinct NI-OTE the scheme of Construction 1 for $\mathbb{Z}_q^{tn} \otimes \mathbb{Z}_q^n$, we obtain the following parameters:

- (Public Key) The public is exactly the CRS of the underlying half-succinct scheme, which consists of $\mathbf{A} \in \mathbb{Z}_2^{n \times tn}$ and $\mathbf{B} \in \mathbb{Z}_2^{n \times tn^3}$.
- (Digest) The digest is a vector in $\mathbb{Z}_q^n$ of ℓ_∞-norm at most $m^r \cdot \|\mathbf{x}\|_\infty$.
- (Encoding) Write $N := \lceil \ell/n \rceil$ and partition $\mathbf{y}$ into N blocks of length at most n. The encoding consists of N independent encodings produced by Construction 2, each such encoding is a tuple of r matrices of size $(tn) \times (tn^2)$. Therefore the total encoding size is $\ell r t^2 n^2$ elements of $\mathbb{Z}_q$.

Moreover, Construction 1 on inputs $\mathbb{Z}_q^{tn} \otimes \mathbb{Z}_q^n$ is $tn(n+1)B(\lambda)$-correct. Hence, by Lemma 14, Construction 2 is α-correct with

$$\alpha := \alpha_0 \cdot (tn^3)^r = tn(n+1)B(\lambda)(tn^3)^r$$

Thus it suffices to set $q/p > 2\alpha$ to obtain a perfectly correct scheme. For $t = \Theta(1)$, then $r = \Theta(\log m)$, the digest size and CRS size are $\mathsf{poly}(\lambda, \log m)$, and the encoding size is $\ell \cdot \mathsf{poly}(\lambda, \log m)$. Thus, we obtain the following theorem.

Theorem 16 (Fully-Succinct NI-OTE)**.** *Assume the hardness of LWE, there exists an encoder-private programmable NI-OTE for $\mathbb{Z}_q^m \otimes \mathbb{Z}_q^\ell$ with digest and public key size $\mathsf{poly}(\lambda, \log m)$ and encoding size $\ell \cdot \mathsf{poly}(\lambda, \log m)$.*

3.4 How to Achieve Hasher-Privacy

The constructions presented so far rely on a deterministic hasher, hence, they are not hasher private. There is however an easy way to achieve this property by relying on a non-succinct NI-OTE protocol that provides privacy at both ends. The idea is to compose the latter with the fully succinct OTE to set up a (possibly noisy) additive secret-sharing of $d \otimes \phi \otimes \mathbf{g}_q$. Specifically, instead of sending d in the clear, the hasher inputs d in the hasher-private non-succinct NI-OTE. Simultaneously, the encoder inputs $\phi \otimes \mathbf{g}_q$ and sends the fully succinct OTE encoding E along with it. Using ψ and E, the hasher can derive the output share of the fully succinct OTE as usual. Moreover, by relying on the fact that the encoder output in the fully succinct OTE is just

$$\mathbf{w} := \left(\prod_{i=1}^{r-1} \mathbf{P}_i \right) \cdot \mathbf{P} \cdot (d \otimes \phi \otimes \mathbf{g}_q),$$

where each $\mathbf{P}_i$ is a public low-norm matrix, the parties can bootstrap the (noisy) secret sharing of $d \otimes \phi \otimes \mathbf{g}_q$ into a (noisy) secret-sharing of $\mathbf{w}$. Putting all these shares together provides a (noisy) secret-sharing of $\mathbf{x} \otimes \mathbf{y}$.

Notice that the hasher-privacy of the non-succinct OTE guarantees the privacy of $\mathbf{x}$, whereas the encoder privacy of both the non-succinct and the fully-succinct OTE ensure that $\mathbf{y}$ remains secret. Finally, the protocol remains fully succinct. Indeed, the non-succinct OTE contributes only with a $\mathsf{poly}(\lambda, |d|, |\phi|) = \mathsf{poly}(\lambda)$ additive overhead. Also the CRS size increases only by a $\mathsf{poly}(\lambda)$ amount.

A Non-Succinct OTE Protocol with Hasher and Encoder Privacy. We sketch how to build an LWE-based non-succinct OTE protocol with privacy at both sides. This can be easily done by following the blueprint of [1, 3, 20].

Construction 3: Fully Private, Non-Succinct NI-OTE

$\mathsf{Setup}(1^\lambda)$**:** Sample $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{k\times m}$ and $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_q^{k\times 2n}$ and return $\mathsf{pk} := (\mathbf{A}, \mathbf{B})$.

$\mathsf{Hash}(\mathsf{pk}, \mathbf{x})$**:** Compute $\mathbf{d} := \mathbf{A}\cdot\mathbf{x} + \mathbf{B}\cdot\mathbf{r}$ where $\mathbf{r} \xleftarrow{\$} \mathbb{Z}_2^{2n}$ and return $d := \mathbf{d}$ and $\psi := (\mathbf{x}, \mathbf{r})$.

$\mathsf{Enc}(\mathsf{pk}, \mathbf{y})$**:** Sample $\mathbf{S} \xleftarrow{\$} \mathbb{Z}_q^{k\times mn}$, $\mathbf{E} \xleftarrow{\$} \chi(\lambda)$, and $\hat{\mathbf{E}} \xleftarrow{\$} \chi(\lambda)$. Compute

$$\mathbf{C} := \mathbf{A}^\intercal\mathbf{S} + \mathbf{E} + \mathbf{I}_m \otimes \mathbf{y}^\intercal,$$
$$\hat{\mathbf{C}} := \mathbf{B}^\intercal\mathbf{S} + \hat{\mathbf{E}}.$$

Return $E := (\mathbf{C}, \hat{\mathbf{C}})$ and $\phi := \mathbf{S}$.

$\mathsf{HashEval}(\mathsf{pk}, E, \psi)$**:** Return $\mathbf{v} := \mathbf{C}^\intercal\cdot\mathbf{x} + \hat{\mathbf{C}}^\intercal\cdot\mathbf{r}$.

$\mathsf{EncEval}(\mathsf{pk}, d, \phi)$**:** Return $\mathbf{w} := -\mathbf{S}^\intercal\cdot\mathbf{d}$.

Observe that in Construction 3, we have that

$$\begin{aligned}\mathbf{w}+\mathbf{v} &= \mathbf{C}^\intercal\cdot\mathbf{x} + \hat{\mathbf{C}}^\intercal\cdot\mathbf{r} - \mathbf{S}^\intercal\cdot\mathbf{d}\\ &= \mathbf{S}^\intercal\mathbf{A}\mathbf{x} + \mathbf{E}^\intercal\cdot\mathbf{x} + \mathbf{x}\otimes\mathbf{y} + \mathbf{S}^\intercal\mathbf{B}\mathbf{r} + \hat{\mathbf{E}}^\intercal\cdot\mathbf{r} - \mathbf{S}^\intercal(\mathbf{A}\mathbf{x}+\mathbf{B}\mathbf{r})\\ &= \mathbf{x}\otimes\mathbf{y} + \mathbf{E}^\intercal\cdot\mathbf{x} + \hat{\mathbf{E}}^\intercal\cdot\mathbf{r},\end{aligned}$$

and $\|\mathbf{E}^\intercal\cdot\mathbf{x} + \hat{\mathbf{E}}^\intercal\cdot\mathbf{r}\|_\infty \le mB(\lambda)\cdot\|\mathbf{x}\|_\infty + 2nB(\lambda)$.

Hasher privacy is an immediate application of the leftover hash lemma, whereas encoder privacy derives from the fact that $\mathbf{A}^\intercal\mathbf{S}+\mathbf{E}$ and $\mathbf{B}^\intercal\mathbf{S}+\hat{\mathbf{E}}$ look random under LWE. Notice that while the hasher message is succinct, the encoder message has size $m^2 n\log q + 2mn^2\log q$ where m is the length of the input $\mathbf{x}$. Finally, notice that we can easily modify this construction to achieve α-approximate correctness for $\alpha = 0$ using the same rounding trick we mentioned in Section 3.1.

3.5 From Succinct NI-OTE to Succinct NI-MOLE

We will also consider an modification of NI-OTE where instead of computing a tensor product, we compute a matrix-vector product. More specifically, we let the hasher take as input a matrix $\mathbf{M}$ and the encoder a vector $\mathbf{y}$ and we require that

$$\mathsf{HashEval}(\mathsf{pk}, E, \psi) + \mathsf{EncEval}(\mathsf{pk}, d, \phi) \approx \mathbf{M}\mathbf{y}.$$

Furthermore, we require the protocol to be succinct, in the sense that the communication complexity should be independent of the number of rows of the matrix $\mathbf{M}$. We refer to this protocol as Non-Interactive Matrix Oblivious Linear Evaluation (NI-MOLE).

It is easy to see that one can generically construct an $(\ell \cdot \log q \cdot \alpha)$-correct NI-MOLE from an α-correct NI-OTE: Simply hash $\mathbf{x} := \mathbf{G}^{-1}(\mathsf{vec}(\mathbf{M}))$ and encode $\mathbf{y} \otimes \mathbf{g}_q$. Then the hasher and the encoder return

$$\mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot \mathsf{HashEval}(\mathsf{pk}, E, \psi),$$
$$\text{and}$$
$$\mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot \mathsf{EncEval}(\mathsf{pk}, d, \phi)$$

respectively. By the correctness of the NI-OTE we have:

$$\begin{aligned}
\mathbf{v} + \mathbf{w} &= \mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot (\mathbf{x} \otimes \mathbf{y} \otimes \mathbf{g}_q) + \mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot \mathbf{e} \\
&= (\mathbf{I}_m \otimes \mathbf{y}^\intercal \otimes \mathbf{g}_q^\intercal) \cdot \mathbf{x} + \mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot \mathbf{e} \\
&= (\mathbf{I}_m \otimes \mathbf{y}^\intercal) \cdot \mathsf{vec}(\mathbf{M}^\intercal) + \mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot \mathbf{e} \\
&= \mathbf{M} \cdot \mathbf{y} + \mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot \mathbf{e} \\
&\approx \mathbf{M} \cdot \mathbf{y}
\end{aligned}$$

ignoring low-order error terms, since $\|\mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q})\|_\infty = 1$ by Lemma 4. Notice that

$$\|\mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q}) \cdot \mathbf{e}\|_\infty \leq \ell \cdot \log q \cdot \alpha \cdot \|\mathbf{x}\|_\infty = \ell \cdot \log q \cdot \alpha,$$

as each row of $\mathsf{Lin}(\mathbf{I}_{m \cdot \ell \cdot \log q})$ has Hamming weight $\ell \cdot \log q$. Thus, henceforth we will assume the existence of succinct NI-OTE and succinct NI-MOLE interchangeably, with the understanding the succinct NI-OTE implies the existence of both.

4 Adaptive Lattice Encodings

In the following we present our new construction of adaptive lattice encodings. Let $k := k(\lambda)$ and $q := q(\lambda)$ be positive integers, and let $\mathbf{G}$ be the k-dimensional gadget matrix $\mathbf{G} := \mathbf{I}_k \otimes \mathbf{g}_q^\intercal$. For element $x \in \mathbb{Z}_q$, vectors $\mathbf{s}, \mathbf{r} \in \mathbb{Z}_q^k$, matrix $\mathbf{A} \in \mathbb{Z}_q^{k \times (k \cdot \log q)}$ and noise term $\mathbf{e} \in \mathbb{Z}^{k \cdot \log q}$, we define the corresponding *adaptive lattice encoding* as:

$$\mathsf{LEnc}_{\mathbf{A}}(x; \mathbf{s}, \mathbf{r}, \mathbf{e}) := \mathbf{s}^\intercal \mathbf{A} + x \cdot \mathbf{r}^\intercal \mathbf{G} + \mathbf{e}^\intercal.$$

Note that, for an appropriately sampled $\mathbf{A}$, $\mathbf{s}$, and $\mathbf{e}$. It is straightforward to see that the encoding is computationally close to uniform under the LWE assumption. Next, we demonstrate the homomorphic properties of such encodings.

Construction 4: Homomorphic Operations on Lattice Encodings

Addition: For every $x_0, x_1 \in \mathbb{Z}_q$, vectors $\mathbf{s}, \mathbf{r} \in \mathbb{Z}_q^k$, matrices $\mathbf{A}_0, \mathbf{A}_1 \in \mathbb{Z}_q^{k \times (k \cdot \log q)}$ and noise terms $\mathbf{e}_0, \mathbf{e}_1 \in \mathbb{Z}_q^{k \cdot \log q}$, we have:

$$\mathsf{LEnc}_{\mathbf{A}_0}(x_0; \mathbf{s}, \mathbf{r}, \mathbf{e}_0) + \mathsf{LEnc}_{\mathbf{A}_1}(x_1; \mathbf{s}, \mathbf{r}, \mathbf{e}_1) = \mathsf{LEnc}_{\mathbf{A}_0 + \mathbf{A}_1}(x_0 + x_1; \mathbf{s}, \mathbf{r}, \mathbf{e}_0 + \mathbf{e}_1).$$

Scalar Multiplication: For every $x, \delta \in \mathbb{Z}_q$, vectors $\mathbf{s}, \mathbf{r} \in \mathbb{Z}_q^k$, matrix $\mathbf{A} \in \mathbb{Z}_q^{k \times (k \cdot \log q)}$ and noise terms $\mathbf{e} \in \mathbb{Z}_q^{k \cdot \log q}$, we have:

$$\mathsf{LEnc}_{\mathbf{A}}(x; \mathbf{s}, \mathbf{r}, \mathbf{e}) \cdot \mathbf{G}^{-1}(\delta \cdot \mathbf{G}) = \mathsf{LEnc}_{\mathbf{A} \cdot \mathbf{G}^{-1}(\delta \cdot \mathbf{G})}(x \cdot \delta; \mathbf{s}, \mathbf{r}, \mathbf{G}^{-1}(\delta \cdot \mathbf{G})^{\intercal} \cdot \mathbf{e}).$$

Multiplication: For every $x_0, x_1 \in \mathbb{Z}_q$, vectors $\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2 \in \mathbb{Z}_q^k$, matrices $\mathbf{A}_0, \mathbf{A}_1 \in \mathbb{Z}_q^{k \times (k \cdot \log q)}$ and noise terms $\mathbf{e}_0, \mathbf{e}_1 \in \mathbb{Z}_q^{k \cdot \log q}$, we have:

$$\begin{aligned} &- \mathsf{LEnc}_{\mathbf{A}_0}(x_0; \mathbf{s}_0, \mathbf{s}_1, \mathbf{e}_0) \cdot \mathbf{G}^{-1}(\mathbf{A}_1) + x_0 \cdot \mathsf{LEnc}_{\mathbf{A}_1}(x_1; \mathbf{s}_1, \mathbf{s}_2, \mathbf{e}_1) \\ &\quad = \mathsf{LEnc}_{-\mathbf{A}_0 \cdot \mathbf{G}^{-1}(\mathbf{A}_1)}(x_0 \cdot x_1; (\mathbf{s}_0, \mathbf{s}_2, -\mathbf{G}^{-1}(\mathbf{A}_1)^{\intercal} \cdot \mathbf{e}_0 + x_0 \cdot \mathbf{e}_1)). \end{aligned}$$

4.1 Homomorphic Operations

We show that our lattice encodings support addition, scalar multiplication, and even multiplication, provided that the encodings are encrypted with correlated secrets. We present a formal description of the algorithms below.

We elaborate on the correctness of the claimed homomorphic operations. For addition, we observe that:

$$\begin{aligned} \mathsf{LEnc}_{\mathbf{A}_0}(x_0; \mathbf{s}, \mathbf{r}, \mathbf{e}_0) + \mathsf{LEnc}_{\mathbf{A}_1}(x_1; \mathbf{s}, \mathbf{r}, \mathbf{e}_1) &= \mathbf{s}^{\intercal}\mathbf{A}_0 + x_0 \cdot \mathbf{r}^{\intercal}\mathbf{G} + \mathbf{e}_0^{\intercal} + \mathbf{s}^{\intercal}\mathbf{A}_1 + x_1 \cdot \mathbf{r}^{\intercal}\mathbf{G} + \mathbf{e}_1^{\intercal} \\ &= \mathbf{s}^{\intercal} \cdot (\mathbf{A}_0 + \mathbf{A}_1) + (x_0 + x_1) \cdot \mathbf{r}^{\intercal}\mathbf{G} + (\mathbf{e}_0^{\intercal} + \mathbf{e}_1^{\intercal}) \\ &= \mathsf{LEnc}_{\mathbf{A}_0 + \mathbf{A}_1}(x_0 + x_1; \mathbf{s}, \mathbf{r}, \mathbf{e}_0 + \mathbf{e}_1). \end{aligned}$$

For scalar multiplication, we have:

$$\begin{aligned} \mathsf{LEnc}_{\mathbf{A}}(x; \mathbf{s}, \mathbf{r}, \mathbf{e}) \cdot \mathbf{G}^{-1}(\delta \cdot \mathbf{G}) &= \mathbf{s}^{\intercal}\mathbf{A}\mathbf{G}^{-1}(\delta \cdot \mathbf{G}) + x \cdot \mathbf{r}^{\intercal}\mathbf{G}\mathbf{G}^{-1}(\delta \cdot \mathbf{G}) + \mathbf{e}^{\intercal}\mathbf{G}^{-1}(\delta \cdot \mathbf{G}) \\ &= \mathbf{s}^{\intercal}\mathbf{A}\mathbf{G}^{-1}(\delta \cdot \mathbf{G}) + (x \cdot \delta) \cdot \mathbf{r}^{\intercal}\mathbf{G} + \mathbf{e}^{\intercal}\mathbf{G}^{-1}(\delta \cdot \mathbf{G}) \\ &= \mathsf{LEnc}_{\mathbf{A} \cdot \mathbf{G}^{-1}(\delta \cdot \mathbf{G})}(x \cdot \delta; \mathbf{s}, \mathbf{r}, \mathbf{G}^{-1}(\delta \cdot \mathbf{G})^{\intercal} \cdot \mathbf{e}). \end{aligned}$$

Finally, for homomorphic multiplication, we have:

$$\begin{aligned} &- \mathsf{LEnc}_{\mathbf{A}_0}(x_0; \mathbf{s}_0, \mathbf{s}_1, \mathbf{e}_0) \cdot \mathbf{G}^{-1}(\mathbf{A}_1) + x_0 \cdot \mathsf{LEnc}_{\mathbf{A}_1}(x_1; \mathbf{s}_1, \mathbf{s}_2, \mathbf{e}_1) \\ &\quad = -(\mathbf{s}_0^{\intercal}\mathbf{A}_0 + x_0 \cdot \mathbf{s}_1^{\intercal}\mathbf{G} + \mathbf{e}_0^{\intercal}) \cdot \mathbf{G}^{-1}(\mathbf{A}_1) + x_0 \cdot (\mathbf{s}_1^{\intercal}\mathbf{A}_1 + x_1 \cdot \mathbf{s}_2^{\intercal}\mathbf{G} + \mathbf{e}_1^{\intercal}) \\ &\quad = \mathbf{s}_0^{\intercal} \cdot (-\mathbf{A}_0\mathbf{G}^{-1}(\mathbf{A}_1)) - x_0 \cdot \mathbf{s}_1^{\intercal}\mathbf{A}_1 + x_0 \cdot \mathbf{s}_1^{\intercal}\mathbf{A}_1 + (x_0 \cdot x_1) \cdot \mathbf{s}_2^{\intercal}\mathbf{G} - \mathbf{e}_0^{\intercal}\mathbf{G}^{-1}(\mathbf{A}_1) + x_0 \cdot \mathbf{e}_1^{\intercal} \\ &\quad = \mathsf{LEnc}_{-\mathbf{A}_0 \cdot \mathbf{G}^{-1}(\mathbf{A}_1)}(x_0 \cdot x_1; (\mathbf{s}_0, \mathbf{s}_2, -\mathbf{G}^{-1}(\mathbf{A}_1)^{\intercal} \cdot \mathbf{e}_0 + x_0 \cdot \mathbf{e}_1)). \end{aligned}$$

Evaluating RMS Programs. From the homomorphic operations described above, one can derive a general routine to evaluate any T-bounded (i.e., the maximum norm of an intermediate variable of the computation is bounded by T) RMS program of depth d. Recall that in an RMS program, one can sum any two variables, whereas multiplication can be done only so long as one of the two variables is an input. Before discussing the evaluation algorithm, let us generalize the notation described above to vectors, with:

$$\mathsf{LEnc}_{\mathbf{A}}(\mathbf{x}; \mathbf{s}, \mathbf{r}, \mathbf{e}) := \mathbf{s}^{\intercal}\mathbf{A} + \mathbf{r}^{\intercal}(\mathbf{x}^{\intercal} \otimes \mathbf{G}) + \mathbf{e}^{\intercal}$$

by increasing the dimensions of the components appropriately. We refer to $\mathbf{s}$ as the *encryption key* and $\mathbf{r}$ as the *authentication key*. Homomorphic operations can be extended to vectors in a straightforward manner. We are now ready to state the algorithm to evaluate RMS programs.

Lemma 17 (Evaluation of RMS Programs)**.** *Let $\mathbf{A} \in \mathbb{Z}_q^{k \times (m \cdot k \cdot \log q)}$ be a matrix, $\mathbf{x} \in \mathbb{Z}_q^{m-1}$ be an input, and let f be a T-bounded depth-$d(\lambda)$ RMS program. Define $\hat{\mathbf{x}}$ as the vertical concatenation of $\mathbf{x}$ and 1. For all $i \in [d]$, let:*

$$\mathbf{c}_i^{\intercal} := \mathsf{LEnc}_{\mathbf{A}}(\hat{\mathbf{x}}; \mathbf{s}_i, \mathbf{s}_{i+1}, \mathbf{e}_i)$$

with $\mathbf{s}_i \in \mathbb{Z}_q^k$ and such that $\max_i \|\mathbf{e}_i\|_\infty \leq \beta$. Then there exist two polynomial-time algorithms $\mathsf{EvalRMSK}$ *and* $\mathsf{EvalRMSC}$ *such that:*

$$\mathbf{A}_f \leftarrow \mathsf{EvalRMSK}(\mathbf{A}, f) \quad \textit{and} \quad \tilde{\mathbf{c}} \leftarrow \mathsf{EvalRMSC}(\mathbf{A}, f, \mathbf{x}, \{\mathbf{c}_i\}_{i \in [d]})$$

with $\tilde{\mathbf{c}}^{\intercal} \in \mathsf{LEnc}_{\mathbf{A}_f}(f(\mathbf{x}), \mathbf{s}_0, \mathbf{s}_d, \tilde{\mathbf{e}})$ such that $\|\tilde{\mathbf{e}}\|_\infty \leq \beta \cdot O(T \cdot (k \cdot \log q)^d)$.

Proof. We refer to any encoding using $\mathbf{s_i}$ as encryption key and $\mathbf{s_j}$ as authentication key a level-(i, j) encoding. Observe that splitting a level-$(i, i+1)$ encoding $\mathbf{c}_i$ into blocks of dimension $k \cdot \log q$, we obtain level-$(i, i+1)$ encodings:

$$\mathsf{LEnc}_{\mathbf{A}_j}(x_j; \mathbf{s}_i, \mathbf{s}_{i+1}, \mathbf{e}_{i,j})$$

where x_j is the j-th entry of $\hat{\mathbf{x}}$, $\mathbf{A}_j$ is the j-th block of $\mathbf{A}$, and $\mathbf{e}_{i,j}$ is the j-th block of $k \cdot \log q$ entries in $\widetilde{\mathbf{e}}_i$).

Using the addition and multiplication by a constant, we can apply linear operations over encodings lying on the same level (i, j). In this way, we obtain a level-(i, j) encoding of the result. On the other hand, the operation increases the norm of the noise in the encodings: If the linear operation is described by a vector $\boldsymbol{\ell}$, the noise magnitude increases by a factor $\sum_v \log \ell_v$. We can also homomorphically compute multiplications between any encoding on level (i, j) and any encoding on level $(j, j+1)$. In this way, we obtain a level-$(i, j+1)$ encoding of the product. This time the noise magnitude increases by a factor $O(k \cdot \log q)$ and by an additive term $T \cdot \beta$ for each multiplication. Finally, we observe that we can convert a level-(i, j) encoding into a level $(i, j+1)$ encoding by simply multiplying by a level-$(j, j+1)$ encoding of 1. Notice that the latter is know given that the last entry of $\hat{\mathbf{x}}$ is a 1. Overall, the growth of the noise norm is bounded by a factor $\beta \cdot O(T \cdot (k \cdot \log q)^d)$.

We also highlight that for all these operations we described, we are able to derive the matrix underlying the output encodings from the matrices underlying the input encodings. Thus both algorithms are well-defined. □

4.2 Compressing Lattice Encodings

We describe a procedure to compress and decompress lattice encodings. Formally, this consists of a triple of algorithms $(\mathsf{Setup}, \mathsf{Compress}, \mathsf{Expand})$ that allows one to sample a compressed version of a lattice encoding, that can be later on expanded into the format described above.

Let $n := n(\lambda), k := k(\lambda)$, $m := m(\lambda)$, and $q := q(\lambda)$ be positive integers. We are going to assume the existence of an α-correct succinct NI-OTE protocol $\mathsf{OTE} = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval})$ with bilinear encoder evaluation for $\mathbb{Z}_q^m \otimes \mathbb{Z}_q^{k \cdot \log q}$ where digests are vectors in $\mathbb{Z}_q^n$ and the encoder private information consists of a vector in $\mathbb{Z}_q^k$. Let $\mathbf{G} := \mathbf{I}_k \otimes \mathbf{g}_q^\intercal$. Our scheme is formally described in Construction 5. Observe that if we instantiate OTE with Construction 2, the size of the compressed encoding scales logarithmically in the size of its input.

Construction 5: Compression of Lattice Encodings

$\mathsf{Setup}(1^\lambda)$**:** Return $\mathsf{ck} := \mathsf{pk} \xleftarrow{\$} \mathsf{OTE.Setup}(1^\lambda)$.

$\mathsf{Compress}(\mathsf{ck}, \mathbf{A}, \mathbf{x}, \mathbf{s}_0, \mathbf{s}_1, \mathbf{r})$**:** Compute $(\mathbf{d}, \psi) := \mathsf{OTE.Hash}(\mathsf{pk}, \mathbf{x})$, then sample $\mathbf{e} \xleftarrow{\$} \chi(\lambda)$. Compute

$$\mathbf{h}^\intercal := \mathbf{s}_0^\intercal \mathbf{A} + \mathbf{r}^\intercal (\mathbf{d}^\intercal \otimes \mathbf{G}) + \mathbf{e}^\intercal$$

and set $E \xleftarrow{\$} \mathsf{OTE.Enc}(\mathsf{pk}, \mathbf{s}_1 \otimes \mathbf{g}_q, \mathbf{r})$. Return $(\mathbf{h}, E)$.

$\mathsf{Expand}(\mathsf{ck}, \mathbf{A}, \mathbf{h}, E, \mathbf{x})$**:** Recompute the hash $(\mathbf{d}, \psi) := \mathsf{OTE.Hash}(\mathsf{pk}, \mathbf{x})$ and set $\mathbf{v} := \mathsf{OTE.HashEval}(\mathsf{pk}, E, \psi)$. Let $\mathbf{P}$ be the matrix of the OTE protocol that can be publicly derived from pk. Return

$$\mathbf{c}^\intercal := \mathbf{h}^\intercal \mathbf{P}^\intercal + \mathbf{v}^\intercal.$$

We show that the expansion algorithm indeed leads to well-formed lattice encoding. By the correctness of the NI-OTE protocol, we have that:

$$\mathbf{w} := \mathsf{OTE.EncEval}(\mathsf{pk}, \mathbf{d}, \mathbf{r}) = \mathbf{P} \cdot (\mathbf{d} \otimes \mathbf{r} \otimes \mathbf{g}_q).$$

Then let us rewrite:

$$\begin{aligned}
\mathbf{c}^\intercal &= \mathbf{h}^\intercal \mathbf{P}^\intercal + \mathbf{v}^\intercal \\
&= \mathbf{s}_0^\intercal \cdot \mathbf{A} + \mathbf{r}^\intercal \cdot (\mathbf{d}^\intercal \otimes \mathbf{G}) \cdot \mathbf{P}^\intercal + \mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{v}^\intercal \\
&= \mathbf{s}_0^\intercal \cdot \mathbf{A} \cdot \mathbf{P}^\intercal + \mathbf{r}^\intercal \cdot (\mathbf{d}^\intercal \otimes \mathbf{I}_k \otimes \mathbf{g}_q^\intercal) \cdot \mathbf{P}^\intercal + \mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{v}^\intercal \\
&= \mathbf{s}_0^\intercal \cdot \mathbf{A} \cdot \mathbf{P}^\intercal + (\mathbf{d}^\intercal \otimes \mathbf{r}^\intercal \otimes \mathbf{g}_q^\intercal) \cdot \mathbf{P}^\intercal + \mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{v}^\intercal \\
&= \mathbf{s}_0^\intercal \cdot \mathbf{A} \cdot \mathbf{P}^\intercal + \mathbf{w}^\intercal + \mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{v}^\intercal \\
&= \mathbf{s}_0^\intercal \cdot \mathbf{A} \cdot \mathbf{P}^\intercal + \mathbf{x}^\intercal \otimes (\mathbf{s}_1^\intercal \otimes \mathbf{g}_q^\intercal) + \mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{e}'^\intercal \\
&= \mathbf{s}_0^\intercal \cdot \mathbf{A} \cdot \mathbf{P}^\intercal + \mathbf{s}_1^\intercal \cdot (\mathbf{x}^\intercal \otimes \mathbf{I}_k \otimes \mathbf{g}_q^\intercal) + \mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{e}'^\intercal \\
&= \mathbf{s}_0^\intercal \cdot \mathbf{A} \cdot \mathbf{P}^\intercal + \mathbf{s}_1^\intercal \cdot (\mathbf{x}^\intercal \otimes \mathbf{G}) + \mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{e}'^\intercal \\
&= \mathsf{LEnc}_{\mathbf{A}\mathbf{P}^\intercal}(\mathbf{x}; \mathbf{s}_0, \mathbf{s}_1, \mathbf{e} \cdot \mathbf{P} + \mathbf{e}')
\end{aligned}$$

by the α-correctness of the NI-OTE. We can bound the norm of the noise term by:

$$\|\mathbf{e}^\intercal \cdot \mathbf{P}^\intercal + \mathbf{e}'^\intercal\|_\infty \leq k \cdot n \cdot \log q \cdot \|\mathbf{e}\|_\infty \cdot \|\mathbf{P}\|_\infty + \|\mathbf{e}'\|_\infty \leq k \cdot n^{2r+1} \cdot \log q \cdot B(\lambda) + \alpha \cdot \|\mathbf{x}\|_\infty.$$

by Equation (5) and by the α-correctness of the NI-OTE. We prove that the compressed encodings satisfy a notion of simulation that we define below.

Theorem 18 (Simulatability)**.** *Consider the following experiment* $\mathsf{CompExp}_{\mathcal{A},\mathsf{CSim}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ *and a simulator* CSim*:*

- *Sample* $\mathsf{ck} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*,* $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{k \times (n \cdot k \cdot \log q)}$*, and* $\mathbf{s}_0, \mathbf{r} \xleftarrow{\$} \mathbb{Z}_q^k$*.*
- *Activate the adversary* $(\mathbf{x}, \mathbf{s}_1, \mathsf{aux}) \leftarrow \mathcal{A}_0(1^\lambda, \mathsf{ck}, \mathbf{A})$*.*
- *Sample* $b \xleftarrow{\$} \{0,1\}$*. If* $b = 0$ *compute:*

  $$(\mathbf{h}_0, E_0) \xleftarrow{\$} \mathsf{Compress}(\mathsf{ck}, \mathbf{A}, \mathbf{x}, \mathbf{s}_0, \mathbf{s}_1, \mathbf{r})$$

  whereas if $b = 1$ *compute:*

  $$(\mathbf{h}_1, E_1) \xleftarrow{\$} \mathsf{CSim}(1^\lambda, \mathsf{ck}, \mathbf{A}).$$

- *Obtain* $b' \leftarrow \mathcal{A}_1(\mathbf{h}_b, E_b, \mathsf{aux})$ *and return* 1 *if and only if* $b = b'$*.*

Then assuming the hardness of LWE and that OTE *is encoder private, there exists a PPT simulator* CSim*, such that for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left| \frac{1}{2} - \Pr\left[\mathsf{CompExp}_{\mathcal{A},\mathsf{CSim}}(1^\lambda) = 1\right] \right| \leq \mathsf{negl}(\lambda)$$

where the probability is taken over the random coins of the experiment.

Proof. Consider the following sequence of hybrid experiments.

- Hybrid $\mathcal{H}_0$: This is the original experiment.
- Hybrid $\mathcal{H}_1$: We provide the adversary $\mathcal{A}_1$ with a pair $(\mathbf{h}, E)$ where $\mathbf{h} \xleftarrow{\$} \mathbb{Z}_q^{n \cdot k \cdot \log q}$ and $E \xleftarrow{\$} \mathsf{OTE.Enc}(\mathsf{pk}, \mathbf{s}_1 \otimes \mathbf{g}_q, \mathbf{r})$ for $\mathbf{r} \xleftarrow{\$} \mathbb{Z}_q^k$.

  Indistinguishability from the previous hybrid follows by a direct reduction to LWE. Indeed, in Hybrid $\mathcal{H}_0$, we have that $\mathbf{h} = \mathbf{A}^\intercal \cdot \mathbf{s}_0 + \mathbf{e} + (\mathbf{d} \otimes \mathbf{G}^\intercal) \cdot \mathbf{r}$, where in particular $\mathbf{A}^\intercal \cdot \mathbf{s}_0 + \mathbf{e}$ is an LWE sample.
- Hybrid $\mathcal{H}_2$: We also simulate $E \xleftarrow{\$} \mathsf{OTE.Sim}(1^\lambda, \mathsf{pk})$.

  Indistinguishability follows from a straightforward reduction to the encoder privacy of OTE.

We conclude by observing that in Hybrid $\mathcal{H}_2$ the pair $(\mathbf{h}, E)$ given to $\mathcal{A}_1$ is independent of the values $\mathbf{x}$ and $\mathbf{s}_1$ chosen by $\mathcal{A}_0$. This concludes the proof. □

5 Reverse Trapdoor Hashing for all Functions

We construct a selectively-secure reverse TDHs for all functions. Note that, for an expressive enough class of functions, in terms of functionality reverse TDHs are identical to the standard notion of TDH [17], since one can always encode the universal circuit as the input, and vice-versa. However, since the encoding key grows with the size of the input, embedding a universal circuit introduces a dependency in the size of the function. Thus, we pay a price in succinctness, when going from TDH to reverse TDH. On the other hand, the opposite direction (reverse TDH $\implies$ TDH) has no such problem, since the size of the hash is anyway constant. Thus, reverse TDH appears to be a more powerful abstraction.

5.1 Definition

Definition 19 (Reverse Trapdoor Hashing)**.** *A reverse trapdoor hashing scheme for the function class $\mathcal{F} = (\mathcal{F}_\lambda)_{\lambda \in \mathbb{N}}$ with input size $m(\lambda)$ and output of size $\ell(\lambda)$ consists of a tuple of PPT algorithms* $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ *with the following syntax:*

$\mathsf{Setup}(1^\lambda)$**:** *The setup algorithm is randomised and takes as input the security parameter 1^λ. The output is a hash key* hk*.*

$\mathsf{Hash}(\mathsf{hk}, f)$**:** *The hashing algorithm is randomised takes as input a hash key* hk *and a function $f \in \mathcal{F}_\lambda$. The output is a digest d and hasher's private information ρ.*

$\mathsf{Gen}(\mathsf{hk}, \mathbf{x})$**:** *The generation algorithm is randomised and takes as input an hash key* hk*, an element $\mathbf{x} \in \mathbb{Z}_2^m$. The output is an encoding key* ek *and a trapdoor* td*.*

$\mathsf{Enc}(\mathsf{hk}, \mathsf{ek}, f, \rho)$**:** *The encoding algorithm is deterministic and takes as input a hash key* hk*, an encoding key* ek *and a function* $f \in \mathcal{F}_\lambda$ *and hasher's private information* ρ*. The output is an encoding* $\mathbf{e} \in \mathbb{Z}_2^\ell$*.*

$\mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d)$**:** *The decoding procedure is deterministic and takes as input a hash key* hk*, a trapdoor* td *and a digest* d*. The output is an encoding* $\mathbf{e}' \in \mathbb{Z}_2^\ell$*.*

Definition 20 (Correctness)**.** *We say that a reverse TDH scheme is correct if there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every valid adversary*[5] $\mathcal{A}$ *and sufficiently large* $\lambda \in \mathbb{N}$*, it holds that:*

$$\Pr\left[\mathsf{Enc}(\mathsf{hk}, \mathsf{ek}, f, \rho) \oplus \mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d) \neq f(\mathbf{x})\right] \leq \mathsf{negl}(\lambda)$$

where the probability is taken over the random choice of $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*,* $(\mathbf{x}, f) \xleftarrow{\$} \mathcal{A}(1^\lambda, \mathsf{hk})$*,* $(d, \rho) \xleftarrow{\$} \mathsf{Hash}(\mathsf{hk}, f)$*, and* $(\mathsf{ek}, \mathsf{td}) \xleftarrow{\$} \mathsf{Gen}(\mathsf{hk}, \mathbf{x})$*.*

Definition 21 (Selective Function Privacy)**.**

Consider the following experiment $\mathsf{FuncExp}_{\mathcal{A}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$*:*

- *Activate the adversary* $(f_0, f_1, \mathsf{aux}) \xleftarrow{\$} \mathcal{A}_0(1^\lambda)$*.*
- *Sample a hash key* $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*.*
- *Sample a random bit* $b \xleftarrow{\$} \{0, 1\}$*.*
- *Compute* $(d, \rho) \xleftarrow{\$} \mathsf{Hash}(\mathsf{hk}, f_b)$*.*
- *Compute* $b' \leftarrow \mathcal{A}_1(\mathsf{hk}, d, \mathsf{aux})$*.*
- *Return* 1 *if and only if* $b = b'$*.*

We say that a reverse trapdoor hashing scheme $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ *is function private if for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left|\frac{1}{2} - \Pr\left[\mathsf{FuncExp}_{\mathcal{A}}(1^\lambda) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

If the above property holds for every adversary (even computationally unbounded ones) we say that the scheme is statistically function private.

Definition 22 (Selective Input Privacy)**.** *Consider the following experiment* $\mathsf{InpExp}_{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ *and a simulator* Sim*:*

- *Activate the adversary* $(\mathbf{x}, \mathsf{aux}) \xleftarrow{\$} \mathcal{A}_0(1^\lambda)$*.*
- *Sample a hash key* $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*.*

[5] An adversary is valid if it outputs a vector $\mathbf{x} \in \mathbb{Z}_2^m$ and a function $f \in \mathcal{F}_\lambda$.

- *Sample a random bit* $b \xleftarrow{\$} \{0,1\}$.
- *If* $b = 0$ *compute* $(\mathsf{ek}_0, \mathsf{td}_0) \xleftarrow{\$} \mathsf{Gen}(\mathsf{hk}, \mathbf{x})$*, else compute* $\mathsf{ek}_1 \xleftarrow{\$} \mathsf{Sim}(1^\lambda, \mathsf{hk})$.
- *Compute* $b' \leftarrow \mathcal{A}_1(\mathsf{hk}, \mathsf{ek}_b, \mathsf{aux})$.
- *Return* 1 *if and only if* $b = b'$.

We say that a reverse trapdoor hashing scheme $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ *is function-private if there exists a PPT simulator* Sim*, such that for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left| \frac{1}{2} - \Pr\left[\mathsf{InpExp}_{\mathcal{A},\mathsf{Sim}}(1^\lambda) = 1\right] \right| \leq \mathsf{negl}(\lambda).$$

5.2 Laconic Function Evaluation with Pre-Encoding

We now define a new variant of LFE. We ask that the encoding procedure is split into two parts: First, the encoder sends a function-independent pre-encoding of the input, then, once the hash of the function is revealed, it sends an input-independent post-encoding. We also require that the function digest and the post-encoding have a particular structure: the former consists of a matrix, the latter consists of a *LWE-like* sample. Finally, we ask that the output of the decoding procedure is produced by rounding the sum between the post-encoding and a (possibly non-linear) function of the pre-encoding. The reader may have already noticed that these properties are satisfied by many LFE schemes studied in the literature [16, 19, 21, 24].

Definition 23 (Laconic Function Evaluation with Pre-Encoding)**.** *Let* $\mathcal{F} := (\mathcal{F}_\lambda)_{\lambda \in \mathbb{N}}$ *be a family of functions, an LFE scheme consists of a tuple of PPT algorithms* $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{Dec})$ *with the following syntax:*

$\mathsf{Setup}(1^\lambda)$**:** *The probabilistic setup algorithm takes as input the security parameter and outputs a public key* pk*.*

$\mathsf{Hash}(\mathsf{pk}, f)$**:** *The hashing algorithm takes as input a public key* pk *and the description of a function* $f \in \mathcal{F}_\lambda$ *with* $f : \{0,1\}^m \to \{0,1\}$*. The output is a digest* $\mathbf{A}_f \in \mathbb{Z}_q^{k \times (k \cdot \log q)}$ *and hasher's private information* ψ*.*

$\mathsf{Enc}(\mathsf{pk}, \mathbf{A}_f, \mathbf{x})$**:** *The encoding algorithm takes as input a public key* pk *a hash* $\mathbf{A}_f$*, and an input string* $\mathbf{x} \in \{0,1\}^m$*. The algorithm is divided into two subroutines.*

$\mathsf{PreEnc}(\mathsf{pk}, \mathbf{x})$**:** *The pre-encoding algorithm is independent of the hash: it takes as input a public key* pk*, an input* $\mathbf{x}$ *(and sometimes a vector* $\mathbf{r} \in \mathbb{Z}_q^{k-1}$*). It returns a pre-encoding of the input* E*, along with a private information* $\mathbf{s} \in \mathbb{Z}_q^k$*.*

$\mathsf{PostEnc}(\mathsf{pk}, \mathbf{A}_f, \mathbf{s})$**:** *The post-encoding algorithm does not depend on the input and returns a post-encoding defined as:*

$$c := \mathbf{s}^\intercal \cdot \mathbf{A}_f \cdot \mathbf{t} + e \in \mathbb{Z}_p$$

where $e \xleftarrow{\$} \chi(1^\lambda)$ *and* $\mathbf{t}$ *is sampled over* $\mathbb{Z}_2^{k \cdot \log q}$ *(not necessarily at random).*

The algorithm outputs an encoding $(E, c, \mathbf{t})$*.*

$\mathsf{Dec}(\mathsf{pk}, (E, c, \mathbf{t}), \psi)$**:** *The decoding algorithm takes as input a public key* pk*, an encoding* $(E, c, \mathbf{t})$ *and hasher's private information* ϕ*. The output is a bit*

$$\lceil \mathsf{Eval}(E, \psi) \cdot \mathbf{t} + c \rfloor_2$$

where Eval *is a polynomial-time deterministic algorithm that returns a vector in* $\mathbb{Z}_q^{k \cdot \log q}$*.*

We define a particular version of correctness that applies in most scheme as in Definition 23

Definition 24 (Special Correctness)**.** *Let* $\widetilde{B} := \widetilde{B}(\lambda)$ *be a function of the security parameter. An LFE scheme with pre-encoding satisfies* $\widetilde{B}(\lambda)$*-special correctness if, for every* $\lambda \in \mathbb{N}$*, function* $f \in \mathcal{F}_\lambda$ *and input* $\mathbf{x} \in \mathbb{Z}_2^m$*, we have*

$$\mathsf{Eval}(E, \psi) = \mathbf{s}^\intercal \cdot \mathbf{A}_f + \mathbf{r}^\intercal \cdot f(\mathbf{x}) \cdot \mathbf{G} + \widetilde{\mathbf{e}}^\intercal$$

where $\mathbf{G} = \mathbf{I}_k \otimes \mathbf{g}_q^\intercal$*,* $\widetilde{\mathbf{e}}$ *is such that* $\|\widetilde{\mathbf{e}}\|_\infty \leq \widetilde{B}(\lambda)$ *and the last entry of* $\mathbf{r}$ *is* -1*.*

We recall the notion of hasher privacy.

Definition 25 (Selective Hasher Privacy)**.** *Consider the following experiment* $\mathsf{HashExp}_{\mathcal{A}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$*:*

- *Activate the adversary* $(f_0, f_1, \mathsf{aux}) \xleftarrow{\$} \mathcal{A}_0(1^\lambda)$*.*
- *Sample a public key* $\mathsf{pk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$*.*
- *Sample a random bit* $b \xleftarrow{\$} \{0, 1\}$*.*
- *Compute* $(\mathbf{A}_f, \rho) \xleftarrow{\$} \mathsf{Hash}(\mathsf{pk}, f_b)$*.*
- *Compute* $b' \leftarrow \mathcal{A}_1(\mathsf{pk}, \mathbf{A}_f, \mathsf{aux})$*.*
- *Return* 1 *if and only if* $b = b'$*.*

We say that an LFE scheme with pre-encoding $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{PreEnc}, \mathsf{PostEnc}, \mathsf{Dec})$ *is hasher-private if for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left| \frac{1}{2} - \Pr\left[\mathsf{HashExp}_{\mathcal{A}}(1^\lambda) = 1\right] \right| \leq \mathsf{negl}(\lambda).$$

If the above property holds for every adversary (even computationally unbounded ones) we say that the scheme is statistically hasher private.

Finally we define a slightly weaker version of the standard encoder privacy, which however suffices for our purposes. Namely, we only require security against a distinguisher that sees the pre-encoding information (and we do not pose any requirement on the post-encoding).

Definition 26 (Selective Pre-Encoding Privacy)**.** *Consider the following experiment* $\mathsf{PreEncExp}_{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ *and a simulator* Sim*:*

- *Activate the adversary* $(\mathbf{x}, \mathsf{aux}) \xleftarrow{\$} \mathcal{A}_0(1^\lambda)$.
- *Sample a public key* $\mathsf{pk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$.
- *Sample a random bit* $b \xleftarrow{\$} \{0,1\}$.
- *If* $b = 0$*, compute* $(E_0, \phi_0) \xleftarrow{\$} \mathsf{Enc}(\mathsf{pk}, \mathbf{x})$*. Otherwise, compute* $E_1 \xleftarrow{\$} \mathsf{Sim}(1^\lambda, \mathsf{pk})$.
- *Compute* $b' \leftarrow \mathcal{A}_1(\mathsf{pk}, E_b, \mathsf{aux})$.
- *Return* 1 *if and only if* $b = b'$.

We say that an LFE scheme with pre-encoding $(\mathsf{Setup}, \mathsf{Hash}, \mathsf{PreEnc}, \mathsf{PostEnc}, \mathsf{Dec})$ *is pre-encoder private if for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left| \frac{1}{2} - \Pr\left[\mathsf{PreEncExp}_{\mathcal{A}}(1^\lambda) = 1\right] \right| \leq \mathsf{negl}(\lambda).$$

Most known LFE schemes satisfy (or can be adapted to satisfy) the above syntactical requirements. We summarize the state of the art in the following:

- Assuming the hardness of LWE, there exists an LFE scheme with pre-encoding for all bounded-depth circuits [21, Appendix E].
- Assuming the hardness of small-secret circular LWE, there exists an LFE scheme with pre-encoding for all (no bound on the depth) circuits [19].
- Assuming the hardness of Ring-LWE (small-secret circular Ring-LWE, resp.) there exists an LFE scheme with pre-encoding for all bounded-depth (unbounded-depth, resp.) RAM programs [16].[6]

For our purposes, the details of these constructions will be irrelevant, provided that they satisfy the above syntax. Henceforth, we just assume that such an LFE exists, with the understanding that the exact efficiency guarantees of our construction will depend on the building block used to instantiate it.

[6] Although the construction presented in [16] is not shown to satisfy function hiding, the generic compiler from [21] can be applied here to upgrade the scheme to satisfy this property. The compiler preserves the asymptotic efficiency of the construction, and we refer the reader to [21] for details.

5.3 Construction

In the following we define our construction for a reverse TDH for functions

$$f : \{0,1\}^m \to \{0,1\}^\ell.$$

We will use the following ingredients instantiating them over the ring $\mathbb{Z}_q$:

- The α-correct succinct MOLE protocol ($\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval}$), see Protocol 1.
- An LFE scheme with pre-encoding. ($\mathsf{Setup}, \mathsf{Hash}, \mathsf{PreEnc}, \mathsf{PostEnc}, \mathsf{Dec}$). Suppose that the scheme satisfies $\widetilde{B}$-special correctness.
- A pseudorandom generator $\mathsf{PRG} : \{0,1\}^\lambda \to \mathbb{Z}_q^\ell$ that can be evaluated uniformly (e.g., instantiated via a pseudorandom function).

For convenience, we assume that q is even. Moreover, we assume that $\alpha, \widetilde{B} \leq q \cdot \mathsf{negl}(\lambda)$. The protocol description is presented below.

Correctness. For correctness, observe that for all $i \in [\ell]$ we have that:

$$\begin{aligned}
&\underbrace{r_i + \mathsf{LFE.Eval}(E, \psi_i) \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - v_i}_{s_0} + \underbrace{-r_i - v_i'}_{s_1} \\
&= \mathsf{LFE.Eval}(E, \psi_i) \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - (v_i + v_i') \\
&= (\mathbf{s}^\intercal \cdot \mathbf{A}_{f_i} + \mathbf{r}^\intercal \cdot f_i(\mathbf{x}) \cdot \mathbf{G} + \widetilde{\mathbf{e}}^\intercal) \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - \mathbf{s}^\intercal \cdot \mathbf{A}_{f_i} \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - e_i \\
&= -q/2 \cdot f_i(\mathbf{x}) \cdot \mathbf{r}^\intercal \cdot \mathbf{u} + \widetilde{\mathbf{e}}^\intercal \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - e_i \\
&= q/2 \cdot f_i(\mathbf{x}) + \hat{e}_i
\end{aligned}$$

where $\hat{e}_i := \widetilde{\mathbf{e}}^\intercal \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - e_i$ and the second equality follows from the α-correctness of the MOLE and the special correctness of the LFE. Notice that $|\widetilde{\mathbf{e}}^\intercal \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - e_i| \leq \alpha(\lambda) + \widetilde{B}(\lambda)$. Thus we have that s_0 and s_1 is a noisy secret sharing of $f_i(\mathbf{x})$. Furthermore, it holds that

$$f_i(\mathbf{x}) = \lceil s_0 + s_1 \rfloor_2 = \lceil s_0 \rfloor_2 \oplus \lceil s_1 \rfloor_2$$

except if $s_0 \in [q/4 - |\hat{e}_i|, q/4 + |\hat{e}_i|] \cup [3q/4 - |\hat{e}_i|, 3q/4 + |\hat{e}_i|]$. Note that the size of the interval is at most $4 \cdot |\hat{e}_i|$, which is a negligible fraction of q. Thus, by the pseudorandomness of PRG, this event happens with negligible probability (independently of the inputs and the public parameters), concluding our proof of correctness.

Security. We prove security in the following.

Theorem 27. *Suppose that* MOLE *is encoder private. If* LFE *is pre-encoding secure, Construction 6 is input private. Finally, if* LFE *is (statistically) function private, the reverse tradpoor hashing scheme is (statistically) hasher private.*

Construction 6: Reverse Trapdoor Hash

$\mathsf{Setup}(1^\lambda)$**:** Sample keys

$$\mathsf{pk} \xleftarrow{\$} \mathsf{LFE.Setup}(1^\lambda), \qquad \mathsf{mpk} \xleftarrow{\$} \mathsf{MOLE.Setup}(1^\lambda).$$

Then, output $\mathsf{hk} := (\mathsf{pk}, \mathsf{mpk})$.

$\mathsf{Gen}(\mathsf{hk}, \mathbf{x})$**:** Compute

$$(E, \mathbf{s}) \xleftarrow{\$} \mathsf{LFE.PreEnc}(\mathsf{pk}, \mathbf{x}), \qquad (E', \phi) \xleftarrow{\$} \mathsf{MOLE.Enc}(\mathsf{mpk}, \mathbf{s})$$

Then, sample $\mathsf{seed} \xleftarrow{\$} \{0,1\}^\lambda$ and output $\mathsf{ek} := (E, E', \mathsf{seed})$ and $\mathsf{td} := (\phi, \mathsf{seed})$.

$\mathsf{Hash}(\mathsf{hk}, f)$**:** Let $\mathbf{u}$ be the last element of the standard basis over $\mathbb{Z}_q^k$. Let $(f_0, \ldots, f_{\ell-1})$ be the functions that compute the i-th bit of the output of f. For every $i \in [\ell]$ compute

$$(\mathbf{A}_{f_i}, \psi_i) \xleftarrow{\$} \mathsf{LFE.Hash}(\mathsf{pk}, f_i).$$

Finally, compute

$$\mathbf{A} \leftarrow \left(\mathbf{A}_{f_0} \cdot \mathbf{G}^{-1}\left(-\frac{q}{2} \cdot \mathbf{u}\right) \Big\| \ldots \Big\| \mathbf{A}_{f_{\ell-1}} \cdot \mathbf{G}^{-1}\left(-\frac{q}{2} \cdot \mathbf{u}\right)\right)$$

$$(d, \psi) \xleftarrow{\$} \mathsf{MOLE.Hash}(\mathsf{mpk}, \mathbf{A}^\intercal)$$

and output d and $\rho := (\psi, \psi_0, \ldots, \psi_{\ell-1})$.

$\mathsf{Enc}(\mathsf{hk}, \mathsf{ek}, f, \rho)$**:** Let $\mathbf{u}$ be the last element of the standard basis over $\mathbb{Z}_q^k$. Compute

$$\mathbf{v} \leftarrow \mathsf{MOLE.HashEval}(\mathsf{mpk}, E', \psi).$$

Parse $\mathbf{v}$ as the vertical concatenation of $(v_0, \ldots, v_{\ell-1})$ and let $(r_0, \ldots, r_{\ell-1}) \leftarrow \mathsf{PRG}(\mathsf{seed})$. For every $i \in [\ell]$, compute

$$e_i \leftarrow \left\lceil r_i + \mathsf{LFE.Eval}(E, \psi_i) \cdot \mathbf{G}^{-1}(-q/2 \cdot \mathbf{u}) - v_i \right\rfloor_2$$

Finally, output $\mathbf{e} := \begin{pmatrix} e_0 \\ \vdots \\ e_{\ell-1} \end{pmatrix}$.

$\mathsf{Dec}(\mathsf{hk}, d, \mathsf{td})$**:** Compute

$$\mathbf{w} \leftarrow \mathsf{MOLE.EncEval}(\mathsf{mpk}, d, \phi).$$

Parse $\mathbf{w}$ as the vertical concatenation of $(w_0, \ldots, w_{\ell-1})$ and derive $(r_0, \ldots, r_{\ell-1}) \leftarrow \mathsf{PRG}(\mathsf{seed})$. Then, for all $i \in [\ell]$, compute

$$e'_i \leftarrow \lceil -r_i - w_i \rfloor_2$$

Finally, output $\mathbf{e}' := \begin{pmatrix} e'_0 \\ \vdots \\ e'_{\ell-1} \end{pmatrix}$.

Proof. It is easy to see that the construction is (statistically) function private if LFE is (statistically) function private.

As for input privacy, we proceed by a series of indistinguishable hybrids.

- Hybrid $\mathcal{H}_0$: This hybrid corresponds to the original game: We provide the adversary $\mathcal{A}_1$ with an encoding key (E, E', seed) generated using $\mathsf{Gen}(\mathsf{hk}, \mathbf{x})$.
- Hybrid $\mathcal{H}_1$: In this hybrid, we generate E' using $\mathsf{MOLE.Sim}(1^\lambda, \mathsf{mpk})$. The rest remains as in the previous hybrid.

  Hybrid $\mathcal{H}_0$ and Hybrid $\mathcal{H}_1$ are computationally indistinguishable thanks to the encoder privacy of the MOLE.
- Hybrid $\mathcal{H}_2$: In this hybrid, we generate E using $\mathsf{LFE.Sim}(1^\lambda, \mathsf{pk})$. The rest remains as in the previous hybrid.

  Hybrid $\mathcal{H}_1$ and $\mathcal{H}_2$ are indistinguishable under the input privacy of LFE.

Notice that in Hybrid $\mathcal{H}_2$, the triple (E, E', seed) provided to the adversary contains no information about $\mathbf{x}$. From this, we can easily derive a simulator. This ends the proof. □

Parameters. Construction 6 inherits its efficiency from two ingredients: The pre-encoding part of the underlying LFE scheme and the succinct MOLE used to compress the collection of post-encoding matrices. Let $s_{\mathsf{pre}}(\lambda, m, \ell)$ denote the size of the pre-encoding in the underlying LFE instantiation and let $s_{\mathsf{hash}}(\lambda, \ell)$ denote the size of the corresponding reverse-TDH digest produced after MOLE compression. Then Construction 6 has digest size $s_{\mathsf{hash}}(\lambda, \ell)$, encoding-key size

$$s_{\mathsf{pre}}(\lambda, m, \ell) + \mathsf{poly}(\lambda, \log \ell, \log q, \log k),$$

and online encoding size exactly ℓ. In particular, any dependence on the circuit depth is inherited from the underlying LFE parameters, including the induced dependence of k and q on that depth.

Theorem 28 (Reverse Trapdoor Hashing)**.** *Assume that MOLE is encoder private and that LFE is pre-encoding secure and function private. Then Construction 6 is an input-private and function-private reverse TDH for functions $f : \{0,1\}^m \to \{0,1\}^\ell$ with digest size $s_{\mathsf{pre}}(\lambda, m, \ell)$, encoding-key size $s_{\mathsf{pre}}(\lambda, m, \ell) + \mathsf{poly}(\lambda, \log \ell, \log q, \log k)$, and online encoding size ℓ.*

5.4 Applications: Laconic OT and Succinct HSS

We briefly record two direct consequences of the reverse trapdoor hash of Construction 6: A public-key two-party homomorphic secret-sharing scheme and a batched laconic oblivious transfer protocol.

Theorem 29 (Succinct HSS from Reverse TDH)**.** *Assume that the hypotheses of Theorem 28 hold. Then there exists a public-key two-party homomorphic secret-sharing scheme for all functions $f : \{0,1\}^m \times \{0,1\}^n \to \{0,1\}^\ell$ with communication complexity $m \cdot \mathsf{poly}(\lambda, \log n)$.*

Proof Sketch. Fix x and let $f_x(\cdot) := f(x, \cdot)$. Alice runs the reverse-TDH hashing algorithm on f_x, obtaining a digest d and local randomness ρ. She sends only the digest d to Bob and keeps ρ as her local share material. Bob runs the reverse-TDH key-generation algorithm on his input y, obtaining an encoding key ek and trapdoor td. He sends ek to Alice and keeps td as his local share material. Alice computes

$$z_A := \mathsf{Enc}(\mathsf{hk}, \mathsf{ek}, f_x, \rho),$$

while Bob computes

$$z_B := \mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d).$$

By correctness of the reverse TDH we have $z_A \oplus z_B = f_x(y) = f(x, y)$. Thus (z_A, z_B) is a valid additive secret-sharing of the output. The communication is exactly the reverse-TDH digest for the function side plus the reverse-TDH encoding key for the input side. Since the digest is succinct in the description of the hashed function, this yields communication polylogarithmic in $|x|$ and linear in the short side $|y|$. □

The following theorem uses Construction 6 instantiated with LFE for RAM programs, whose security follows from the hardness of Ring-LWE.

Theorem 30 (Laconic OT from Reverse TDH)**.** *Assume that the hypotheses of Theorem 28 hold and that pseudorandom functions exist. Then there exists a batched laconic OT protocol with download rate* $1/2$*.*

Proof Sketch. Let $D = \{D_i\}_{i\in[N]} \in \{0,1\}^N$ be the receiver's choice bits, and let the sender hold pairs of messages

$$\{(m_{i,0}, m_{i,1})\}_{i\in I}$$

for a batch of queried indices $I \subseteq [N]$. Define the RAM program f_D which has the database D hardwired and, on input a PRF key k, returns for every queried pair (i, b):

$$f_D(k; i, b) = \begin{cases} 0 & \text{if } b = D_i, \\ \mathsf{PRF}(k, i) & \text{if } b \neq D_i. \end{cases}$$

The receiver hashes f_D using the reverse TDH, obtaining digest d and local randomness ρ, and sends the digest to the sender. The sender samples a fresh PRF key k, runs the reverse-TDH generation algorithm to obtain an encoding key ek for k, and computes, for each requested (i, b),

$$c_{i,b} := \mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d)_{i,b} \oplus m_{i,b},$$

where td is the sender's trapdoor output by key generation. The sender transmits ek together with all ciphertext bits $\{c_{i,b}\}_{i\in I, b\in\{0,1\}}$. For the selected branch $b = D_i$, correctness of reverse TDH gives

$$\mathsf{Enc}(\mathsf{hk}, \mathsf{ek}, f_D, \rho)_{i,b} = \mathsf{Dec}(\mathsf{hk}, \mathsf{td}, d)_{i,b},$$

and since in that case $f_D(k; i, b) = 0$, the receiver can recover m_{i,D_i}. For the unselected branch $b \neq D_i$, the value revealed by f_D is $\mathsf{PRF}(k, i)$, so the corresponding pad is pseudorandom and computationally hides $m_{i,1-D_i}$. The sender sends one mask per two transferred messages, so the asymptotic transfer rate for a growing $|I|$ is $1/2$. □

6 Rate-1 Adaptive LFE for all Bounded-Depth Functions

6.1 Input-Succinct Reverse Trapdoor Hashing for RMS

As a stepping stone, we present a variant of a trapdoor hashing (TDH) scheme [17] with reversed syntax, achieving almost optimal encoding key size but supporting a restricted family of functions.

Let $m := m(\lambda)$, $\ell := \ell(\lambda)$, $p := p(\lambda)$, $d := d(\lambda)$, $T := T(\lambda)$, $k := k(\lambda)$ and $t := t(\lambda)$ be positive integers. Construction 7 allows the evaluation of functions that map any pair $(\mathbf{x}, \mathbf{a})$, where $\mathbf{x} \in \{0,1\}^m$ and $\mathbf{a} \in \mathbb{Z}_p^{t \cdot k}$, into $f(\mathbf{x}) \otimes \mathbf{a}$ where $f : \{0,1\}^m \to \{0,1\}^\ell$ is described by a depth-d, T-bounded RMS program.

Our construction only achieves *adaptive* input privacy, but only a weak notion of correctness: Although the encoding key leaks no information about $\mathbf{x}$ and $\mathbf{a}$, in order to successfully run the encoding procedure, we are required to know $\mathbf{x}$ (but not $\mathbf{a}$). Although these properties may seem artificial, they will later be useful in combination with techniques of [12], to build adaptive LFE with rate-1 encodings (Section 6.2).

Concerning efficiency, the digest size in our construction is logarithmic in m, t and linear in ℓ, d. Moreover, the size of the encoding key is $t \cdot \mathsf{poly}(\lambda, \log m, \log \ell, d)$. Notice that we can achieve logarithmic dependency in m only because the encoding procedure needs $\mathbf{x}$ in order to run successfully.

The Construction. We now invite the reader to take a look at Construction 7. The scheme relies on the compressed, adaptive lattice encodings of Section 4. We instantiate them over $\mathbb{Z}_q$, where $q = \Delta \cdot p$ where $\Delta := \Delta(\lambda)$ is a positive integer. We use $(\mathsf{Setup}, \mathsf{Compress}, \mathsf{Expand})$ to denote the algorithms of Construction 5 and we use $n := n(\lambda)$ to denote the digest size of the α-correct OTE with which it is instantiated. Let $\mathbf{P}$ be the matrix used for the hasher evaluation in the OTE protocol. We also rely on an $\hat{\alpha}$-correct succinct NI-MOLE protocol $\mathsf{MOLE} = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Enc}, \mathsf{HashEval}, \mathsf{EncEval})$ for matrices of size $\mathbb{Z}_q^{(t \cdot \ell \cdot k \cdot \log q) \times k}$.

Adaptive Correctness. In order for the construction to be fully correct, we need to choose a sufficiently large modulus q. Specifically, it must hold that

$$\log q \cdot p \cdot \frac{n \cdot \|\mathbf{P}\|_\infty \cdot T \cdot B \cdot (k \cdot \log q)^{d+2} + \alpha \cdot T \cdot (k \cdot \log q)^d + B + \hat{\alpha}}{q} = 2^{-\omega(\log \lambda)}.$$

Construction 7: Input-Succinct Reverse TDH for RMS

$\mathsf{Setup}(1^\lambda)$**:** Run the setup for the MOLE and the compressed lattice encodings

$$\mathsf{ck} \xleftarrow{\$} \mathsf{Setup}(1^\lambda), \qquad \mathsf{mpk} \xleftarrow{\$} \mathsf{MOLE.Setup}(1^\lambda).$$

Then, sample $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{k \times (n \cdot k \cdot \log q)}$, $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_q^{k \times (t \cdot k \cdot \log q)}$ and return $\mathsf{hk} := (\mathsf{ck}, \mathsf{mpk}, \mathbf{A}, \mathbf{B})$.

$\mathsf{Hash}(\mathsf{hk}, f)$**:** Compute:

$$\mathbf{A}_f \leftarrow \mathsf{EvalRMSK}(\mathbf{A}\mathbf{P}^\intercal, f), \qquad \mathbf{F} \leftarrow -\mathbf{A}_f \cdot (\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B})).$$

Then output $(\mathbf{d}, \psi) \xleftarrow{\$} \mathsf{MOLE.Hash}(\mathsf{mpk}, \mathbf{F}^\intercal)$.

$\mathsf{Gen}\left(\mathsf{hk}, \mathbf{x}, \mathbf{a}; \{\mathbf{r}_i\}_{i \in [d]}\right)$**:** Let $\hat{\mathbf{x}}$ be the vertical concatenation of $\mathbf{x}$ and 1. For every $i \in [d+1]$, sample $\mathbf{s}_i \xleftarrow{\$} \mathbb{Z}_q^k$ and set:

$$(\mathbf{h}_i, E_i) \xleftarrow{\$} \mathsf{Compress}(\mathsf{ck}, \mathbf{A}, \hat{\mathbf{x}}, \mathbf{s}_i, \mathbf{s}_{i+1}, \mathbf{r}_i).$$

Next, sample $\mathbf{e} \xleftarrow{\$} \chi(\lambda)$ and set

$$\mathbf{b}^\intercal := \mathbf{s}_d^\intercal \mathbf{B} + \mathbf{a}^\intercal \cdot (\mathbf{I}_t \otimes \mathbf{G}) + \mathbf{e}^\intercal,$$

where $\mathbf{G} = \mathbf{I}_k \otimes \mathbf{g}_q^\intercal$.

Proceed by generating $(C, \phi) \xleftarrow{\$} \mathsf{MOLE.Enc}(\mathsf{mpk}, \mathbf{s}_0)$ and sampling $\mathsf{seed} \xleftarrow{\$} \{0,1\}^\lambda$.

Output $\mathsf{ek} := (C, \mathbf{b}, \{\mathbf{h}_i, E_i\}_{i \in [d]}, \mathsf{seed})$ and $\mathsf{td} := (\phi, \mathsf{seed})$

$\mathsf{Enc}(\mathsf{hk}, \mathsf{ek}, \psi, \mathbf{x}, f)$**:** Let $\hat{\mathbf{x}}$ be the vertical concatenation of $\mathbf{x}$ and 1. For every $i \in [d]$, compute

$$\mathbf{c}_i^\intercal \leftarrow \mathsf{Expand}(\mathsf{ck}, \mathbf{A}, \mathbf{h}_i, E_i, \hat{\mathbf{x}})$$

and set

$$\mathbf{c}^\intercal \leftarrow \mathsf{EvalRMSC}(\mathbf{A}\mathbf{P}^\intercal, f, \mathbf{x}, \{\mathbf{c}_i\}_{i \in [d]}).$$

Next, derive

$$\mathbf{z} \leftarrow -(\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \mathbf{c} + f(\mathbf{x}) \otimes \mathbf{b}.$$

Finally, compute

$$\mathbf{v} \leftarrow \mathsf{MOLE.HashEval}(\mathsf{mpk}, C, \psi), \qquad \tilde{\mathbf{u}} \leftarrow (\mathbf{I}_{t \cdot \ell \cdot k} \otimes \mathbf{G}^{-1}(\Delta))^\intercal (\mathbf{z} - \mathbf{v}),$$
$$\tilde{\mathbf{y}} \leftarrow \lceil \tilde{\mathbf{u}} + \mathsf{PRG}(\mathsf{seed}) \rfloor_p .$$

Output $\tilde{\mathbf{y}}$.

$\mathsf{Dec}(\mathsf{hk}, \mathbf{d}, \mathsf{td})$**:** Compute

$$\mathbf{w} \leftarrow \mathsf{MOLE.EncEval}(\mathsf{mpk}, \mathbf{d}, \phi), \qquad \mathbf{u} \leftarrow (\mathbf{I}_{t \cdot \ell \cdot k} \otimes \mathbf{G}^{-1}(\Delta))^\intercal \mathbf{w},$$
$$\mathbf{y} \leftarrow \lceil -\mathbf{u} - \mathsf{PRG}(\mathsf{seed}) \rfloor_p .$$

Then, output $\mathbf{y}$.

Notice that in the OTE of Section 3.3, it holds that $\|\mathbf{P}\|_\infty = n^{2r}$ where $r = O(\log m)$.[7] We begin by observing that, by the correctness of Construction 5, for every $i \in [d]$:

$$\mathbf{c}_i^\intercal = \mathbf{s}_i^\intercal \mathbf{A}\mathbf{P}^\intercal + \mathbf{s}_{i+1}^\intercal(\hat{\mathbf{x}}^\intercal \otimes \mathbf{G}) + \mathbf{e}_i^\intercal$$

where $\|\mathbf{e}_i\|_\infty \leq k \cdot n \cdot \|\mathbf{P}\|_\infty \cdot \log q \cdot B(\lambda) + \alpha$. Therefore, by Lemma 17, we have that:

$$\begin{aligned}\mathbf{c}^\intercal &= \mathsf{EvalRMSC}(\mathbf{A}\mathbf{P}^\intercal, f, \mathbf{x}, \{\mathbf{c}_i\}_{i\in[d]}) \\ &= \mathbf{s}_0^\intercal \mathbf{A}_f + \mathbf{s}_d^\intercal(f(\mathbf{x})^\intercal \otimes \mathbf{G}) + \widetilde{\mathbf{e}}^\intercal.\end{aligned}$$

where $\|\widetilde{\mathbf{e}}\|_\infty \leq n \cdot \|\mathbf{P}\|_\infty \cdot B \cdot T \cdot (k \cdot \log q)^{d+1} + \alpha \cdot T \cdot (k \cdot \log q)^d$. We obtain that:

$$\begin{aligned}\mathbf{z} &= -(\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \mathbf{c} + f(\mathbf{x}) \otimes \mathbf{b} \\ &= -(\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \big(\mathbf{A}_f^\intercal \mathbf{s}_0 + (f(\mathbf{x}) \otimes \mathbf{G}^\intercal)\mathbf{s}_d + \widetilde{\mathbf{e}}\big) + f(\mathbf{x}) \otimes \mathbf{b} \\ &= \mathbf{F}^\intercal \mathbf{s}_0 - (\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal (f(\mathbf{x}) \otimes \mathbf{G}^\intercal)\mathbf{s}_d - (\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \widetilde{\mathbf{e}} + f(\mathbf{x}) \otimes \mathbf{b} \\ &= \mathbf{F}^\intercal \mathbf{s}_0 - f(\mathbf{x}) \otimes (\mathbf{B}^\intercal \mathbf{s}_d) - (\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \widetilde{\mathbf{e}} + f(\mathbf{x}) \otimes \big(\mathbf{B}^\intercal \mathbf{s}_d + (\mathbf{I}_t \otimes \mathbf{G})^\intercal \mathbf{a} + \mathbf{e}\big) \\ &= \mathbf{F}^\intercal \mathbf{s}_0 + f(\mathbf{x}) \otimes \big((\mathbf{I}_t \otimes \mathbf{G})^\intercal \mathbf{a}\big) - (\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \widetilde{\mathbf{e}} + f(\mathbf{x}) \otimes \mathbf{e}.\end{aligned}$$

Finally, by the $\hat{\alpha}$-correctness of the MOLE, we have

$$\mathbf{v} + \mathbf{w} = \mathbf{F}^\intercal \mathbf{s}_0 + \widehat{\mathbf{e}}$$

for some error vector $\widehat{\mathbf{e}}$ such that $\|\widehat{\mathbf{e}}\|_\infty \leq \hat{\alpha}$. Therefore

$$\begin{aligned}\mathbf{z} - \mathbf{v} - \mathbf{w} &= \mathbf{F}^\intercal \mathbf{s}_0 + f(\mathbf{x}) \otimes \big((\mathbf{I}_t \otimes \mathbf{G})^\intercal \mathbf{a}\big) - (\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \widetilde{\mathbf{e}} + f(\mathbf{x}) \otimes \mathbf{e} - \mathbf{F}^\intercal \mathbf{s}_0 - \widehat{\mathbf{e}} \\ &= f(\mathbf{x}) \otimes \big((\mathbf{I}_t \otimes \mathbf{G})^\intercal \mathbf{a}\big) - (\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \widetilde{\mathbf{e}} + f(\mathbf{x}) \otimes \mathbf{e} - \widehat{\mathbf{e}}.\end{aligned}$$

Define

$$\boldsymbol{\varepsilon} := -(\mathbf{I}_\ell \otimes \mathbf{G}^{-1}(\mathbf{B}))^\intercal \widetilde{\mathbf{e}} + f(\mathbf{x}) \otimes \mathbf{e} - \widehat{\mathbf{e}}.$$

It holds that:

$$\|\boldsymbol{\varepsilon}\|_\infty \leq n \cdot \|\mathbf{P}\|_\infty \cdot B \cdot T \cdot (k \cdot \log q)^{d+2} + \alpha \cdot T \cdot (k \cdot \log q)^d + B + \hat{\alpha}.$$

We observe that:

$$(\mathbf{I}_{t\cdot\ell\cdot k} \otimes \mathbf{G}^{-1}(\Delta))^\intercal (\mathbf{z} - \mathbf{v} - \mathbf{w}) = \Delta \cdot (f(\mathbf{x}) \otimes \mathbf{a}) + (\mathbf{I}_{t\cdot\ell\cdot k} \otimes \mathbf{G}^{-1}(\Delta))^\intercal \boldsymbol{\varepsilon}.$$

Notice also that

$$\big\|(\mathbf{I}_{t\cdot\ell\cdot k} \otimes \mathbf{G}^{-1}(\Delta))^\intercal \boldsymbol{\varepsilon}\big\|_\infty \leq \log q \cdot \|\boldsymbol{\varepsilon}\|_\infty.$$

Finally, we observe that $\tilde{\mathbf{y}} + \mathbf{y} \neq f(\mathbf{x}) \otimes \mathbf{a} \pmod p$ only if one of the entries of $\mathbf{u} - \mathsf{PRG}(\mathsf{seed})$ is less than $\log q \cdot \|\boldsymbol{\varepsilon}\|_\infty$ away from an odd multiple of $\Delta/2$. Since seed is independent of $\mathbf{u}$ and by the security of the PRG, the probability of this event is at most $p \cdot \log q \cdot \|\boldsymbol{\varepsilon}\|_\infty / q + \mathsf{negl}(\lambda)$ (this holds irrespectively of the inputs chosen by the adversary). By hypothesis, this quantity is negligible.

[7] By a careful choice of parameters, r can also be made a constant: E.g., by setting $t = \lambda$ in Construction 2.

Adaptive Privacy. We now show that our construction satisfies adaptive privacy: under the LWE assumption with superpolynomial modulus-noise ratio, we can simulate $(C, \mathbf{b}, \{\mathbf{h}_i, E_i\}_{i\in[d]}, \mathsf{seed})$ without knowing any information about $\mathbf{x}$ and $\mathbf{a}$ even if these are chosen by the adversary after seeing the output of the setup.

Theorem 31 (Adaptive Privacy)**.** *Consider the following experiment* $\mathsf{AdaptivePrivacy}_{\mathcal{A},\mathsf{Sim}}(1^\lambda)$ *parametrized by an adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ *and a simulator* Sim*:*

- *Sample* $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda)$.
- *Activate the adversary* $(\mathbf{x}, \mathbf{a}, \mathsf{aux}) \leftarrow \mathcal{A}_0(1^\lambda, \mathsf{hk})$.
- *Sample* $b \xleftarrow{\$} \{0,1\}$*. If* $b = 0$*, sample* $\boldsymbol{r}_i \xleftarrow{\$} \mathbb{Z}_q^k$ *for every* $i \in [d]$*. Then, compute*

$$(\mathsf{ek}_0, \mathsf{td}_0) \xleftarrow{\$} \mathsf{Gen}(\mathsf{hk}, \mathbf{x}, \mathbf{a}; \{\mathbf{r}_i\}_{i\in[d]})$$

  If $b = 1$ *compute:*

$$\mathsf{ek}_1 \xleftarrow{\$} \mathsf{Sim}(1^\lambda, \mathsf{hk}).$$

- *Obtain* $b' \leftarrow \mathcal{A}_1(\mathsf{ek}_b, \mathsf{aux})$ *and return* 1 *if and only if* $b = b'$*.*

Then assuming the hardness of LWE with superpolynomial modulus-noise ratio and that MOLE *and* OTE *are encoder private, there exists a PPT simulator* Sim*, such that for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that, for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\left|\frac{1}{2} - \Pr\left[\mathsf{AdaptivePrivacy}_{\mathcal{A},\mathsf{Sim}}(1^\lambda) = 1\right]\right| \leq \mathsf{negl}(\lambda)$$

where the probability is taken over the random coins of the experiment.

Proof. We prove the theorem through a series of indistinguishable hybrids.

- Hybrid $\mathcal{H}_0$: This is the original experiment: We provide the adversary $\mathcal{A}_1$ with $(C, \mathbf{b}, \{\mathbf{h}_i, E_i\}_{i\in[d]}, \mathsf{seed})$ where $((C, \mathbf{b}, \{\mathbf{h}_i, E_i\}_{i\in[d]}, \mathsf{seed}), \mathsf{td}) \xleftarrow{\$} \mathsf{Gen}(\mathsf{hk}, \mathbf{x}, \mathbf{a}; \{\mathbf{r}_i\}_{i\in[d]})$.
- Hybrid $\mathcal{H}_1$: In this hybrid, we generate C using $\mathsf{MOLE.Sim}(1^\lambda, \mathsf{mpk})$. The rest remains as in the previous hybrid.

  Indistinguishability from Hybrid $\mathcal{H}_0$ follows from the encoder privacy of MOLE.
- Hybrid $\mathcal{H}_2^i$: In this hybrid, for every $j < i$, we generate $(\mathbf{h}_j, E_j) \xleftarrow{\$} \mathsf{CSim}(1^\lambda, \mathsf{ck}, \mathbf{A})$, where CSim is the simulator of Theorem 18. The rest remains as in the previous hybrid.

  We observe that Hybrid $\mathcal{H}_1$ is identical to Hybrid $\mathcal{H}_2^0$. Moreover, for every $i \in [d]$, Hybrid $\mathcal{H}_2^i$ is computationally indistinguishable from Hybrid $\mathcal{H}_2^{i+1}$ due to Theorem 18 (notice that in Hybrid $\mathcal{H}_2^i$, the pair $(\mathbf{h}_{i-1}, E_{i-1})$ is independent of $\mathbf{s}_i$).

- Hybrid $\mathcal{H}_3$: In this hybrid, we sample $\mathbf{b} \xleftarrow{\$} \mathbb{Z}_q^{t \cdot k \cdot \log q}$. The rest remains as in the previous hybrid.

  Hybrid $\mathcal{H}_2^d$ is computationally indistinguishable from $\mathcal{H}_3$ under LWE. Notice that in Hybrid $\mathcal{H}_2^d$, the terms $C, \{\mathbf{h}_i, E_i\}_{i \in [d]}$ no longer contain information about $\mathbf{s}_d$. This allows us to reduce indistinguishability to LWE. Indeed, all information about $(\mathbf{I}_t \otimes \mathbf{G})^\intercal \mathbf{a}$ in $\mathbf{b}$ is masked by

  $$\mathbf{B}^\intercal \mathbf{s}_d + \mathbf{e}.$$

  This can be viewed as an LWE sample with respect to the matrix $\mathbf{B}^\intercal$ and secret $\mathbf{s}_d$.

Notice that in Hybrid $\mathcal{H}_3$ all material provided to $\mathcal{A}_1$ is independent of $\mathbf{x}$ and $\mathbf{a}$. □

6.2 Rate-1 Adaptive LFE for all Bounded-Depth Functions

We now present our adaptive LFE scheme for bounded-depth functions. We described it in Construction 8. Let $\ell := \ell(\lambda)$, $m := m(\lambda)$, and $D := D(\lambda)$ be positive integers. The construction allows the evaluation of any function $f : \{0,1\}^m \to \{0,1\}^\ell$ described by a polynomial-sized circuit of depth at most $D(\lambda)$ and width bounded by $W(\lambda) = t^r \cdot n$, where t is the parameter in the fully succinct OTE. The digest size is $\mathsf{poly}(\lambda, \log m, \log \ell, \log D)$, whereas the size of the encodings is $m + \ell + D \cdot \mathsf{poly}(\lambda, \log m, \log \ell, \log D)$.

The construction builds upon the reverse trapdoor hashing scheme $\mathsf{RTDH} := (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ of Construction 7. Notice that we can interpret the outputs of Enc and Dec as a secret-sharing over $\mathbb{Z}_p$ where $p = m^r \cdot 2^{\omega(\log \lambda)}$, setting the modulus appropriately. Moreover, we assume that RTDH is instantiated using the OTE protocols in Construction 2 and 1. The basic idea is the following: we pick a constant $d = O(1)$ and we rewrite f as the composition of $L = D/\log d$ functions of depth at most $\log d$. Each of these can be regarded as a depth-d RMS program, which can therefore be evaluated using RTDH.

Adaptive Correctness. Towards proving correctness, we prove the following lemma, which states that the term $\mathbf{h}_{i,j}$ computed by the server during the decoding procedure is an adaptive lattice encoding of $\mathsf{OTE.Hash}(\mathsf{tpk}, \mathbf{x}_i)$. In other words, ek_i is an encoding key for $\mathbf{x}_i = (f'_{i-1} \circ \cdots \circ f'_0)(\mathsf{Bits}(\mathsf{ct}), \mathbf{z})$.

Lemma 32. *For every $i \in [L]$ and $j \in [d]$, we have that*

$$\mathbf{h}_{i,j} = \mathbf{A}^\intercal \mathbf{s}_{i,j} + (\mathbf{d}'_i \otimes \mathbf{G}^\intercal)\mathbf{r}_{i,j} + \mathbf{e}_i.$$

where $(\mathbf{d}'_i, \psi'_i) \leftarrow \mathsf{OTE.Hash}(\mathsf{tpk}, \mathbf{x}_i)$, $\mathbf{s}_{i,j}$ is the encryption key used in the encoding $\mathbf{h}'_{i,j}$ ($\mathbf{h}_{0,j}$, if $i = 0$) and $\|\mathbf{e}_i\|_\infty \leq B(\lambda)$. This holds with overwhelming probability, irrespectively of the inputs chosen by the adversary.

Construction 8: Rate-1 Adaptive LFE for Bounded-Depth Circuits (to be continued)

$\mathsf{Setup}(1^\lambda)$**:** Output $\mathsf{pk} := \mathsf{hk} \xleftarrow{\$} \mathsf{RTDH.Setup}(1^\lambda)$

$\mathsf{Hash}(\mathsf{pk}, f)$**:** For any vector $\mathbf{z}$, let $f_{\mathbf{z}}$ be the function that maps a key K to $f(\mathbf{z} \oplus \mathsf{PRG}(K))$. Let f' be the function that maps a pair $(\mathsf{ct}, \mathbf{z})$ to $\mathsf{GSW.Eval}(\mathsf{ct}, f_{\mathbf{z}})$ where the evaluation is performed as in [11]. Rewrite f' as

$$f' = f'_{L-1} \circ f'_{L-2} \circ \cdots \circ f'_0$$

where each f'_i is a boolean function described by a T-bounded depth-d RMS program, all with the same input and output size $W = t^r \cdot n$. For every $i \in [L-1]$, let $\hat{f}_i$ be the function that maps $\mathbf{x}$ to $\mathsf{OTE.Hash}(\mathsf{tpk}, f'_i(\mathbf{x}))$ and set $\hat{f}_{L-1} \leftarrow f'_{L-1}$, where tpk denotes the OTE public key that is used in Construction 5. Notice that $\mathsf{OTE.Hash}$ is linear so its depth is 0.[8] Therefore, $\hat{f}_i$ is described by a $\max(T, m^r)$-bounded depth-d RMS program.[9] Let $\hat{f}$ the function that maps any vector $\mathbf{x}$ to $(\hat{f}_0(\mathbf{x}), \ldots, \hat{f}_{L-1}(\mathbf{x}))$. Output $(\mathbf{d}, \psi) \leftarrow \mathsf{RTDH.Hash}(\mathsf{hk}, \hat{f})$.

$\mathsf{Enc}(\mathsf{pk}, \mathbf{d}, \mathbf{x})$**:** Sample $\mathbf{M} \xleftarrow{\$} \mathbb{Z}_q^{(k-1)\times(k\cdot\log q+\lambda)}$, $\mathbf{r} \xleftarrow{\$} \mathbb{Z}_q^{k-1}$ and $\mathbf{e} \xleftarrow{\$} \chi(\lambda)$ and generate a GSW key

$$\mathsf{sk}_{\mathsf{GSW}} := \begin{pmatrix} \mathbf{r} \\ -1 \end{pmatrix}, \qquad \mathsf{pk}_{\mathsf{GSW}} := \begin{pmatrix} \mathbf{M} \\ (\mathbf{M}^\intercal \mathbf{r} + \mathbf{e})^\intercal \end{pmatrix}.$$

Sample $K \xleftarrow{\$} \{0,1\}^\lambda$ and encrypt the input

$$\mathbf{z} \leftarrow \mathbf{x} \oplus \mathsf{PRG}(K), \qquad \mathsf{ct} \xleftarrow{\$} \mathsf{GSW.Enc}(\mathsf{pk}_{\mathsf{GSW}}, K).$$

For every $i \in [L], j \in [d]$ sample randomness $\mathbf{r}_{i,j} \xleftarrow{\$} \mathbb{Z}_q^k$ and define:

$$\mathbf{a}_i := \begin{pmatrix} \mathbf{G}^{-1}(\mathbf{r}_{i,0}) \\ \vdots \\ \mathbf{G}^{-1}(\mathbf{r}_{i,d-1}) \end{pmatrix}, \qquad \mathbf{a}_L := \mathbf{G}^{-1}(\mathsf{sk}_{\mathsf{GSW}}).$$

Then, set $\mathbf{x}_0 := (\mathsf{Bits}(\mathsf{ct}), \mathbf{z})$ and compute

$$(\mathsf{ek}_0, \mathsf{td}_0) \xleftarrow{\$} \mathsf{RTDH.Gen}(\mathsf{hk}, \mathbf{x}_0, \mathbf{a}_1; \{\mathbf{r}_{0,j}\}_{j\in[d]})$$
$$\forall i > 0: \quad (\mathsf{ek}'_i, \mathsf{td}_i) \xleftarrow{\$} \mathsf{RTDH.Gen}(\mathsf{hk}, \mathbf{0}, \mathbf{a}_{i+1}; \{\mathbf{r}_{i,j}\}_{j\in[d]})$$

Construction 8: (cont.)

For every $i \in [L]$, let $\mathsf{ek}'_i = (C_i, \mathbf{b}_i, \{\mathbf{h}'_{i,j}, E_{i,j}\}_{j\in[d]}, \mathsf{seed}_i)$. Compute

$$\mathbf{y}_i \leftarrow -\mathsf{RTDH.Dec}(\mathsf{hk}, \mathbf{d}, \mathsf{td}_i) \bmod p$$

and set $\overline{\mathsf{ek}}_i := (C_i, \mathbf{b}_i, \{E_{i,j}\}_{j\in[d]}, \mathsf{seed}_i)$. Let n be the digest size of OTE. For every $i \in [L-1]$, take the block of $\mathbf{y}_i$ corresponding to the evaluation of $\hat{f}_i$ and split it into $d \cdot n$ subblocks $\hat{\mathbf{y}}_{i,0}, \ldots, \hat{\mathbf{y}}_{i,n\cdot d-1}$ of dimension $k \cdot \log q$ and set

$$\begin{aligned}\mathbf{y}_{i,j}^{\intercal} &\leftarrow \left(\hat{\mathbf{y}}_{i,j}^{\intercal} \parallel \hat{\mathbf{y}}_{i,d+j}^{\intercal} \parallel \ldots \parallel \hat{\mathbf{y}}_{i,d\cdot(n-1)+j}^{\intercal}\right)\\ \mathbf{w}_{i,j} &\leftarrow \mathbf{h}'_{i+1,j} - \left(\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^{\intercal} \otimes \mathbf{I}_{\log q}\right)\left(\mathbf{y}_{i,j} \otimes \mathbf{g}_q\right) \pmod q.\end{aligned}$$

Let $\mathbf{u}$ be the last element of the standard basis of $\mathbb{Z}_q^k$ and let $\mathbf{y}^*_{L-1}$ be the block of $\mathbf{y}_{L-1}$ corresponding to the output of $\hat{f}_{L-1}$. Compute

$$\mathbf{w}_{L-1} \leftarrow -\mathbf{G}^{-1}(-\Delta\cdot(\mathbf{I}_\ell \otimes \mathbf{u}))^{\intercal}\mathsf{Lin}(\mathbf{I}_{\ell\cdot k^2\cdot\log q})(\mathbf{I}_{\ell\cdot k^2\cdot\log q} \otimes \mathbf{g}_q^{\intercal} \otimes \mathbf{I}_k \otimes \mathbf{g}_q^{\intercal})\,\mathbf{y}^*_{L-1} \pmod q.$$

Sample $\mathsf{seed} \xleftarrow{\$} \{0,1\}^\lambda$ and set $\mathbf{w} \leftarrow \lceil \mathbf{w}_{L-1} + \mathsf{PRG}(\mathsf{seed})\rfloor_2$, parse $\mathsf{ek}_0 = (C_0, \mathbf{b}_0, \{\mathbf{h}_{0,j}, E_{0,j}\}_{j\in[d]}, \mathsf{seed}_0)$ and output

$$E := (\mathsf{ct}, \mathbf{z}, \mathsf{seed}, \mathsf{ek}_0, \{\overline{\mathsf{ek}}_i\}_{i\in[L]}, \{\mathbf{w}_{i,j}\}_{i\in[L-1],j\in[d]}, \mathbf{w}).$$

$\mathsf{Dec}(\mathsf{pk}, E, f, \psi)$**:** Initially, set $\mathbf{x}_0 \leftarrow (\mathsf{Bits}(\mathsf{ct}), \mathbf{z})$. Then, for $i = 0, \ldots, L-2$:

- compute $\tilde{\mathbf{y}}_i \leftarrow \mathsf{RTDH.Enc}(\mathsf{hk}, \mathsf{ek}_i, \psi, \mathbf{x}_i, \hat{f})$
- take the block of $\tilde{\mathbf{y}}_i$ corresponding to the evaluation of $\hat{f}_i$ and split it into $d \cdot n$ subblocks $\overline{\mathbf{y}}_{i,0}, \ldots, \overline{\mathbf{y}}_{i,n\cdot d-1}$ of dimension $k \cdot \log q$ and set

$$\begin{aligned}\tilde{\mathbf{y}}_{i,j}^{\intercal} &\leftarrow \left(\overline{\mathbf{y}}_{i,j}^{\intercal} \parallel \overline{\mathbf{y}}_{i,d+j}^{\intercal} \parallel \ldots \parallel \overline{\mathbf{y}}_{i,d\cdot(n-1)+j}^{\intercal}\right)\\ \mathbf{h}_{i+1,j} &\leftarrow \mathbf{w}_{i,j} + \left(\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^{\intercal} \otimes \mathbf{I}_{\log q}\right)\left(\tilde{\mathbf{y}}_{i,j} \otimes \mathbf{g}_q\right) \pmod q.\end{aligned}$$

- set $\mathsf{ek}_{i+1} \leftarrow (\overline{\mathsf{ek}}_{i+1}, \{\mathbf{h}_{i+1,j}\}_{j\in[d]})$
- compute the input to the next RMS program $\mathbf{x}_{i+1} \leftarrow f'_i(\mathbf{x}_i)$

Finally, let $\mathbf{u}$ be the last element of the standard basis of $\mathbb{Z}_q^k$. Derive

$$\begin{aligned}&\tilde{\mathbf{y}}_{L-1} \leftarrow \mathsf{RTDH.Enc}(\mathsf{hk}, \mathsf{ek}_{L-1}, \psi, \mathbf{x}_{L-1}, \hat{f})\\ &\tilde{\mathbf{w}}_{L-1} \leftarrow \mathbf{G}^{-1}(-\Delta\cdot(\mathbf{I}_\ell \otimes \mathbf{u}))^{\intercal}\mathsf{Lin}(\mathbf{I}_{\ell\cdot k^2\cdot\log q})(\mathbf{I}_{\ell\cdot k^2\cdot\log q} \otimes \mathbf{g}_q^{\intercal} \otimes \mathbf{I}_k \otimes \mathbf{g}_q^{\intercal})\,\overline{\mathbf{y}}^*_{L-1} \pmod q.\end{aligned}$$

where $\overline{\mathbf{y}}^*_{L-1}$ denotes the block of $\tilde{\mathbf{y}}_{L-1}$ corresponding to the evaluation of $\hat{f}_{L-1}$.

Output $\lceil \tilde{\mathbf{w}}_{L-1} - \mathsf{PRG}(\mathsf{seed})\rfloor_2 \oplus \mathbf{w}$

Proof. We proceed by induction over i. The claim is trivially true for $i = 0$. Now, we show that if it holds for i, it holds also for $i + 1$.

We observe that

$$\begin{aligned}\mathbf{h}_{i+1,j} &= \mathbf{w}_{i,j} + (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})(\tilde{\mathbf{y}}_{i,j} \otimes \mathbf{g}_q) \pmod q \\ &= \mathbf{h}'_{i+1,j} - (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})(\mathbf{y}_{i,j} \otimes \mathbf{g}_q) + (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})(\tilde{\mathbf{y}}_{i,j} \otimes \mathbf{g}_q) \pmod q \\ &= \mathbf{h}'_{i+1,j} + (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})\big((\tilde{\mathbf{y}}_{i,j} - \mathbf{y}_{i,j}) \otimes \mathbf{g}_q\big) \pmod q.\end{aligned}$$

Now, by the correctness of RTDH and the inductive hypothesis, we recall that $\tilde{\mathbf{y}}_i - \mathbf{y}_i = \hat{f}(\mathbf{x}_i) \otimes \mathbf{a}_{i+1} \pmod p$. This holds with overwhelming probability, irrespectively of the inputs chosen by the adversary. We also recall that this subtractive secret-sharing was initially over $\mathbb{Z}_q$, i.e. the parties held vectors $\tilde{\mathbf{u}}_i$ and $\mathbf{u}_i$ such that $\tilde{\mathbf{u}}_i - \mathbf{u}_i = \frac{q}{p} \cdot \big(\hat{f}(\mathbf{x}_i) \otimes \mathbf{a}_{i+1}\big) + \boldsymbol{\varepsilon}_i$ where $\|\boldsymbol{\varepsilon}_i\|_\infty$ is low. This secret-sharing was rerandomised using $\mathsf{PRG}(\mathsf{seed}_i)$ and then rounded. We argue that $\mathbf{y}_i$ is pseudorandom. In particular, since $p = 2^{\omega(\log \lambda)}$, with overwhelming probability over the choice of seed_i, all entries of $\mathbf{y}_i$ are in the interval $[-p/2, p/2 - m^r)$. Since $\hat{f}(\mathbf{x}_i) \otimes \mathbf{a}_{i+1}$ has all ℓ_∞-norm at most m^r, we conclude that (with overwhelming probability over the choice of seed_i) $\tilde{\mathbf{y}}_i$ and $\mathbf{y}_i$ are a subtractive secret-sharing of $\hat{f}(\mathbf{x}_i) \otimes \mathbf{a}_{i+1}$ even over the integers. Continuing, we observe that the i-th n-entry block of $\hat{f}(\mathbf{x}_i)$ coincides with $\mathbf{d}'_{i+1}$ where $\mathbf{d}'_{i+1}$ is the output of $\mathsf{OTE.Hash}(\mathsf{tpk}, f'_i(\mathbf{x}_i)) = \mathsf{OTE.Hash}(\mathsf{tpk}, \mathbf{x}_{i+1})$. Moreover, we observe that, by the way we constructed $\mathbf{a}_{i+1}$, $\tilde{\mathbf{y}}_{i,j}$ and $\mathbf{y}_{i,j}$, we have that

$$\tilde{\mathbf{y}}_{i,j} - \mathbf{y}_{i,j} = \mathbf{d}'_{i+1} \otimes \mathbf{G}^{-1}(\mathbf{r}_{i+1,j}).$$

Putting everything together, we obtain

$$\begin{aligned}\mathbf{h}_{i+1,j} &= \mathbf{h}'_{i+1,j} + (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})\big((\tilde{\mathbf{y}}_{i,j} - \mathbf{y}_{i,j}) \otimes \mathbf{g}_q\big) \\ &= \mathbf{h}'_{i+1,j} + (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})\big((\mathbf{d}'_{i+1} \otimes \mathbf{G}^{-1}(\mathbf{r}_{i+1,j})) \otimes \mathbf{g}_q\big) \\ &= \mathbf{h}'_{i+1,j} + (\mathbf{d}'_{i+1} \otimes \mathbf{G}^\intercal)\mathbf{r}_{i+1,j}. \qquad (8)\end{aligned}$$

Since $\mathbf{h}'_{i+1,j} = \mathbf{A}^\intercal \mathbf{s}_{i+1,j} + \mathbf{e}_{i+1}$ where $\|\mathbf{e}_{i+1}\|_\infty \leq B(\lambda)$, we obtain that

$$\mathbf{h}_{i+1,j} = \mathbf{A}^\intercal \mathbf{s}_{i+1,j} + (\mathbf{d}'_{i+1} \otimes \mathbf{G}^\intercal)\mathbf{r}_{i+1,j} + \mathbf{e}_{i+1}.$$

This ends the proof of the lemma. □

Continuing with our analysis, by Lemma 32, we have that $\tilde{\mathbf{y}}_{L-1}$ and $\mathbf{y}_{L-1}$ form a subtractive secret sharing of $\hat{f}(\mathbf{x}_{L-1}) \otimes \mathbf{a}_L$ over $\mathbb{Z}_p$. This holds with overwhelming probability, irrespectively of the inputs chosen by the adversary. Furthermore, similarly to how we argued in the proof of Lemma 32, it is possible to prove that the probability that any entry of $\mathbf{y}_{L-1}$ lies outside of $[-p/2, p/2-1)$ is negligible (irrespectively of the inputs chosen by the adversary). Given that all entries of $\hat{f}(\mathbf{x}_{L-1}) \otimes \mathbf{a}_L$ belong to $\{0, 1\}$, we conclude that, with overwhelming probability, $\tilde{\mathbf{y}}_{L-1}$ and $\mathbf{y}_{L-1}$ form a subtractive secret sharing of $\hat{f}(\mathbf{x}_{L-1}) \otimes \mathbf{a}_L$ even over $\mathbb{Z}$. By the way we constructed $\hat{f}, \overline{\mathbf{y}}^*_{L-1}$ and $\mathbf{y}^*_{L-1}$, we infer that

$$\overline{\mathbf{y}}^*_{L-1} - \mathbf{y}^*_{L-1} = \hat{f}_{L-1}(\mathbf{x}_{L-1}) \otimes \mathbf{a}_L.$$

We also notice that $\mathbf{a}_L = \mathbf{G}^{-1}(\mathsf{sk}_{\mathsf{GSW}})$, whereas

$$\hat{f}_{L-1}(\mathbf{x}_{L-1}) = f'_{L-1}(\mathbf{x}_{L-1}) = (f'_{L-1} \circ \cdots \circ f'_0)(\mathbf{x}_0) = \mathsf{Bits}(f'(\mathsf{ct}, \mathbf{z})) = \mathsf{Bits}(\mathsf{vec}(\mathsf{GSW.Eval}(\mathsf{ct}, f_{\mathbf{z}}))).$$

Let $\hat{\mathsf{ct}} := \mathsf{GSW.Eval}(\mathsf{ct}, f_{\mathbf{z}})$. We obtain that

$$\begin{aligned}
\tilde{\mathbf{w}}_{L-1} + \mathbf{w}_{L-1} &= \mathsf{G}^{-1}(-\Delta \cdot (\mathbf{I}_\ell \otimes \mathbf{u}))^\intercal \mathsf{Lin}(\mathbf{I}_{\ell \cdot k^2 \cdot \log q})(\mathbf{I}_{\ell \cdot k^2 \cdot \log q} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_k \otimes \mathbf{g}_q^\intercal)\left(\overline{\mathbf{y}}^*_{L-1} - \mathbf{y}^*_{L-1}\right) \\
&= \mathsf{G}^{-1}(-\Delta \cdot (\mathbf{I}_\ell \otimes \mathbf{u}))^\intercal \mathsf{Lin}(\mathbf{I}_{\ell \cdot k^2 \cdot \log q})(\mathbf{I}_{\ell \cdot k^2 \cdot \log q} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_k \otimes \mathbf{g}_q^\intercal)\left(\hat{f}_{L-1}(\mathbf{x}_{L-1}) \otimes \mathbf{a}_L\right) \\
&= \mathsf{G}^{-1}(-\Delta \cdot (\mathbf{I}_\ell \otimes \mathbf{u}))^\intercal (\mathbf{I}_{\ell \cdot k \cdot \log q} \otimes \mathsf{sk}_{\mathsf{GSW}}^\intercal)\, \mathsf{vec}(\hat{\mathsf{ct}}).
\end{aligned}$$

By the correctness of GSW evaluation, we have that

$$(\mathbf{I}_{\ell \cdot k \cdot \log q} \otimes \mathsf{sk}_{\mathsf{GSW}}^\intercal)\, \mathsf{vec}(\hat{\mathsf{ct}}) = \Delta \cdot f(\mathbf{x}) + \widehat{\mathbf{e}}$$

for some error vector $\widehat{\mathbf{e}}$ such that $\|\widehat{\mathbf{e}}\|_\infty \leq B \cdot D \cdot \mathsf{poly}(\lambda)$. Notice that $f_{\mathbf{z}}(K) = f(\mathbf{z} \oplus \mathsf{PRG}(K)) = f(\mathbf{x})$. We conclude that

$$\mathbf{w}'_{L-1} + \mathbf{w}_{L-1} = \Delta \cdot f(\mathbf{x}) + \widehat{\mathbf{e}}. \tag{9}$$

So, unless any of the entries of $\mathbf{w}_{L-1} + \mathsf{PRG}(\mathsf{seed})$ is less than $\|\widehat{\mathbf{e}}\|_\infty$ away from $q/4$ or $-q/4$, we have that

$$f(\mathbf{x}) = \lceil \mathbf{w}'_{L-1} - \mathsf{PRG}(\mathsf{seed}) \rfloor_2 \oplus \lceil \mathbf{w}_{L-1} + \mathsf{PRG}(\mathsf{seed}) \rfloor_2.$$

Since seed is sampled independently of $\mathbf{w}_{L-1}$, the probability of the bad event is at most $\mathsf{poly}(\lambda) \cdot \|\widehat{\mathbf{e}}\|_\infty / q) + \mathsf{negl}(\lambda)$. This holds irrespectively of the inputs chosen by the adversary. Given our choice of q for RTDH, this is a negligible amount.

Security. Next, we prove that our construction is encoder private.

Theorem 33. *Assume the hardness of LWE with superpolynomial modulus-noise ratio. Then, Construction 8 is an adaptively encoder private LFE.*

Proof. We prove our claim by relying on a series of indistinguishable hybrids.

- Hybrid $\mathcal{H}_0$: This hybrid corresponds to the original game: we provide the adversary with a tuple $(\mathsf{ct}, \mathbf{z}, \mathsf{seed}, \{\mathbf{h}_{0,j}\}_{j \in [d]}, \{\overline{\mathsf{ek}}_i\}_{i \in [L]}, \{\mathbf{w}_{i,j}\}_{i \in [L-1], j \in [d]}, \mathbf{w})$ generated using $\mathsf{LFE.Enc}(\mathsf{pk}, \mathbf{d}, \mathbf{x})$.
- Hybrid $\mathcal{H}_1$: In this hybrid, we change the distribution of $\mathbf{w}$: using pk, f, $(\mathsf{ct}, \mathbf{z}, \mathsf{seed}, \{\mathbf{h}_{0,j}\}_{j \in [d]}, \{\overline{\mathsf{ek}}_i\}_{i \in [L]}, \{\mathbf{w}_{i,j}\}_{i \in [L-1], j \in [d]})$ and following the same operations as in the decoding procedure, we compute $\mathbf{w}'_{L-1}$. Then, we set $\mathbf{w} \leftarrow f(\mathbf{x}) \oplus \lceil \mathbf{w}'_{L-1} - \mathsf{PRG}(\mathsf{seed}) \rfloor_2$.

  This hybrid is statistically indistinguishable from Hybrid $\mathcal{H}_0$ due to the correctness of the primitive.

- Hybrid $\mathcal{H}_2$: In this hybrid, we change the distribution of $\mathbf{w}_{i,j}$ for every $i \in [L-1]$ and $j \in [d]$: using $\mathsf{pk}, f, (\mathsf{ct}, \mathbf{z}, \{\mathbf{h}_{\mathbf{0},\beta}\}_{\beta\in[d]}, \{\overline{\mathsf{ek}}_\gamma\}_{\gamma\in[i]}, \{\mathbf{w}_{\gamma,\beta}\}_{\gamma\in[i],\beta\in[d]})$ and following the same operations as in the decoding procedure, we compute $\mathbf{y}'_{i,j}$. Then, we set

$$\mathbf{w}_{i,j} \leftarrow \mathbf{h}'_{i+1,j} + (\mathbf{d}'_{i+1} \otimes \mathbf{G}^\intercal)\mathbf{r}_{i+1,j} - (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})(\tilde{\mathbf{y}}_{i,j} \otimes \mathbf{g}_q) \quad (10)$$

  where $(\mathbf{d}'_{i+1}, \psi'_{i+1}) \leftarrow \mathsf{OTE.Hash}(\mathsf{tpk}, \mathbf{x}_{i+1})$ and $\mathbf{x}_{i+1} \leftarrow (f'_i \circ \cdots \circ f'_0)(\mathsf{Bits}(\mathsf{ct}), \mathbf{z})$.

  This hybrid is statistically indistinguishable from Hybrid $\mathcal{H}_1$. Indeed, $\mathbf{w}_{i,j}$ satisfies the relation in (10) even in $\mathcal{H}_1$. This is highlighted in Lemma 32, specifically in (8).

- Hybrid $\mathcal{H}_3$: In this hybrid, for every $i \in [L] \setminus \{0\}$, we compute

$$(\mathsf{ek}_i, \mathsf{td}_i) \xleftarrow{\$} \mathsf{RTDH.Gen}(\mathsf{hk}, \mathbf{x}_{i+1}, \mathbf{a}_{i+1}; \{\mathbf{r}_{i,j}\}_{j\in[d]})$$

  where $\mathbf{x}_{i+1} \leftarrow (f'_i \circ \cdots \circ f'_0)(\mathsf{Bits}(\mathsf{ct}), \mathbf{z})$. Let $\mathsf{ek}_i = (C_i, \mathbf{b}_i, \{\mathbf{h}_{i,j}, E_{i,j}\}_{j\in[d]}, \mathsf{seed}_i)$. For every $i \in [L-1]$ and $j \in [d]$, we set

$$\mathbf{w}_{i,j} \leftarrow \mathbf{h}_{i+1,j} - (\mathbf{I}_{n\cdot k} \otimes \mathbf{g}_q^\intercal \otimes \mathbf{I}_{\log q})(\tilde{\mathbf{y}}_{i,j} \otimes \mathbf{g}_q).$$

  We also argue that $\mathcal{H}_2$ is perfectly indistinguishable from Hybrid $\mathcal{H}_3$. Indeed, the only difference between the output of $\mathsf{RTDH.Gen}(\mathsf{hk}, \mathbf{x}_{i+1}, \mathbf{a}_{i+1}; \{\mathbf{r}_{i,j}\}_{j\in[d]})$ and $\mathsf{RTDH.Gen}(\mathsf{hk}, \mathbf{0}, \mathbf{a}_{i+1}; \{\mathbf{r}_{i,j}\}_{j\in[d]})$ is that, in the first case, the encoding $\mathbf{h}_{i,j}$ is "shifted" by $(\mathbf{d}'_{i+1} \otimes \mathbf{G}^\intercal)\mathbf{r}_{i+1,j}$ where $(\mathbf{d}'_{i+1}, \psi'_{i+1}) \leftarrow \mathsf{OTE.Hash}(\mathsf{tpk}, \mathbf{x}_{i+1})$ (see Construction 5). In the second case, no shift is essentially applied. This is because, by the linearity of our OTE hashing (see Construction 1 and Construction 2), $\mathsf{OTE.Hash}(\mathsf{tpk}, \mathbf{0})$ outputs $\mathbf{0}$.

- Hybrid $\mathcal{H}_4^\iota$: In this hybrid, for every $i < \iota$, we generate

$$\mathsf{ek}_i := (C_i, \mathbf{b}_i, \{\mathbf{h}_{i,j}, E_{i,j}\}_{j\in[d]}, \mathsf{seed}_i) \xleftarrow{\$} \mathsf{RTDH.Sim}(1^\lambda, \mathsf{hk})$$

  We observe that Hybrid $\mathcal{H}_3$ is identical to $\mathcal{H}_4^0$. Moreover, by Theorem 31, for every $\iota \in [L-1]$, we have that $\mathcal{H}_4^\iota$ is computationally indistinguishable from $\mathcal{H}_4^{\iota+1}$. Notice that here we are implicitly relying on the fact that, in Hybrid $\mathcal{H}_4^\iota$, the tuple $\{\mathsf{ek}_i\}_{i<\iota}$ no longer contains information about $\{\mathbf{r}_{\iota,j}\}_{j\in[d]}$.

- Hybrid $\mathcal{H}_5$: In this hybrid, we sample $\mathsf{pk}_{\mathsf{GSW}} \xleftarrow{\$} \mathbb{Z}_q^{k\times(k\cdot\log q+\lambda)}$.

  We argue that $\mathcal{H}_4^L$ is computationally indistinguishable from $\mathcal{H}_5$. Indeed, under LWE, the term $\mathbf{M}^\intercal\mathbf{r} + \mathbf{e}$ is indistinguishable from random. Here, we are implicitly relying on the fact that, in $\mathcal{H}_4^L$, the tuple $\{\mathsf{ek}_i\}_{i\in[L]}$ no longer contains information about $\mathbf{r}$.

- Hybrid $\mathcal{H}_6$: In this hybrid, we generate $\mathsf{ct} \xleftarrow{\$} \mathbb{Z}_q^{k\times(\lambda\cdot k\cdot\log q)}$.

  Hybrid $\mathcal{H}_6$ is statistically indistinguishable from $\mathcal{H}_5$. Indeed, now $\mathsf{pk}_{\mathsf{GSW}}$ is a uniformly random matrix, so we can apply the leftover hash lemma to argue that ct is indistinguishable from random.

- Hybrid $\mathcal{H}_7$: In this hybrid, we sample $\mathbf{z} \xleftarrow{\$} \mathbb{Z}_2^m$.

  Since ct no longer contains information about the PRG seed K, we can argue that $\mathcal{H}_6$ is computationally indistinguishable from $\mathcal{H}_7$ thanks to the security of the PRG.

Notice that in $\mathcal{H}_7$ all the material provided to the adversary can be computed by a simulator with no information about $\mathbf{x}$ except $f(\mathbf{x})$. This ends the proof of security. □

Parameters. Construction 8 uses a single input-succinct reverse-TDH hash, and therefore its public key and digest size are

$$\mathsf{poly}(\lambda, \log m, \log \ell, \log D).$$

Moreover, the encoding consists of the masked input, the final output share, and $L = \lceil D/\log d \rceil$ auxiliary encodings. Hence the total encoding size is

$$m + \ell + D \cdot \mathsf{poly}(\lambda, \log m, \log \ell, \log D).$$

Thus, we obtain the following theorem.

Theorem 34 (Rate-1 Adaptively Secure LFE)**.** *Assume the hardness of LWE, there exists an adaptively secure LFE for depth-D functions $f : \{0,1\}^m \to \{0,1\}^\ell$ with public key and digest size $\mathsf{poly}(\lambda, \log m, \log \ell, \log D)$ and encoding size $m + \ell + D \cdot \mathsf{poly}(\lambda, \log m, \log \ell, \log D)$.*

7 A Counterexample for Adaptive LWE

We present a counterexample to a conjecture from [21]. The conjecture (adaptive LWE) says that the probability of that any polynomial-time attacker wins the following experiment is negligibly close to 1/2.

- The challenger samples random matrices $\{\mathbf{A}_i \xleftarrow{\$} \mathbb{Z}_q^{n\times m}\}_{i\in[k]}$ and sends them to the attacker.
- The attacker chooses an $x \in \{0,1\}^{k-1}$ and sends it to the challenger. Let $\hat{x} := (x, 0)$.
- The challenger samples an $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$ and a bit $b \xleftarrow{\$} \{0,1\}$.
- If $b = 0$ it computes:

  $$\{\mathbf{b}_i^\intercal := \mathbf{s}^\intercal(\mathbf{A}_i - \hat{x}_i \cdot \mathbf{G}) + \mathbf{e}_i^\intercal\}_{i\in[k]}$$

  where $\mathbf{e}_i \xleftarrow{\$} \chi(\lambda)$.

- If $b = 1$ it samples $\{\mathbf{b}_i \xleftarrow{\$} \mathbb{Z}_q^m\}_{i\in[k]}$.
- The attacker wins if, given $\{\mathbf{b}_i\}_{i\in[k]}$, it correctly guesses b.

We show that, if k is allowed to grow independently of the LWE parameters (n, q, χ), the assumption is false. This roughly corresponds to the *optimistic parameter* settings proposed in [21]. We sketch the attack in the following. We assume for convenience that q is a power of 2, but the attack can be adapted to other moduli.

Let $\mathbf{A} := (\mathbf{A}_0 \parallel \ldots \parallel \mathbf{A}_{k-1})$. By the homomorphic properties of the lattice encodings, we know that there exists a matrix $\mathbf{H}$ with $\|\mathbf{H}\|_\infty = 1$ such that, for any matrix $\mathbf{M} \in \mathbb{Z}_q^{n\times n}$, if $\mathbf{x} := \mathsf{Bits}(\mathbf{M})$, we have that:

$$(\mathbf{s}^\intercal(\mathbf{A} - \mathbf{x} \otimes \mathbf{G}) + \mathbf{e}^\intercal) \cdot \mathbf{H} = \mathbf{s}^\intercal(\mathbf{A} \cdot \mathbf{H} - \mathbf{M}) + \mathbf{e}^\intercal \cdot \mathbf{H} \approx \mathbf{s}^\intercal(\mathbf{A} \cdot \mathbf{H} - \mathbf{M})$$

since $\|\mathbf{e}^\intercal\mathbf{H}\|_\infty \approx 0$. To recover the i-th most significant bit of (each component of) $\mathbf{s}$, it is sufficient to set $\mathbf{M}_i := \mathbf{A} \cdot \mathbf{H} - 2^i \cdot \mathbf{I}_n$ and $\mathbf{x}_i := \mathsf{Bits}(\mathbf{M}_i)$. Then compute the (component-wise) most significant bit of:

$$(\mathbf{s}^\intercal(\mathbf{A} - \mathbf{x}_i \otimes \mathbf{G}) + \mathbf{e}^\intercal) \cdot \mathbf{H} \approx \mathbf{s}^\intercal(\mathbf{A} \cdot \mathbf{H} - \mathbf{M}_i) = 2^i \cdot \mathbf{s}^\intercal.$$

We can then recover the entire secret key $\mathbf{s}$, by setting the input $\mathbf{x}$ to be the concatenation of $(\mathbf{x}_1, \ldots, \mathbf{x}_{\log q})$. This shows that the conjecture is false if k is allowed to be arbitrarily bigger than n. Notice in the provable parameter setting (where the encodings are secure under the *subexponential* hardness of LWE), our attack fails as the bound on k is too small to encode $\mathbf{M}_i := \mathbf{A} \cdot \mathbf{H} - 2^i \cdot \mathbf{I}_n$.

References

[1] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 421–452. Springer, Cham, Switzerland, 2022. doi: 10.1007/978-3-031-15985-5_15.

[2] Damiano Abram, Peter Scholl, and Sophia Yakoubov. Distributed (correlation) samplers: How to remove a trusted dealer in one round. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 790–820. Springer, Cham, Switzerland, 2022. doi: 10.1007/978-3-031-06944-4_27.

[3] Damiano Abram, Lawrence Roy, and Peter Scholl. Succinct homomorphic secret sharing. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part VI*, volume 14656 of *Lecture Notes in Computer Science*, pages 301–330. Springer, Cham, Switzerland, 2024. doi: 10.1007/978-3-031-58751-1_11.

[4] David A. Mix Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1. *Journal of Computer and System Sciences*, 38(1):150–164, 1989. doi: 10.1016/0022-0000(89)90037-8.

[5] Rishabh Bhadauria, Pedro Branco, and Nico Döttling. Rate-1 registration-based encryption and laconic oblivious transfer. (Personal Communication).

[6] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 410–428. Springer Berlin Heidelberg, Germany, 2013. doi: 10.1007/978-3-642-40041-4_23.

[7] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 533–556. Springer Berlin Heidelberg, Germany, 2014. doi: 10.1007/978-3-642-55220-5_30.

[8] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part I*, volume

9814 of *Lecture Notes in Computer Science*, pages 509–539. Springer Berlin Heidelberg, Germany, 2016. doi: 10.1007/978-3-662-53018-4_19.

[9] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 489–518. Springer, Cham, Switzerland, 2019. doi: 10.1007/978-3-030-26954-8_16.

[10] Elette Boyle, Abhishek Jain, Sacha Servan-Schreiber, and Akshayaram Srinivasan. Simultaneous-message and succinct secure computation. In *Advances in Cryptology - EUROCRYPT 2025, Part V*, volume 15605 of *Lecture Notes in Computer Science*, pages 207–239. Springer, 2025. doi: 10.1007/978-3-031-91092-0_8.

[11] Zvika Brakerski and Vinod Vaikuntanathan. Lattice-based FHE as secure as PKE. In Moni Naor, editor, *ITCS 2014: 5th Conference on Innovations in Theoretical Computer Science*, pages 1–12. Association for Computing Machinery, 2014. doi: 10.1145/2554797.2554799.

[12] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. Private constrained PRFs (and more) from LWE. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 264–302. Springer, Cham, Switzerland, 2017. doi: 10.1007/978-3-319-70500-2_10.

[13] Zvika Brakerski, Nico Döttling, Sanjam Garg, and Giulio Malavolta. Leveraging linear decryption: Rate-1 fully-homomorphic encryption and time-lock puzzles. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019: 17th Theory of Cryptography Conference, Part II*, volume 11892 of *Lecture Notes in Computer Science*, pages 407–437. Springer, Cham, Switzerland, 2019. doi: 10.1007/978-3-030-36033-7_16.

[14] Chongwon Cho, Nico Döttling, Sanjam Garg, Divya Gupta, Peihan Miao, and Antigoni Polychroniadou. Laconic oblivious transfer and its applications. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part II*, volume 10402 of *Lecture Notes in Computer Science*, pages 33–65. Springer, Cham, Switzerland, 2017. doi: 10.1007/978-3-319-63715-0_2.

[15] Yevgeniy Dodis, Shai Halevi, Ron D. Rothblum, and Daniel Wichs. Spooky encryption and its applications. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 93–122. Springer Berlin Heidelberg, Germany, 2016. doi: 10.1007/978-3-662-53015-3_4.

[16] Fangqi Dong, Zihan Hao, Ethan Mook, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and ABE for RAMs from (ring-)LWE. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part III*, volume 14922 of *Lecture Notes in Computer Science*, pages 107–142. Springer, Cham, Switzerland, 2024. doi: 10.1007/978-3-031-68382-4_4.

[17] Nico Döttling, Sanjam Garg, Yuval Ishai, Giulio Malavolta, Tamer Mour, and Rafail Ostrovsky. Trapdoor hash functions and their applications. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 3–32. Springer, Cham, Switzerland, 2019. doi: 10.1007/978-3-030-26954-8_1.

[18] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92. Springer Berlin Heidelberg, Germany, 2013. doi: 10.1007/978-3-642-40041-4_5.

[19] Yao-Ching Hsieh, Huijia Lin, and Ji Luo. Attribute-based encryption for circuits of unbounded depth from lattices: Garbled circuits of optimal size, laconic functional evaluation, and more. *SIAM Journal on Computing*, 55 (3):FOCS23–90–FOCS23–146, 2026. doi: 10.1137/24M1629316.

[20] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 678–708. Springer, Cham, Switzerland, 2021. doi: 10.1007/978-3-030-77870-5_24.

[21] Willy Quach, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and applications. In *59th Annual Symposium on Foundations of Computer Science (FOCS 2018)*, pages 859–870. IEEE Computer Society Press, 2018. doi: 10.1109/FOCS.2018.00086.

[22] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):34:1–34:40, 2009. doi: 10.1145/1568318.1568324.

[23] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 687–717. Springer, Cham, Switzerland, 2021. doi: 10.1007/978-3-030-84252-9_23.

[24] Hoeteck Wee. Circuit ABE with $\mathsf{poly}(\mathrm{depth}, \lambda)$-sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part III*, volume 14922 of *Lecture Notes in Computer Science*, pages 178–209. Springer, Cham, Switzerland, 2024. doi: 10.1007/978-3-031-68382-4_6.